C++ тілінде машинаға бағытталған бағдарламалау

Электрондық оқулық




```
#include <cstdint>
volatile std::uint32_t* const REG =
(0x4000000);

void write(std::uint32_t value) {
    *REG = value;
}

std::uint32_t read() {
    return *REG;
}
```

```
mov   eax, [0x4000000]
add   eax, 1
mov   [0x4000000], eax

mov   r0, #0x1
str   r0, [r1]
ldr   r2, [r1]
```

НАЙЗАҒАРАЕВА А.А.

С++ ТІЛІНДЕ МАШИНАҒА БАҒЫТТАЛҒАН БАҒДАРЛАМАЛАУ
ЭЛЕКТРОНДЫҚ ОҚУЛЫҚ

АСТАНА 2026

АЛҒЫ СӨЗ

«C++ тілінде машинаға бағытталған бағдарламалау» электрондық оқулық білім алушылардың C++ тілінде бағдарламалар құру, деректерді өңдеу және компьютер жадымен жұмыс істеу негіздерін меңгеруіне арналған. Оқулық бағдарламалау тілінің мүмкіндіктерін компьютерде орындалатын әрекеттермен байланыстыра түсіндіруге және теориялық білімді практикалық тапсырмалар арқылы бекітуге бағытталған.

Машинаға бағытталған бағдарламалауды меңгеру деректердің жадыда орналасуын, адрестер мен нұсқауыштардың қызметін, ресурстарды басқару тәсілдерін түсінуді қажет етеді. C++ тілі осы ұғымдарды зерттеуге мүмкіндік береді. Сонымен қатар тілдің кластар, мұрагерлік, полиморфизм және үлгілер сияқты құралдары төменгі деңгейдегі әрекеттерді реттелген бағдарламалық құрылымдар арқылы ұйымдастыруға көмектеседі.

Оқулықтың мақсаты — білім алушыларға C++ тілінің негізгі құралдарын жүйелі түрде таныстыру, бағдарламалық есептерді талдау және оларды шешуге арналған алгоритмдер мен бағдарламалар әзірлеу дағдыларын қалыптастыру. Материалдарды меңгеру барысында деректерді дұрыс сипаттауға, функцияларды қолдануға, объектілердің өмірлік циклін түсінуге және бағдарламаның орындалу нәтижелерін тексеруге назар аударылады.

Оқулықта сілтемелер, кластар мен объектілер, конструкторлар мен деструкторлар, достық функциялар, операторларды қайта жүктеу және үлгілер қарастырылады. Мұрагерлікке, кластар иерархиясына, виртуалды әдістер мен абстрактылы кластарға арналған тақырыптар бағдарламалық құрылымдарды жобалау тәсілдерін түсіндіреді. Атаулар кеңістіктері, файлдық енгізу-шығару, ерекше жағдайларды өңдеу, типтерді анықтау мен түрлендіру және стандартты кітапхана құралдары бағдарламалау білімін толықтырады.

Зертханалық жұмыстар енгізу-шығару операторларын, шарттар мен циклдерді, массивтерді, функцияларды, нұсқауыштарды және деректер құрылымдарын қолдануға арналған. Тапсырмаларды орындау кезінде білім алушы бастапқы деректерді таңдап, бағдарлама құрады, нәтижелерді тексереді және қателердің себептерін талдайды. Бақылау сұрақтары мен тест тапсырмалары негізгі ұғымдарды қайталауға және меңгерілген білімді бағалауға мүмкіндік береді.

Электрондық оқулық аудиториялық сабақтарда, зертханалық жұмыстарды орындауда және өздігінен білім алуда пайдалануға болады. Материалды тиімді меңгеру үшін тақырыптарды ретімен оқып, бағдарламалық мысалдарды орындау және оларды әртүрлі деректермен тексеру ұсынылады.

Оқулық ақпараттық технологиялар бағытында білім алатын студенттерге және C++ тілінде бағдарламалауды үйренушілерге арналған. Ұсынылған материалдар алгоритмдік ойлауды дамытуға, компьютер ресурстарымен саналы жұмыс істеуге және түсінікті әрі сенімді бағдарламалық код жазуға негіз болады.

Құрастырушы: Найзағараева А.А. С.Сейфуллин атындағы Қазақ агротехникалық зерттеу университеті, «Ақпараттық жүйелер» білім беру бағдарламалары тобының аға оқытушысы, PhD.

МАЗМҰНЫ

Алғы сөз

1. Дәрістер жинағы
1.1. Объектіге бағытталған программалау технологиясы. C++ тілі
1.2. Деректер типінің сілтемелері
1.3. Класс абстрактылы тип ретінде
1.4. Кластар достары. Операторларды қайта жүктеу
1.5. Функция және класс үлгілері
1.6. Мұрагерлік
1.7. Кластар иерархиясы
1.8. Полиморфизм
1.9. Абстрактылы кластар
1.10. Көріну аймақтары мен атаулар кеңістіктері
1.11. Стандартты кітапхана құралдары
1.12. Файлдық енгізу-шығару
1.13. Ерекше жағдайларды өңдеу
1.14. Динамикалық типті анықтау және типтерді түрлендіру
1.15. C++ стандартты кітапханасы. Жалпыланған программалау

2. Зертханалық жұмыстар
2.1. Қарапайым есептер. Деректерді енгізу және шығару
2.2. Шартты операторларды қолдану
2.3. Көпнұсқалы таңдау. `switch` операторы
2.4. Цикл операторларын қолдану
2.5. Мәтіндік ақпаратпен және массивтермен жұмыс
2.6. Пайдаланушы функцияларын құру
2.7. Рекурсивті функциялар
2.8. Функциялар және нұсқауыштар
2.9. Құрылымдарды анықтау және қолдану
2.10. Деректерді сұрыптау және іздеу
2.11. Динамикалық деректер құрылымдарын программалау
2.12. Графикалық құрастырулар және деректер құрылымдарын өңдеу
2.13. Ерекше жағдайларды өңдеу
2.14. Динамикалық типті анықтау және типтерді түрлендіру
2.15. Стандартты кітапхана және жалпыланған программалау

3. Білімді бақылау материалдары
3.1. Тест тапсырмалары — 100 сұрақ
3.2. Тест тапсырмаларының жауап кілті

Қорытынды
Пайдаланылған әдебиеттер тізімі

Дәріс №1. Объектіге бағытталған программалау технологиясы. C++ тілі

Дәрістің мақсаты: объектіге бағытталған программалау технологиясы түсініктерімен таныстыру.

Дәрістің мазмұны: класс ұғымы және оның элементтері. C++ тілі ортасының артықшылықтары.

Программалаушылар көптеген жылдар бойы бизнесті жүргізу әдістерін автоматтандыратын программалаларды жазып үлкен жұмыс атқарды. Бизнесте программалар – бизнес операцияларына қызмет етуі талап етеді. Программалаушыларға көмекке қайта пайдаланылатын кодтар (reusable code) есебінен, жұмысты жеңілдететін процедуралық программалау келді. Процедуралық программалау программалар жазуда үлкен өзгеріс болды, бірақ объектіге бағытталған әлемді қайта құру мәселесі шешілмеген күйінде қалды. Бұл мәселе 1980 жылы объектіге бағытталған программалаудың (ОБП) пайда болуымен шешілді. Нақты әлемнің объектілеріне еліктеуді C++ немесе Java сияқты объектіге бағытталған программалау тілдерінде кластарды анықтау арқылы жүзеге асыруға болады. Класс деректер мүшелері мен әдістер мүшелерінен тұрады. Деректер мүшелері (*data members*) кейде өрістер (*fields*) деп аталады және объект атрибуттарын сақтау үшін пайдаланылады. Әдістер мүшелері (*member methods*) кейде функциялық мүшелер (*member functions*) деп аталады және объектінің жәй-күйін анықтайды. Қазіргі уақытқа сай алдыңғы қатарлы C ++ және Java объектіге бағытталған программалау тілдерімен процедуралық тілдердің арасындағы негізгі айырмашылық, біріншісінде мұраға, инкапсуляция, полиморфизм сияқты т.б. объектіге бағытталған тұжырымдамаларды пайдалануға арналған синтаксисі бар екенінде.

C++ тілінің стандартты кітапханаларында көп мөлшерде жазылған төменгі деңгейлі коды бар, бірақ негізінен бұл стек және кезек сияқты төмен деңгейлі деректер құрылымдардың жинақтарын пайдалану үшін бейімделген. Деректер қорымен, суреттермен, интернетпен жұмыс істеу сияқты қарапайым программалау есептері C++ тілінің өзінде емес, операциялық жүйелердің қосымша кітапханалары түрінде іске асырылған.

C++ тілі пайдалануға қарапайымдылығына байланысты өнімділігі жоғары тіл ретінде белгілі C тіліне негізделген. C++ - бұл компиляциялайтын тіл. Бұл, бастапқы кодты жазғанда, ол анықтаған платформа үшін орындалатындай етіп, оны компиляциядан және компоновкадан өткізіп, басқа платформада орындалуы үшін қайта компиляция жасауды білдіреді.

Қолданушы интерфейсі әртүрлі платформаларда ерекшеленетіндіктен, қосымшаны C++-ке көшіру - программаны бір операциялық жүйеден басқасына көшіру барысындағы басты мәселе. C++ ешқандай қауіпсіздікті қамтамасыз етпейді. C++ тілінде программа жадының кез келген бөлігіне және кез келген ресурстарға қол жеткізе алады.

C++ қоқыс жинауды қолдамайды, яғни оның мүмкіндігі Java тіліндегі сияқты ресурстарды (жады) автоматты түрде тазалайды. C++ тілінде жады блогын бөле отырып, сіз оның босауына жауап бересіз. C++ тілі енгізу- шығару консолінен басқа ешқандай қолданушы интерфейсінің элементтерін ұсынбайды. Ол мұнда Windows жүйесіндегі сияқты ешқандай графикалық пайдаланушы интерфейсін қолдаудың жоғын білдіреді. Windows ортасының өзінің C++ те пайдалануға болатын жеке программалық интерфейсі бар (*application programming interface*, API). C ++ тілі көпжақты мұрагерлік қолдау көрсетеді, ол дегеніміз бір кластың бір мезгілде бірнеше базалық кластары болуы мүмкін дегенді білдіреді.

C++ шаблондар (*templates*) аталымды механизмін ұсынады. Шаблондар печенье пісіретін формалар сияқты кластар мен функцияларды анықтау барысында, жаңа кластар мен функцияларды құру үшін қолданылады.

C++ тілінің түйінді сөздері программа объектілерінің жады кластарымен, типтердің аттарымен, көріну спецификациясымен, модификаторлармен және операторлармен байланысты. C++ тілі операциясының белгілері өрнектерді қалыптастыру және әрі қарай есептеуді қамтамасыз етеді. Мәнді алу үшін өрнек қағида болады. Операцияның бір белгісі әртүрлі өрнектерде қолданылу мүмкін және контекстке байланысты әртүрлі интерпретациялануы мүмкін. Әрине, барлық операциялардың өз рангісі болып, приоритеттер иерархиясын құрап, ассоциативтік (оңнан солға немесе солдан оңға) ережеге бағынады.

C++ тілінің операторлары мен функциялары өз кезегінде классикалық алгоритмдік тілдердің құрылысын мұра еткен C тілімен толықтай дерлік сәйкес келіп, шамалы жетілдірілген. Операторлар әдеттегідей іс-әрекеттерді және осы іс-әрекеттердің программада орындалу логикасын (ретін) анықтайды. C++ тілінде әртүрлі функциялардың атауы бірдей болуы мүмкін. Бұл жағдайда әртүрлі функциялар ретінде бір-бірінен формальді параметрлер санымен немесе типімен ерекшеленетін функцияларды түсінеміз. Тек қана кері қайтарылатын мәннің типі бойынша айырмашылықтар рұқсат етілмейді. *Inline* модификаторы бар функциялар алмастыру функциялары деп аталады. Ереже бойынша, бұл барлық жұмысы жалғыз оператормен атқарылатын және оның көмегімен программада бірнеше рет қолданылатын кодты жүзеге асыратын функциялар.

C++ тілінде бар және C тілінде жоқ кейбір мүмкіндіктерді қарастырайық:

- C++ тілінде комментарияларды - // белгісі арқылы енгізуге болады;
- C++ тілінде енгізу-шығарудың қосымша жабдықтары бар;
- функцияны қайта жүктеу - бірнеше функциялар үшін бірдей атауларды пайдалану. Бір функцияға кейде әр типті объектілермен ұқсас әрекеттерді орындау талап етіледі;
- программада динамикалық жады облысын таңдап және жоюға мүмкіндік беретін *new* және *delete* операциялары;
- көріну аймағын кеңейту операциясы.

Кластар және оның әдістері (ОБП түйінді ұғымдары):
- конструкторлар мен деструкторлар кластар объектілерін құратын және жоятын арнайы әдістер болып табылады;
- кластың мүшесі болмайтын, бірақ ондағы сипатталған айнымалылар мен әдістерге қол жеткізуге мүмкіндігі бар достық функциялары;
- қайта жүктеу операциялары – тек стандартты типтегі деректерге ғана емес, әртүрлі кластардың объектілеріне де қолдануға мүмкіндік беретін тілдің көптеген негізі операцияларының мағынасын өзгерту мүмкіндігі;
- туынды кластар. Осы класс объектілері барлық ашық айнымалылар мен түбірлік (родительский) класс әдістерін мұра етеді, бірақ оған қоса өзінің жеке айнымалылары мен әдістері болуы мүмкін.
- полиморфизм - виртуалды функциялар жабдықтары арқылы жүзеге асырылады;
- хабарламалар көмегімен объектілерді басқаруға болады. Объектіге хабарламаны жіберу функцияны шақыруды еске түсіреді.

Бақылау сұрақтары:
1) Компьютерлік жүйелерді құру барысында объектіге бағытталған программалауды қолдану неліктен маңызды?
2) Мұралықтың объектіге бағытталған программадағы орнын түсіндіріңіз.
3) ОБП күрделі компьютерлік жүйелерді қандай жолмен қолдауға көмек береді?
4) Адам объектісінен мұра етілетін Университет объектілерін сипаттаңыз.

ДӘРІС №2. ДЕРЕКТЕР ТИПІНІҢ СІЛТЕМЕЛЕРІ

Дәрістің мақсаты: объектіге сілтемені қолданудың әдістерін ұғындыру.
Дәрістің мазмұны: сілтемені анықтау және оны қолдану әдістері.
Нұсқауыштан айырмашылығын көрсету. Функцияға сілтемені пайдалану.

C++ тілінде нұсқауышқа туыс элемент – сілтеме (reference) бар. Сілтеме барлық жағдайда жасырын нұсқауыш болып, оны кезкелген мақсат үшін айнымалының тағы бір аты ретінде қолдануға болады. Сілтемені үш түрлі әдіспен қолдануға болады. Біріншіден, сілтемені функцияға жіберуге болады. Екіншіден, оны функциядан қайтарып алуға болады. Ақырында, тәуелсіз сілтемені құруға болады. Сілтеменің ең маңызды қолданылуы – бұл оның функция параметрі ретінде берілуі.

Сілтемені анықтау үшін & белгісі пайдаланылады, егер ол келесі контексте қолданылатын болса: *type* & сілтеме_ аты инициализатор;

Сол жақтағы мәні болатын инициализатор ретінде болуы міндетті, яғни жадыда орын алатын объектінің атауы болуы керек. Инициализацияны анықтаудан кейін сілтеменің мәні сол объектінің адресі болады. Мысалы: *int* k=2; *int*& ref=k; сілтемені анықтауда‘&’ символы типтің бөлігі болып табылмайды, яғни ref *int* типті және дәл осылай программада қабылдануы керек.

Функциональді түрде сілтеме өзін-өзі қарапайым типті сияқты айнымалы ретінде ұстайды. Нұсқауыш үшін керек болатындай, сілтеме «қарайтын» жады бөлігінің мәнін иемдену үшін қайта атауды нақты орындау қажет емес.

Сілтеме айнымалы не көрсеткішке ұқсас толық құқылы объект емес. Инициализациядан кейін сілтеменің мәнін өзгертуге болмайды, ол әрқашан инициализациямен байланысқан жады бөлігіне «қарайды». Бір де бір операция сілтемеге әсер етпей, онымен байланысқан объектіге қатысты болады. Мұны сілтеменің негізгі қасиеті деп есептеуге болады. Сонымен, сілтеме толықтай бастапқы объект атымен бірдей.
Анықталған болсын:

```
int ar[]={1,2,3,4};      //ar - массив
int *p=ar;               //p - нұсқауыш
int & ref=ar[0];         //ref – массивтің бірінші элементіне сілтеме
int*& rp=ar;             //нұсқауышқа сілтеме (массив атына)
```

Келтірілген мысалдан сілтемелер мен нұсқауыштарға келесі теңдіктер орындалады:

p==&ref, *p==ref, rp==ar; ref== ar[0].

Сілтемеге Sizeof операциясын қолдану нәтижесі оның өлшемі емес, онымен аталған объектінің өлшемі болып табылады.

Сілтеме шын объект болмаған соң, сілтемені анықтауда және қолдануда

шектеулер бар. Біріншіден, сілтеме *void* типті бола алмайды. Сілтемені *new* операциясы көмегімен құруға болмайды, яғни сілтемеге жаңа жады бөлігін бөлуге болмайды. Басқа сілтемелерге сілтеме анықталмаған. Сілтемелерге нұсқауыштар жоқ және сілтеме массивін құру мүмкін емес.

Сілтеменің мәні өзгермегенімен, бір объект сілтемелер және нұсқауыштардың кез келген санымен адрестелуі мүмкін:

```
int k=7;                    // k айнымалысы анықталған int &r1=k;                    // r1сілтемесі  k-мен байланысты int *p=&r1;          // p нұсқауышы k-ні адрестейды int &r2=r1;                  // r2 сілтемесі k-ны көрсетеді
```

Енді k айнымалысының мәніне төрт әдіспен жетуге болады:
- k атауымен;
- r1 сілтемесі;
- *p нұсқауышы және r2 сілтемесі арқылы.

Сілтеме айнымалыға қарағанда, сілтеме өзімен байланысқан тұрақты объектінің мәнін өзгертуге мүмкіндік бермейді. Мысалы:

```
int k=5;
const int &ref=k;     //ref – тұрақтыға сілтеме Осы анықтамалардан:
k =0;                  // қалыпты меншіктеу операторы
ref =0;                // қате, себебі сілтеме тұрақтыға жарияланған
```

Сілтемелерді анықтауда оларды міндетті түрде атаулау (инициализация) талап етіледі. Алайда, сілтемелердің сипаттамасында оның болуы міндетті емес, бірақ та тыйым салынбайды.

Осындай сілтемелердің сипаттамаларына мыналар қатысты:
- сыртқы сілтемелер сипаттамасы (*extern* ерекшелегішімен (спецификаторымен));
- класс компоненттеріне сілтемелер сипаттамасы;
- функцияның формальды параметрлерін ерекшелеу;
- функция қайтаратын мән типінің сипатталуы.

C++ тіліне сілтемелерді енгізудің негізгі себебі (тек қана C++ үшін емес), функциялармен параметрлер аппараты арқылы алмасу тиімділігін және функцияны шақыруды сол тұрғыдағы мән ретінде мақсатты қолдану мүмкіндігін арттыру қажеттілігінен болды. Мысалы:

```
int f1(int &k)          // функция бүтін санды қайтарады
{return k;}
int &f2(int &k)         // функция k айнымалысына сілтемені қайтарады
{return k;}
```

Екі функцияда k енгізілетін параметрінің сандық мәніне сәйкес келетін мәнді қайтарады. Екінші жағдай үшін шақырудың келесі түрі мүмкін:

```
int main(void)
{int n=3;
```

```
f2(n)=7;                    // n  жетіге тең болады return 0;
}
```

Функцияға нұсқауышқа ұқсас функцияға сілтеме форматы анықталады: функция_типі (&сілтеме_аты) (функция_параметрлерінің_ерекшеліктері) =атаулаушы өрнек; Мысалы:

```
int func(float, int);                    // функция прототипі int (&ref) (float, int)=func;                    // сілтеменің анықталуы
```

Функция атының жақшасыз (және параметрлерсіз) қолданылуы функция адресі ретінде қабылданады.

Функцияға сілтеме функцияның негізгі атының барлық құқықтарына ие, яғни оның синонимі (псевдонимі) болып табылады. Функцияға сілтеме (нұсқауышқа сілтеме) мәнін өзгерту мүмкін емес, сондықтан функцияға нұсқауыштар сілтемеге қарағанда үлкен ауқымға ие. Келесі программа функцияны шақыруды негізгі аты бойынша, нұсқауышпен және сілтеме бойынша орындауды суреттейді:

```
#include <iostream.h> void func(char lit)
{cout<<endl<<lit;} int main (void)
{void (*pf)(char);                    // pf - функцияға нұсқауыш void(&ref) (char)=func;                    // ref - функцияға сілтеме func('A'); //          атымен шақыру
pf=func;                    // нұсқауышқа функция адресі меншіктеледі (*pf) ('B');                    // нұсқауыш көмегімен адресі бойынша шақыру ref('C');                    // функцияны сілтеме арқылы шақыру
return 0;
}
```

Функциямен сілтемені қайтару бірнеше функцияларға бірнеше қайтара әрекеттесуді ұйымдастыруға мүмкіндік береді. Осы әрекеттесу нәтижесінде бір объектіні бірнеше қайтара әр түрлі ережемен өзгертуге болады.

Бақылау сұрақтары:

1. Сілтеме дегеніміз не?
2. Сілтеме параметрлерінің қандай артықшылығы мен кемшіліктері бар?

ДӘРІС №3. КЛАСС АБСТРАКТЫЛЫ ТИП РЕТІНДЕ

Дәрістің мақсаты: класс ұғымын толық түсіндіру.

Дәрістің мазмұны: «Клиент-сервер» технологиясы бойынша класс әдістерін анықтау. Конструктор мағынасы. Деструктор мағынасы.

Деректер абстракциясына негізделетін программаларды жүзеге асыру және құрастыру, жобалау принциптерінің жиынтығы, шешілмелі есептердің ерекшеліктері мен түсініктерін анағұрлым толық көрсететін деректердің жаңа типтерін құруды қарастырады. С++ тілінде программалаушының өзіндік деректер типтерін енгізуге және олармен кластар көмегімен орындалатын операцияларды анықтауға мүмкіндігі болады.

Класс - бұл бар типтер негізіндегі туынды құрылымдық (структурированный) тип. Класты келесідей ең оңай әдіс арқылы анықтауға болады:

Класс_кілті класс_аты {компоненттер тізімі};

мұнда класс_кілті - class, struct, union қызметші сөздерінің бірі; класс_аты - еркін таңдалатын идентификатор;

компоненттер тізімі - класқа қатысты функциялар мен типтелген деректердің сипаттаушылары мен анықтамалары. Компоненттер обьект күйін анықтайды және класс объектісінің тәртібін шартқа негіздейді. Класс компоненттері деректер, функциялар, кластар, санауыштар, биттік өрістер, достық функциялар мен типтер атаулары бола алады. Мысалы:

```
class Classid
{ int k;                    // к, х, у жабық мүшелер - үнсіздік бойынша деректер
double x,y;
public: char lit, ch;          // lit,ch ашық мүшелер - деректер int f3(int,int);    //
кластың ашық әдістері
int Get x (void)               //  кластың орнына қою әдісі
{ return x; } private:
void f1 (void);                // жабық мүшелер - класс әдістері int f2 (int);
};
```

Қатынау спецификаторлары public: және private: қатынау спецификаторлары класс мүшелеріне өтуге мүмкін беретін режимді анықтайды: private - класс мүшелерін жабыққа айналдырады, ал public - ашық. Кластың жабық мүшелеріне тек берілген класс мүшелерінің қатынауға рұқсаты бар, сонымен қатар функциялардың – класс достары. Кластың ашық мүшелеріне кез келген функциялардың қатынауға мүмкіншіліктері бар. Олар бар болып табылатын бағдарлама мен класс объектілерінің байланысы үшін арналған.

Класты жобалап отырып, оның қай мүшесін ашық, ал қайсысын жабық етуді мұқият ойластырған жөн. Көптеген жағдайда класс анықтамасы блокта локализацияланбаған және класс атауының іс-әрекет облысы бүкіл файл болып табылады.

Жай кластық әдістерін анықтау класс анықтамасының ішінде болуы мүмкін және мұндай жағдайда олар автоматты түрде орын ауыстырғандар болып

табылады. Орын ауыстыратын функцияларды егер функция қарапайым және қысқа болса, қолданған жөн, мысалы, Getx ( ) функциясы.

Әдетте, қауіпсіздікті жоғарылату мақсатымен кластың мүше- берілгендерін жалпыға бірдей рұқсат етілмейтін етеді және еркін бағдарлама ортасынан олармен тікелей байланысуға болмайды. Бірақ оған қарамастан олармен кластың барлығына рұқсат етілген әдістерін қолданып ерікті бағдарламалық ортадан да жұмыс істеуге болады (public:). Жоғарыда қарастырылған мысал үшін келесі әдіс қолданылады:

```
int Getx (void)
{ return x; }
```

жабық мүше мәнін берілген x қайтарады.

Арнайы түрде мұндай әдістердің «жүдеу» денелерін тиімділікті жоғарылату үшін орын ауыстырушы етіп класс анықтамасының ішінде енгізеді. Класс анықтамасынан тыс орналастыратын әдістерді анықтаған жағдайда әдістер тақырыбына inline спецификаторы қосылады.

«Клиент-сервер» технологиясы бойынша класс әдістерін анықтау класс анықтамасынан тыс орналастырған жөн. Көру облысының «::» рұқсат ету операторы арқылы компиляторға берілген анықталатын әдіс қандай класқа қатысты екендігін хабарлайды, мысалы:

```
int Classid:: f2 (int x)
{ әдіс денесі }
```

Класс анықтамасы берілген класс объектілерін құрмайды. Объектілер оларды анықтау жолымен ғана құрады, мысалы:

```
Classid obj1, obj2, obj Array [10];
```

Класс объектісінің өлшемі кластың статикалық емес мүше берілгендерінің өлшемдер қосындысымен жадыда анықталады. Класс әдістері класс объектісі үшін бөлінген жады облысының орнын алмайды.

Бірнеше файлдардан тұратын бағдарламалық жобалармен жұмыс жасағанда класс анықтамасы берілген класс объектілері қолданатын немесе оның әдістері анықталатан файлдарда болуға міндетті. Сондықтан класс анықтамасын арнайы түрде ол қажет болып табылатын файлдарда #include дерективасы көмегімен қосылатын тақырыптық файлға (хедер-файл) орналастырылады. Егер класс әдісінің анықтамасы анықталудан тыс орналасса, онда ол міндетті түрде өзі қолданылатын файлдарда анықталған болуы керек. Мұндай әдістің анықтамасы класс анықтамасымен бірге тақырыптық файлда болуы керек.

Белгілі бір класс объектісінің ашық мүшелеріне қатынауға рұқсат алуды тура «.» және жанама «» таңдау операторлары көмегімен жүзеге асырылады.

C++ тілінде объектілерді жою мен инициализациялау жұмыстарын орындайтын, обьектілердің жойылуы мен құрылуы кезінде автоматты түрде шақырылатын, кластың арнайы әдістері сәйкесінше конструкторлар мен

деструкторлар деп аталады. Класта бірнеше конструкторларды құруға болады. Әрбір конструктордың өзінің басқалардан айырмашылығы бар параметрлері болады. Параметрлері жоқ конструктор үнсіздік конструкторы. Деструктор әрқашан жалғыз және параметрлері болмайды.

Тек ашық мүшелері бар және конструкторы жоқ класс объектісі мәндер тізімі арқылы жай құрылымдық айнымалы сияқты инициализациялануы мүмкін.

Егер класс конструкторларының біреуі де ашық функция – класс мүшесі болып табылмаса, онда мұндай кластың мүшелері құрыла алмайды. Мұндай кластар басқа кластар (мұрагерлік) үшін базалық кластар (родительские) болып табылады.

Класс объектісін құрған кезде ол үшін сәйкес келетін конструктор автоматты түрде шақырылады. Объекттің деректер мүшесі (объект айнымалыларының) инициализациясы конструктордың орындалуы сияқты инициализациясы кезінде де орындала алады.

Конструктор параметрі өз класы бола алмайды, бірақ көшіру конструкторы сияқты оған жасалған сілтеме болуы мүмкін.

Кластың деректер компонентінің жалғыз данасы болуы үшін және әрбір жаңа кластың объектісінің құрылуы кезінде тираждалмауы үшін ол класты статикалық сияқты анықтау керек, яғни Static атрибуты болуы керек.

Кластың статикалық компоненттік функциялары нақты обьект атауы жоқ кластың Static берілгендеріне қатынасуды қамтамасыз етеді. Кластың Static функциялары функциялардың жай (статикалық емес) компоненттік барлық негізгі ерекшеліктерін сақтайды. Класқа қатысты функция нақты объекттің деректерін өңдеу үшін шақырған кезде бұл функцияға автоматты және айқын емес функция шақырылған объект көрсеткіші беріледі. Бұл нұсқауыштың бекітілген *this* аты бар және программалаушы үшін кластың әрбір функциясы келесі түрде анықталған:

класс_аты*const this=өңделетін_обьекттің_адресі;

this нұсқауышын қолдану тиімділігі көрінетін жағдайлар:

- берілген класс компонентінің аты класс функциясының формальды параметрінің атымен сәйкес келгенде;
- класс функциясының денесінде айқын түрде бұл функция шақырылған объект адресін көрсету қажет болғанда;
- функция параметрі ретінде бұл кластың функция атын жазған кезде объекттің нақты атына жол берілмегенде (керек объектіге мұндай функция сілтемесін немесе нұсқауышын берудің орнына).

Бақылау сұрақтары:

1) Белгілі бір кластың екі объектісін анықтағанда, деректер мүшелері өздерінің әртүрлі (бірдей) мәндеріне ие бола ала ма?
2) Кластың қандай деректер мүшелерін қандай жолмен иниализациялау керек?
3) this нұсқаушысы нені білдіреді?

Дәріс №4. Кластар достары. Кластарға арналған стандартты операцияларды қайта жүктеу

Дәрістің мақсаты: класс достары және достық функциясы ұғымдарын беру.

Дәрістің мазмұны: достық функциясының ерекшеліктері. Көшіріп алу конструкторы. Беттік және тереңдетілген көшіріп алу. Кластарға арналған операциялардың қайта жүктелуі.

Класс мүшесі болмауына қарамастан кейбір жағдайларда функция кластың жабық мүшелеріне тиесілікке (доступ) ие болғаны жөн. Егер функция класс досы немесе мүшесі болып табылатын класты friend қызметші сөзі көмегімен жарияласа, осы әрекетті С++ ортасы орындауға мүмкіндік береді,.

Достық функциялары мен кластарды қолданудың бірқатар қызықты ерекшеліктерін көрсететін мысалды қарастырайық:

```
#include <iostream.h>
class Class1;                              // класты алдын ала жариялау class Class2;
{ public : void ff (Class1 &ref);
};
class Class1
{ private : int x;                         // достық функциялары мен кластарға арналған
//жол беру спецификаторының мәні жоқ
friend int  main (void);                   // main – достық функциясы бола алады friend
void f (Class1 &ref);
friend class Class3;
friend void Class2 :: ff (Class1 &ref);
};   //Class1 анықтамасының соңы
class Class3                               // Class3 – Class1 класына дос
{ public : void func (Class1 &ref)
{ ref.x=9;
```

```
return;
}
};
void Class2 :: ff (Class1 &ref)
{ ref.x=11; return;
}
//f – Class1 класына дос қарапайым сыртқы функция void f (Class1 &ref)
{ ref.x=7; return;
}
int main (void)
{Class1 obj1; obj1.x=5;
cout << “obj.1x=” << obj1.x << endl; //x=5 f(obj1);
cout<<“obj1.x=” << obj1.x <<endl;                    //x=7 Class3 obj3;
obj3.func(obj1)
cout<< “obj1.x=” << obj1.x <<endl;                    //x=9 Class2 obj2;
Obj2.ff(obj1);
cout<< “obj1.x=” <<obj1.x<<endl;                    //x=11 return 0;
}                    // main соңы
```

Достық функциясының ерекшеліктері:

- достық функция класс объектісіне автоматты түрде this нұсқауышын ала алмайды, сондықтан оның шақыруында міндетті түрде аргумент ретінде оған жұмыс істеуге қажет класс объектісін көрсету керек;
- консольды қосымшаның басты функциясы класқа дос болуы мүмкін;
- қажетсіз достық функциялары мен кластарын қолдануға ұмтылмау керек, себебі бұл инкапсуляция концепциясына кері болып табылады.

Кластарға арналған операциялардың қайта жүктелуі (перезагрузка).

С++ тілінің көптеген @ операциялары қолданушы типтерінің объектілерімен жұмыс істеу үшін қайта жүктелуі мүмкін. Мұндай мүмкіндіктер объект-кластар үшін де болады. @ операциясының қайта жүктелуі operator @ фунция-операциясы көмегімен жүзеге асырылады, оны класс мүшесі немесе достық функциясына айналдыру (қажет емес) немесе қарапайым (глобальды) функцияға айналдыруға болады.

Соңғы екі жағдайда функция-оператор ең болмаса бір сілтеме немесе нұсқауыш, класс типі бар аргументті қабылдау керек. Сыртқы функция - операциясы үшін сәйкес класс мүшелерінің тиесілігін (доступность) ескерген жөн.

Функция - оператор атауы operator қызметші сөзінен, және одан кейін орналасқан қайта анықталатын операция белгісінен тұрады.

Операцияларды қайта жүктеу типті беру, функцияны шақыру ережелері ISO/IES14882 халықаралық стандартында берілген.

Көшіріп алу конструкторы. Беттік және тереңдетілген көшіріп алу

Класс құрамына арнаулы түр конструкторы – көшіріп алу конструкторы жатады. Көшіріп алу конструкторының жалғыз параметрі ретінде, осы класс объектісіне сілтемені қолданады:

Classid : Classid (const Classid & obj)
{ конструктор денесі }

Көшіріп алу конструкторы жаңа обьект класс типімен бар обьектіні көшіріп алу жолымен құрылған кезде шақырылады:
- сол типтің басқа бар объектімен инициализациясы және класс типі бар жаңа объект анықтаған кезде;
- мән бойынша класс типі бар параметр - объект класс әдісіне берілген кезде;
- return операторы арқылы класс типі бар объект мәнін класс әдісінен қайтарылған кезде.

Егер программалаушы бір де бір көшіріп алу конструкторын құрмаса, онда компилятор автоматты түрде қалыпты көшіріп алу конструкторын құрады. Мұндай конструктор кластың мәліметтер өрістерінің элемент бойынша көшірілуін орындайды (беттік көшіріп алу). Егер өрістердің ең болмаса біреуі динамикалық жадының белгілі бір облысына нұсқауыш болып табылмаса, онда қалыпты конструкторды қолдану программаның дұрыс емес, яғни бұрыс жұмысына әкеледі. Осы жағдайда қалыпты көшіріп алу конструкторының орнына өзінің көшіріп алу конструкторын жобалап алған жөн:
- беттік көшіруді орындайтын конструктор (нұсқауыштың объектіге көшірілуі орындалады, ал объект көшірмесі динамикалық жадыда құрылады);
- тереңдетілген көшіріп алуы бар конструктор (динамикалық жадыда объект көшірмесі құрылады).

Қорытындысында мынаны қайталайық, егер класта динамикалық жадымен жұмыс орындалмаса және конструктор қажет болмаса, онда қалыпты көшіріп алу конструкторын қолдану көмегімен көшіріп алу конструкторын анықтау керек емес. Осындай кластарды қолданатын бағдарламалар дұрыс жұмыс істейді.

Мүмкіндігінше мән бойынша, класс типі бар параметрді кластың әдісіне және класс типі бар объекттің мәнін емес, сілтемесін оған класс әдісіне қайтарудан аулақ болған жөн. Бұл жағдайда көшіріп алу конструкторы қажет емес, бұл қосымшаның шапшаңдығының жоғарылауына әкеледі.

Көрсетілген мақсатқа жету үшін келесі ережелерді қолданған жеткілікті:
- мән бойынша класс типі бар параметрді, класс әдісіне берудің орнына бұл параметрді сілтememен берген жөн, ал сәйкес аргументтің модификациясын тоқтату үшін функция параметрін const модификаторымен қамтамасыз ету керек;
- класс типі бар объектілер үшін арифметикалық операцияларды қайта жүктегенде қайта жүктеу әдісін объектке қайтаратындай етіп жобалау керек (оны әрқашанда орындауға болады).

Бақылау сұрақтары:
1) C++ тілінің операцияларын қайта жүктеу операциялары қандай?
2) Көшіріп алу конструкторы қашан шақырылады?
3) Бірігіп көшу операторларының префиксті және постфиксті қайта жүктеуін қалай айыруға болады?
4) Short int типті операндалар үшін қосынды операциясын қайта жүктеуге бола ма?

ДӘРІС №5. ҮЛГІЛЕР (ШАБЛОНЫ)

Дәрістің мақсаты: көп қолданылатын және қарсылыққа тұрақты программаларды құру мүмкіндіктері.

Дәрістің мазмұны: функция-үлгілер. Туыстық функциялар. Туыстық кластар.

Жоғары деңгейлі C++ тілінің келесідей екі негізгі сипаттамасы болып мыналар табылады: үлгілер (templates) (шаблоны) және ерекше жағдайлардың өңделуі (exception handling). Бұл сипаттамалар қазіргі уақыттағы барлық компиляторлардан қолдау табады және программалауда анағұрлым қызықты екі мақсатқа жетуге: көп рет қолданылатын және қарсылыққа төзімді программаларды құруға мүмкіндік береді.

Үлгілер көмегімен туыстық функцияларды (general functions) және туыстық кластарды (general classes) құруға болады. Туыстық функцияда немесе класта, класс немесе функция жұмыс істейтін мәліметтер типі параметр ретінде беріледі. Бұл бір класты немесе функцияны функция немесе кластың жаңа нұсқауын қолданбай-ақ деректердің әртүрлі бірнеше типтерін қолдануға мүмкіндік береді. Сөйтіп, үлгілер көп рет қолданылатын программаларды құруға мүмкіндік береді.

Көптеген алгоритмдер логикалық жағынан бірдей деректер типінен тәуелсіз, мысалы, сұрыптау алгоритмдері. Туыстық функцияны құрудың арқасында деректер типінен тәуелсіз алгоритм мәнісін анықтауға болады.

Туыстық функция template шешуші сөзі арқылы құрылады. Функция- үлгі анықтамасының типтік формасы:

```
template < class T > мәнді_қайтару функция_аты (параметрлер_тізімі)
{функция денесі}
```

Мұндағы T - берілгендер типінің жалған аты, оны компилятор функцияның нақты нұсқасын құрғанда берілгендердің шын типінің атымен автоматты түрде ауыстырады.

Мағынасы ұқсас функциялар бар болғанда түрлі деректер типтері үшін екі шаманың ең кішісін табу үшін MyMin атты жалғыз үлгілік функцияны анықтауға болады:

```
template < class T > T MyMin (T x, T y)
{ if (x<=y) return x; else return y;}
```

Ол потенциалды түрде белгісіз T типімен де жұмыс істей алады.

Функциялар үлгілерінің келесі тақырыптары синтаксистік жағынан дұрыс болып табылады:

```
template< class T > T Fun1(T x, T y, int z) template<class T > double Fun2 (Tx, Ty)
template<class T, class W > W Fun3(Tx,Wy,bool z).
```

1) Үлгінің формальды параметрлерінің анықтауыштары (T,W) ең болмаса формальды параметрлер тізімінде бір рет болса да болу керек.

2) Үлгілік функцияның денесі ішінде үлгінің формальды параметрлері локальды айнымалыларды анықтау операторларында, типті беру операторларында және т. б. өзге түрлі деректердің нақты типтерімен бірдей қолданылады, яғни жұмыс істеуде деректер типтерінің толық құқылы атаулары сияқты.
3) Кәдімгі функциялар сияқты үлгілік функцияларға да прототиптерді жазуға болады, сонымен қатар оларға inline, static спецификаторларын да жазуға болады (бұл спецификаторлар үлгінің формальды параметрлер тізімінен кейін және функциямен қайтарылатын мән типіне дейін орналасу керек).
4) Функция үлгілерін анықтау мәтіні шақыру орындалатын файлдарда немесе тақырыптық файлдарда (h-файлдарда) орналастырылу керек.
5) Үлгілік функция анықталу алдында template кілттік сөзімен нұсқаулық орналасу керек.
6) Функция үлгілерінің механизмін дәлелсіз қолданудың ешқандай қажеті жоқ! Біріншіден, функция үлгілерінің деректердің абстрактты типтерімен байланысының пайдасы бар. Үлгілер механизмі глобальды функциялар үшін емес кластар мен олардың әдістерінің үлгілері үшін ерекше пайдалы!

Туыстық кластар.

Туыстық функцияларын толықтыру үшін туыстық кластарды да анықтауға болады. Осы кластың объектілерін құру кезінде барлық қажетті алгоритмдер анықталып, өңделетін деректердің нақты типтері параметрлері ретінде кейінірек беріледі.

Туыстық кластар, класс құрамында жұмыстың жалпы логикасын қамтығанда пайдалы. Мысалы, бүтіндер кезегін жүзеге асыратын алгоритм символдар кезегімен де жұмыс істейді. Сонымен пошталық адрестердің байланысқан тізімін жүзеге асыратын механизм автомобильдерге арналған қосалқы бөлшектердің байланысқан тізімін де қолдайтын болады. Туыстық класс арқылы түрлі берілгендер типтеріне арналған байланысқан тізімдер кезегін жүзеге асыратын класты құруға мүмкіндік береді. Компилятор берілген объектіні құруда, объектінің тип негізінде дұрыс типін автоматты басқарады.

Класс үлгісінің сипатталу синтаксисі келесі түрде болады:

- template < үлгі_параметрлерін_сипаттау > класс_анықтамасы
- Параметрлер ретінде стандарттыға ұқсас қолданушымен анықталатын типтер үлгілер мен типтелген конструктор қолданылады.

Бір байланысты тізімді жүзеге асыратын жай туыстық класты құрайық:

```
#include< iostream.h > template< class T > class List
{ T data; List*next; public:
List(T d);                                  //  класс конструкторы Void add(List*node)
{node next=this; next=0;} List*getnext() {return next;} T getdata(){return data;}
}
template< classT >List<T>::List(Td)
{data=d; next=0;} int main(void)
{List< char > start('a');                   //start-обьект List< char >*p,*last;
//тізімді құру last=&start;
for(int i=1; i<26;i++)
```

```
{p=newList<char> (‘a’+i); p->add(last); last=p;
}
//тізімді шығару p=&start; while(p)
{cout<<p->getdata(); p=p->getnext();
}
return 0;
}
```

Көрініп тұрғандай, туыстық класты жариялау туыстық функция жариялануына ұқсас. Тізімде сақталынатын берілгендер типі класты жариялағанда туыстыққа айналады, бірақ ол деректердің нақты типін беретін объект жарияланбай көрінбейді. Объектті құрғанда көрсетілген мәліметтер типін өзгерту жолымен тізімде сақталған мәліметтер типін өзгертуге болады.

Кейбір қорытындыларды келтірейік және үлгілерді сипаттаудың негізгі ережелерін атап өтейік:

1) Класс үлгісінің ішінде тип параметрі тип спецификациясы қолданылатын кез келген жерде қолданылады.
2) Үлгі параметрінің әрекет ету облысы – үлгі параметрінің сипаттау нүктесінен класс үлгісінің соңына дейін.
3) Класс үлгісінің әдістері автоматты түрде функция үлгілеріне айналады. Егер әдіс класс үлгісінен тыс сипатталса, онда әдіс тақырыбы келесі құрылымға ие болу керек:

template<үлгі_параметрін_сипаттау> типті_қайтару класс_аты < үлгі_параметрлері>::

Үлгі параметрлерін сипаттау әдіс тақырыбын сандық және позициялық түрінде сақтай отырып, класс үлгісіне сәйкеес келу керек.

4) Локальды кластар үлгілерді өз элементтері ретінде қолдана алмайды.
5) Класс үлгілерінің әдістері виртуальды бола алмайды.
6) Класс үлгілеріне статикалық элементтер, достық функциялар мен кластар ене алады.
7) Класс үлгісінің ішінде достық функциялардың үлгілерін анықтауға болмайды.
8) Класс үлгілері үлгіліктерден және жай кластардан туынды бола алады, сонымен қатар үлгілік және жай кластар үшін базалық болып табыла алады.

Бақылау сұрақтары:

1) Класс үлгілерін анықтау мүмкіндігі не?
2) Үлгілік функцияға арналған машиналық кодтың шын генерациясы қалай және қашан жүргізіледі?
3) Үлгілік функцияны шақырғанда нақтыландыру қалай орындалады?
4) Белгілі бір функционалдық үлгінің қай функциясы мамандanған деп аталады?
5) Кезекті жүзеге асыратын туыстық класты құрыңыз және көрсетіңіз.

ДӘРІС №6. МҰРАГЕРЛІК

Дәрістің мақсаты: кластар мұрагерлігін түсіндіру.

Дәрістің мазмұны: кластың қорғалған мүшелері. Конструкторлар, деструкторлар және мұрагерлік.

Мұрагерлік – C++ тіліндегі объектіге бағытталған программалаудың басты механизмдерінің бірі. Оның көмегімен жалпыдан жекеге ауыса отырып өте күрделі кластарды құруға болады, сонымен қатар қорытынды кластардан айрықшаланатын жаңа кластарды өсіруге болады.

Жаңа класты жобалай отырып, алдын-ала оның объектілері қандай анағұрлым жалпы ерекшеліктерге ие болу керектігін анықтау және ұқсас дайын кластың бар жоқтығын тексеру қажет. Басқаша айтқанда, басында қайта өндіретін класс жоспарын «ірі көріністермен» құрған жөн, ал кейін бір уақытта жаңа қасиеттерді ала отырып, тәртіп пен қасиеттерді (яғни мүше деректер мен класс әдістері) мұраға алатын жаңа кластарды құрылған класс негізінде құра отырып, рет-ретімен бөлшектенуге көшеміз.

Түрлі класс объектілері мен кластардың өздері алдын-ала құрастырған кластар иерархиясына сай келетін объектілер иерархиясы құрылған кезде мұрагерлік қатынасында бола алады.

Бір класс екінші класс мұрагері болғанда жазбаның келесі негізгі пішіні қолданылады:

{ class_ класс_ аты_ as базалық_ класс_ аты _ туынды
{ туынды класының анықтамасы

Мұндағы as – рұқсат ету спецификаторы (access specifier) базалық класс элементтері (base class) туынды класқа (derived class) қалай мұрагер болатынын анықтайды.

Егер as public болса, онда барлық ашық мүшелері туындысы да ашық болып қалады. Егер as private шешуші сөзі болса, онда базалық кластың барлық мүшелері туынды класта жабыққа айналады. Екі жағдайда да базалық кластың барлық private мүшелері туынды класта қалай мұрагер болуына тәуелсіз, жабық және рұқсатсыз болып қалады.

Егер as private болса, онда базалық кластың ашық мүшелері туынды класта жабық болуына қарамастан олар туынды кластың функция мүшелері үшін рұқсат етілген болып қалатынын ескерген жөн.

Техникалық жағынан рұқсат ету спецификаторы міндетті емес. Егер ол көрсетілмеген туынды класс class шешуші сөзімен анықталса, онда базалық класс struct қызметші сөзімен үнсіздік бойынша ашық түрде мұра болады.

Түсініктілік үшін рұқсат ету спецификаторын нақты түрде беру дұрыс. Public спецификаторымен мұрагерлік мысалын көрсетейік:

class Base
{ int x; public:

```
void setx (int n) {x=h;}
void showx( ) { cout « x «endl ;}
};
class Derived: public Base
{ int y; public:
void setx (int n) {x=h;}
void showx( ) { cout « x «endl ;}
};
int main (void)
{ Derived obj; obj.setx (10);
obj.sety (20); obj showx( ); obj showy( );
return 0;
}
```

Base класы ашық болып, мұраға қалдырылатындықтан оның ашық функциялары туынды класқа ашық болып қалады және сондықтан да программаның кез келген бөлігінен рұқсат етілген болады. Бұл функцияларды main ( ) функциясынан сәйкесінше дұрыс шақыру керек.

Базалық кластың мүшелері рұқсат етілмеген болады, сондықтан туынды класс ішінде x айнымалысына тікелей рұқсат алуға талаптану әрекеті дұрыс емес. Ал базалық кластың ашық функциясы арқылы бұл мүмкін болады.

Егер класс сияқты жабық мұраға қалса, онда базалық кластың барлық мүше деректері мен туынды класта жабық болады және одан тыс қана рұқсат етілген болады. (Derived типті берілген объекттер үшін олар жабық болады).

Private спецификаторын қолдана отырып мұра ету кезінде базалық кластың ашық мүшелері туынды класта жабық болады.

```
# include < iostream.h >
class Base
{ int x; public:
void setx (int n) {x=h;}
void showx( ) { cout « x «endl ;}
};
class Derived: public Base
{ int y; public:
void setxy(int n, int m) {set(n); y=m;}
void showxy( ) {showx( ) ; cout « y «endl ;}
};
int main (void)
{ Derived obj; obj.setx (10);
obj.sety (20); obj showx( ); obj showy( ); return 0;
}
```

Бұл жағдайда show x ( ) және set x ( ) функциялары туынды класс ішінде рұқсат етілген болады, бұл өте дұрыс, себебі олар бұл кластың жабық мүшелері

болып табылады.

Кластың қорғалған мүшелері.

Қажет болғанда, базалық класс мүшелері жабық болып қалуы үшін, туынды класс қолжетімді болуы мүмкін. Осы идеяны іске асыруда C++ те protected: спецификаторы қарастырылған. protected: спецификаторының қолжетімділігі private: спецификаторынының жалғыз ерекшелігі - базалық кластың қорғалған мүшелері осы базалық кластың барлық туынды кластар мүшелеріне қолжетімділігіне. Базалықтан тыс немесе туынды кластардың қорғалған мүшелері қолжетімсіз.

Protected спецификаторы кез келген жерінде, әдетте кластың private мүшелерінен кейін public – класс мүшелерінің алдында орналасуы мүмкін.

Базалық класс туынды кластың ашық (public) сияқты мұрасы болғанда базалық кластың жабық мүшесі туынды кластың қорғалған мүшесіне айналады. Базалық класс жабық (private) сияқты мұра болғанда, онда базалық кластың қорғалған мүшесі туынды кластың жабық мүшесіне айналады.

Сөйтіп мынаны ескерген жөн, туынды клас объектілері үшін мыналар рұқсат етілген: базалық кластың объектілері үшін ашық мүше деректері мен барлық ашық әдістері рұқсат етілген:

- класс мүшелері болып табылмайтын функциялар;
- глобальды айнымалылар.

Туынды класс ішінде базалық кластың базалық қорғалған мүшелері рұқсат етілген.

Базалық класс тізімінде туынды класты жариялауға public модификациялары қолданылады: protected: private: олардың көмегімен базалық кластың мүшелері рұқсат етілмеуі мүмкін.

Бұл C++ тілінің маңызды функциясы болып, объекттіге бағытталған программалаудың жақсы стилінің белгісі табылады.

Базалық кластың қорғалған мүшелері туынды кластың қорғалған мүшелері болады және main функциясы ішінде олар рұқсат етілген болады.

Protected рұқсат ету спецификаторын құрылыммен бірге де қолдануға болады.

Конструкторлар, деструкторлар және мұрагерлік.

Егер базалық және туынды кластардың конструкторлары мен деструкторлар бар болса, онда конструкторлар мұрагерлік тәртіппен, ал деструктор кері тәртіпте орындалады.

Базалық класта инициализация бірінші болып орындалуы, ал туынды класс деструкторы объект жойылғанға дейін шақырылуы керек.

Базалық және туынды кластардың барлық қажет аргументтері туынды класс объектілері құрылғанда туынды класс конструкторына беріледі. Кейін туынды класс конструкторын жариялаудың кеңейтілген формасын қолдана отырып, сәйкес аргументтер әрі қарай базалық класқа беріледі. Аргументтерді туындыдан базалық класқа берілу синтаксисі төменде көрсетілген:

туынды_класс_конструкторы (аргументер_тізімі): базалық_класс (аргументтер_ тізімі)

Базалық және туынды кластар үшін бірдей элементтерді қолдану рұқсат

етілген. Сонымен туынды класс үшін базалық аргументтерді елемеуге және тікелей оларды базалық класқа беру рұқсат етілген.

Бақылау сұрақтары:

1) Базалық және туынды кластар әдістерінің бірдей аттары болуы мүмкін бе?
2) Туынды кластың болашақ ұрпағына базалық кластың әдістері мен деректеріне мұра бола ала ма?
3) Three класы Two класымен түзілген болсын, ал Two класы One класымен түзіледі. Бұл әдістің қай түрін Three класы алады?
4) Базалық класта public түрінде сипатталып өткен әдісті туынды класта private: түрде сипаттауға бола ма?
5) Protected қызметші сөзі қандай мақсатта қолданылады?

ДӘРІС №7. КЛАСТАР ИЕРАРХИЯСЫ

Дәрістің мақсаты: кластардың иерархиялық құрылымын түсіндіру.

Дәрістің мазмұны: базалық класс. Туынды класс. Виртуалды базалық кластар.

Туынды класс бір базалық кластан артық класқа мұра ете алатын екі әдіс бар. Біріншіден туынды класс көп деңгейлі кластар иерархиясын құра отырып, базалық ретінде басқа туынды класс үшін қолдануға болады. Екіншіден, туынды класс бір базалық кластан артық класты тікелей мұра ете алады. Класс туындыға базалық класс ретінде қолданылса, өз кезегінде басқа туынды класс үшін базалық болса, онда барлық үш кластың конструкторлары мұрагерлік ретімен шақырылады. Деструкторлар кері тәртіппен шақырылады.

Егер туынды класс бірнеше базалық кластың мұрасына тікелей ие болса, онда мынадай кеңейтілу қолданылады:

```
class туынды _класс _аты:as базалық_ класс_аты1; as базалық_класс_аты N
{ туынды класс денесі }
```

Мұндай жағдайда конструкторлар туынды кластың хабарламасында берілген тәртіп бойынша солдан оңға қарай орындалады. Егер осындай мұраға ие болу әдісі кезінде базалық кластардың конструкторларына аргументтерді беру қажет болса, туынды класс оларды туынды класс конструкторларының хабарламасының кеңейтілген формасын қолдана отырып береді:

```
туынды_класс_конструкторы (аргументтер_тізімі); базалық_класс1_аты (аргументтер_тізімі); базалық_класс 2_аты (аргументтер_ тізімі);
....., базалық_класс N_аты (аргументтер_тізімі)
{туынды класс конструкторының денесі }
```

Егер туынды класс кластар иерархиясының мұрасына ие болса, әрбір туынды класс тізбекті алдыңғы базалық класқа барлық қажет аргументтерді береді. Бұл ережені орындамау программаны компиляциялау кезінде қателік туғызады.

Қызметші сөз class көмегімен анықталған класс базалық сияқты туынды бола алады. Мұндай жағдайда үнсіздік бойынша класс мүшелері жабық болып табылады.

Қызметші сөз Struct көмегімен анықталған кластар базалық сияқты туынды да бола алады, және үнсіздік бойынша класс мүшелері ашық болып табылады.

Қызметші сөз union бар класс белгілі бір класқа қатысты базалық та, туынды да бола алмайды. Туынды класс кезіндегі жағдайларды түсіндіретін бағдарламаны қарастырайық:

- басқадан туынды класс мұрасына ие болады;
- екі базалық кластың мұрасына тікелей ие болады;

- бірнеше базалық кластың мұрасына тікелей ие болады.

```
# include <iostream.h> class Base1
{ int a;
public: Base1(int x) {a=x; } int geta() { return a; }
} ;
class Der1:public Base1
{ int b;
public: Der1(int x,int y):Base1(y)
{ b=x;}
int get b() {return b;}
};
class Der2 :public Der1
{ int c;
public:Der2 (int x,int y,int z):Der1(y,z)
{ c=x;} void show()
{ cout«geta ()« ´ ´«getb()«´ ´ «getc()«endl;}
};
int main (void)
{ Der2 obj(1,2,3); obj.show( ) ;
cout « obj.geta ( ) «´ ´« obj.getb( ) « endl ; return 0;}
```

Базалық кластың ашық мүшелерінің мұрасына ие болғанда олар туынды кластың ашық мүшелеріне айналады. Сондықтан, егер geta Base1 класына мұрагерленсе, онда get a ( ) функциясы Der1 класының ашық мүшесіне және кейін Der2 класының ашық мүшесіне айналады.

Туынды класс тікелей екі базалық класқа мұрагерленетін алдыңғы бағдарламаның қайта өндірілген нұсқасын қарастырайық:

```
# include <iostream.h> class Base1
{ int a;
public: Base1(int x) {a=x; } int geta () { return a; }
} ;
class Base2
{ int b;
public: Base2 (int x) {b=x;} int getb () {return b;}
};
class Der :public Base1, public Base2
{int c;
public: Der(int x,int y,int z):Base1(z),Base2(y)
{ c=x;}
void show ()
{ cout «geta () « ´ ´«getb() «´ ´ «getc()«endl;}
};
int main (void)
```

{Der obj (1,2,3); obj.show ( ) ; return 0;}

Келесі бағдарламада туынды класс тікелей бірнеше базалық класқа мұрагерленетін кезде конструкторлар мен деструкторлар шақырылатын тәртіпті көрсетейік:

```
# include <iostream.h> class Base1
{ int a;
public: Base1() { cout «  Base1 класс конструкторының жұмысы\ n"; }
~ Base1() { cout « " Base1 класс деструкторының жұмысы\ n"; } int x) {a=x; }
int geta () { return a; }
} ;
class Base2
{ int b;
public: Base2 () { cout «" Base2 класс конструкторының жұмысы\ n"; }
~ Base 2(){ cout «" Base2 класс деструкторының жұмысы\ n"; }
};
class Der :public Base1, public Base2
public:Der() { cout«" Der класс конструкторының жұмысы\ n"; }
~Der () { cout «" Der класс деструкторының жұмысы\ n"; }
};
int main (void)
{ Der obj; return 0;}
```

Бірнеше базалық кластар тікелей мұрагерленсе, конструкторлар тізіммен берілетін тәртіпте солдан оңға қарай шақырылады, ал конструкторлар кері тәртіпте, сондықтан бағдарлама мынаны экранға шығарады:

- Base1 класының конструкторының жұмысы;
- Base2 класының конструкторының жұмысы;
- Der класының конструкторының жұмысы;
- Der класының деструкторының жұмысы;
- Base2 класының деструкторының жұмысы;
- Base1 класының деструкторының жұмысы.

Виртуалды базалық кластар.

Туынды класс бір базалық класқа бір реттен артық жанама түрде мұрагерленсе және базалық класс барлық туынды кластар үшін виртуалды сияқты мұрагерленсе туынды класс объектісіндегі базалық кластың екі көшірмесінің пайда болуын болдырмауға болады. Мұндай мұрагерлік базалық класқа мұрагерленетін кез келген келесі туынды класта базалық кластың екі (немесе одан да көп) көшірмелерінің пайда болуына жол бермейді. Мұндай жағдайда базалық класқа өту жолының спецификаторының алдына virtual шешуші сөзін қою керек.

С++ тілінің синтаксисі туынды класқа базалық класты тікелей бірнеше рет беруге тыйым салады:

```
Class A {…. };
```

Class B: A,A{….}; // қате
Сонымен бірге базалық класты жанама түрде беруге тыйым салынады: Class A {….};
Class B: public A {….}; // қате Class C: public A, public B {….} // қате

«Қарапайым» мұрагерлік кезінде туынды класс объектісінің құрамына базалық кластың ішкі объектісі кіреді, ал виртуалды мұрагерлік кезінде туынды класс объектісінің құрамына виртуалды базалық кластың ішкі объектісіне жасырын көрсеткіші енеді, оны компилятор виртуалды базалық кластан мұрагерленген мүшелерге жол беруге арналған объектімен жұмыс барысында айқынсыз қолданады.

Class A {…. };
Class B: virtual public A {….};
Class C: virtual public A, public B // осылай жазуға болады
{….};

Мысалы, Der3 класында Base класының екі көшірмесін болдырмайтын виртуалды базалық класс қолданылады:

include<iostrem.h> class Base
{public : int i ; };
class Der1 : vertual public Base
{public: int j ; };
class Der2 : vertual public Base
{public: int k ; };
class Der3 : publicDer1, public Der2
{public: int f( ){return i*j*k;}} ; int main(void)
Der3 obj ;

//Бір мағынасыздық болмайды, себебі Base класының тек бір көшірмесі ұсынылған.

obj. i=10; obj.j=3; obj.k=5;
cout « ″ нәтежиесі″ « obj .f( ) « endl; return 0;
}

Бақылау сұрақтары:
1) Кластар иерархиясы қалай анықтауға болады?
2) Объектер иерархиясында хабарламаларды өңдеудің сұлбасы қандай?
3) Егер базалық кластың мүшелері жаңа қайта туынды класта анықталған атаулардың мұрасына ие болса, онда оларға туынды кластан рұқсат беріле ме?
4) Рұқсат етудің қандай спецификаторларын объектілер мұрагерлік иерархиясында хабарламаларды өңдегенде қолдана алады?

ДӘРІС №8. ПОЛИМОРФИЗМ

Дәрістің мақсаты: кластың үш қасиетінің бірі полиморфизмды түсіндіру.

Дәрістің мазмұны: кластардың виртуалды әдістері. Виртуалды деструкторлар.

Объектіге бағытталған программалаудың (ОБП) үшінші принципі полиморфизм («көп пішімдік») болып табылады. Полиморфизм (опера- цияларды, функцияларды қайта жүктеу, кластар мен функциялардың мүшелері) операцияның, кластың немесе функцияның бір аттас деректердің түрлі типтері үшін қолданылады.

С++ тілінде полиморфизм екі формаға ие болады. Статикалық байланыстыру механизмі көмегімен жүзеге асырылатын функциялар мен операцияларды қайта жүктеу. Оған кластар мен функциялардың үлгілері қатысады.

Статикалық байланыстыру операция, кластың, функцияның нақты данасын анықтау бағдарлама компиляциясының сатысында орындалады(осы байланыстыру) .

Виртуалды функцияларды қолдану, бұл динамикалық байланыстыру механизмі арқылы жүзеге асырылады. Бұл форма ОБП негізгі принциптерінің бірі сияқты полиморфизмнің бір формасы болып табылады.

Динамикалық байланыстыру статикалықпен салыстырғанда виртуалды функцияның нақты данасын анықтау компиляция сатысында емес, программа орындаған кезде орындалатындығын білдіреді.

Кластардың виртуалды әдістері.

Компилятордан қандай да бір класс әдісін шақырғанда шақырылатын класс атын компиляция сатысында оған сәйкес машиналық кодпен байланыстырудың екі әдісі бар.

Бірақ егер класс әдісін шақыру үшін аты бір және бірдей аргументтер жиыны бар базалық және туынды кластары бар класқа деген көрсеткіш қолданылса (сигнатурасы бірдей), онда компилятор берілетін объект қай класқа қатысты екендігін және ол үшін қандай әдісті шақыру керектігін анықтай алмайды.

Бұл коллизияны шешу үшін С++ тілі виртуалды функцияларды қолдану мүмкіндігін ескереді. Егер қандай да бір класта virtual сияқты сипатталған әдіс бар болса, онда мұндай класқа виртуалды функциялар кестесіне жасырын мүше-көрсеткіш қосылады, сонымен қатар компиляция кезінде емес, программа жұмысы кезінде деректер тибңнің объектісі үшін сәйкес келетін виртуалды әдісті таңдауға мүмкіндік беретін арнайы кодты басқарады (кейін әдісті біріктіру және динамикалық түрде әдісті байланыстыру).

Виртуалды функция (virtual fuction) базалық кластың ішінде жарияланады және туынды класта қайта анықталады. Негізінен, виртуалды функция полиморфизм негізінде жатқан «бір интерфейс», «көптеген әдістер» ойларын жүзеге асырады. Виртуалды функция базалық кластың ішінде осы функцияның интерфейсі түрін анықтайды. Әрбір виртуалды қайта анықталуы туынды класта оның спецификациясымен байланысты жүзеге асырылуын анықтайды. Сөйтіп қайта анықтау нақты әдісті құрады.

Егер екі немесе одан да көп түрлі кластар виртуалды функция құрамына жататын базалықтан туындылар болып табылса, онда, егер базалық кластың нұсқауышы осы туынды кластардың түрлі объектілеріне сілтеме жасаса, виртуалды функциялардың түрлі нұсқаларын орындайды. Себебі компилятор нұсқауыш сілтеме жасайтын объект типіне негізделе отырып виртуалды функция нұсқасын анықтайды. Бұл үдеріс динамикалық полиморфизм принциптерін жүзеге асыру болып табылады.

Виртуалды функциялардың иерархиялық тәртібі бар программа мысалын қарастырайық:

```
# include<iostream.h> class Base
{public : int i ; Base (int x) {i=x;}
Vertual void func ( )
{cout «″ базалық кластың func ( ) функциясын орындау″; cout « i « endl ;}
};
class Der1 :  public Base
{public: Der1 (int x): Base (x) {} void func( )
{cout″ Der кластың func ( ) функциясын орындау: ″; cout « i*i « endl;}
};
class Der2 :  public Base
{public: Der2 (int x): Base (x) {}
func ( ) функциясы ауыстырылмайды
};
int main(void)
Base *p; Base obj (10); Der1 d1_obj(10); Der2 d2_obj(10); p=& obj;
p- func( )                          // базалық кластың func ( ) функциясы
p=&d1_obj;
p- func( ) ;                        // Der1 туынды класының func ( ) функциясы
p- func( )                          // базалық кластың func ( ) функциясы return 0 ;
}
```

Егер виртуалды функция туынды класта қайта анықталмаған болса, онда базалық кластан оның нұсқасы қолданылады. Объект көрсеткіші арқылы адрестелетін тип виртуалды функциямен ауыстырылатын белгілі бір нұсқасын шақыруды анықтайды. Бұған қарсылық ретінде виртуалды емес нұсқауыш арқылы шақырылатын интерпретация нұсқауыш типіне ғана тәуелді.

Бірқатар виртуалды әдістермен жұмыс істеудің негізгі ережелерін қарастырайық:

1) Виртуалды болып кез келген функциялар емес, тек қандай да бір клатың компоненттік функциялары табылады.

2) Функция виртуалды болып анықталған соң, оның туынды класта қайта анықталуы (сигнатурасы бірдей) бұл класта жаңа виртуалды функцияны құрады, сонымен виртуал спецификаторы енді қолданылмауы мүмкін.

3) Туынды класта аты мен сигнатурасы бірдей функцияны, бірақ базалық кластың виртуалды функциясымен салыстырғанда нәтижесінің басқа типімен анықтауға болмайды.

4) Егер виртуалды әдіс туынды класта қайта анықталмаса, онда осы туынды класс объектісі үшін оны шақырған кезде ол анықталған базалық кластың иерархиясы бойынша жақын виртуалды сәйкес әдіске қатынасатын болуы керек.
5) Туынды класс мүшесі базалық кластың виртуалды әдіс атымен сай келетін әдіс аты, бірақ сигнатурасы басқа болуы мүмкін. Онда ол басқа виртуалды емес әдіс болады.
6) Виртуалды әдістер мұрагерленеді, яғни олар туынды класта қайта анықтау айырмашылығы бар әрекеттерді беру керек болған кезде ғана талап етіледі. Қайта анықтау кезінде рұқсат ету құқықтарын өзгертуге болмайды.
7) Егер виртуалды әдіс туынды класта қайта анықталған болса, онда осы класс объектілері көріну облысына (:: операциясы) рұқсат ету операциясы көмегімен базалық кластың виртуалды әдісіне өту рұқсатын ала алады.

Виртуалды деструкторлар.

Класс конструкторы виртуалды болуы мүмкін емес, себебі ол типі белгілі объектті құрған кезде ғана шақырылады. Деструктор виртуалды бола алады. Оның қажеттілігі базалық класқа нұсқауышпен адрестелетін объекттің бұзылуымен байланысты. Егер бұл нұсқауыш өзінің жеке деструкторы бар туынды класс объектісіне сілтеме жасаса, онда көрсетілген объекттердің корректті бұзылу мәселесі виртуалды әдістерге ұқсас виртуалды деструкторлармен шешіледі. Егер осындай жағдайда деструктор виртуалды сияқты жарияланса, онда барлығы дұрыс орындалып, сәйкес туынды кластың деструкторы шақырылатын болады. Кейін туынды класс деструкторы автоматты түрде базалық класс деструкторын шақырады және белгіленген объект тұтастай жойылады. Одан мына ереже шығады: егер класта виртуалды әдістер жарияланса, онда деструктор да виртуалды болу керек.

Бақылау сұрақтары:
1. Туынды класс объект үшін қандай да бір класс әдісін шақыруға бола
2. Егер жұмыс көрсеткіштері қолданумен жүргізілсе, онда кластар
3. иерархиясында виртуалды әдістер не үшін керек?
4. Қайта жүктелген әдіс құрамына үнсіздік бойынша берілген параметрлер ене ала ма?
5. Қандай жағдайда класс әдістерін виртуалды жасау керек емес?
6. Қандай түрде C++ тілінде полиморфизм жүзеге асырылады?

ДӘРІС №9. АБСТРАКТЫЛЫ КЛАСТАР

Дәрістің мақсаты: абстрактты кластар түсінігін беру.

Дәрістің мазмұны: Локальды кластар. Класс құрылымы бойынша жасалатын ұсыныстар.

Ереже бойынша базалық класта жарияланған виртуалды функция ешқандай маңызды әдістерді орындамайды. Бұл әдеттегі жағдай, себебі жеке түрде базалық класта деректердің аяқталған типі анықталмайды. Оның орнына тек мүше функциясының базалық жиыны мен жетпейтіндердің бәрін туынды класс анықтайтын айнымалылар болады. Базалық кластың виртуалды функциясында негізгі әрекет болмаған кезде осы базалықтан кез келген туынды класта мұндай функция міндетті түрде қайта анықталған болуы керек. Оны C++ тілінде жүзеге асыру үшін таза виртуалды (pure virtual function) деп аталатындар қолдау табады.

Таза виртуалды функциялар абстрактты әдістер базалық класта анықталмайды. Оған тек қана осы функциялардың прототиптері енеді. Таза виртуалды функция үшін осындай негізгі форма қолданылады:

аты _типі (параметрлер_ тізімі) = 0

Функцияларды нөлге теңестіру компиляторға базалық класта осы функцияның денесі жоқ екендігін хабарлайды. Мұндай жағдайда ол әрбір туынды класта міндетті түрде ауыстырылып отыруы керек. Әйтпесе компиляция кезінде қате туындайды.

Ең болмаса бір абстрактты әдісі бар класс абстрактты деп аталады.

Абстрактылы класс басқа кластар үшін базалық ретінде қызмет ете алады және абстрактылы класс объектісін құру мүмкін емес. Абстрактылы кластан туынды кластар, абстрактты әдістер анықталуы керек немесе абстрактты сияқты қайта жариялануы керек.

Абстрактты класс бар деп есептейік:

```
Class Base
{ protected :virtual void f(char)=0; void func(int);
};
```

Base абстрактты класс негізінде туынды класты түрінде құруға болады: Class Der1:public Base

{….void f (char); }; Class Der2:public Base

{….void func (int); };

Der1 класында f ( ) абстрактты әдісі осы типтің нақты виртуалды функциясымен ауыстырылған. Base: : func ( ) функциясы Der1 класымен мұрагерленеді және оның әдістері мен қатынауын мұрагерленеді. Der1 класы абстрактты емес. Der2 класында Base:: func ( ) функциясы қайта анықталған,

ал Base:: f () виртуалды функциясы мұрагерленген. Сонымен бірге класс Der2 абстрактылы болады және базалық ретінде ғана қолданылады.

Әрбір клас сияқты абстрактты класс анықталған конструкторға ие болуы мүмкін. Конструктордан кластар әдістерін шақыруы мүмкін, бірақ кез келген тура немесе тура емес таза виртуалды функцияларға қатынасы бағдарламаны орындаған кездегі қателерге әкеледі.

Кейін нақтылауға ұсынылатын пәндік облыстың жалпы түсініктерін беру үшін абстрактылы кластар механизмі өндірілген. Бұл жалпы түсініктерді әдетте тікелей қолдану мүмкін емес, бірақ құруға жарамды жеке туынды кластарды құруға болады. Мысалы, абстрактты «Фигура» класынан
«Үшбұрыш», «Шеңбер» және тағы басқа кластарды құруға болады.

Абстрактылы әдістер кластармен жұмыс істеуге арналған анағұрлым маңызды ережелерді құрайық:

1) Әдіс параметрінің типі, мәнді қайтаратын әдіс сияқты абстрактылы класс бола алмайды.
2) Егер абстрактылы базалық класқа оның инициализациясы үшін уақытша объектті құру талап етілмесе, онда көрсеткіш құруға, сонымен қатар осындай класқа жасалатын сілтемеге рұқсат етілген (және ол жиі қолданылады).
3) Абстрактылы класс әдістері осы кластың абстрактылы әдістерін шақыра алады. Осындай жағдайда объект типіне сәйкес келетін туынды класта анықталған әдіс шақырылатын болады.
4) Егер абстрактылы кластан туынды класта барлық абстрактылы әдіс функциялары шақырылса, онда туынды класс соған қоса абстрактылы болып табылады.
5) Абстрактылы кластан кез келген туынды класс одан абстрактылы әдістерін мұрагерленеді. Туынды кластың объектісін құру мүмкіндігін алу үшін онда барлық абстрактылы әдістерді қайта анықтау керек.
6) Абстрактылы класс конструкторлар мен деструкторларға ие бола алады.

Абстрактылы кластың конструкторы туынды класс объектілерін құрған кезде, ал деструктор олардың қатынауы кезінде шақырылатын болады. Базалық класс деструкторы туынды кластарда анықталған «ішкі объектілер» бұзылғаннан кейін шақырылады. Сондықтан абстрактылы базалық класс деструкторы өз класының абстрактылы әдістерін шақырмау керек, себебі мұндай шақыру бағдарламаны орындағанда қатеге әкеледі.

Локальды кластар.

Класс блоктың ішінде берілуі мүмкін, мысалы функция анықтамасының ішінде. Мұндай класс локальды деп аталады. Класс локализациясы кластың анықталу облысынан тыс оның компаненттеріне байланысты қатынамауды тұжырымдайды (ол анықталған немесе сипатталған функция немесе блоктың денесінен тыс).

Локальды класс статикалық деректерге ие бола алмайды, себебі локальды кластың компоненттері класс мәтінінен тыс анықталуы мүмкін емес.

Локальды кластың ішінде оның айналасындағы облысынан тек типтер атауларын, статикалық (static), сыртқы (extern) айнымалыларды, сыртқы функциялар мен санауыш элементтерін қолдануға рұқсат етілген. Локальды кластардың компоннетті функциялары тек енгізілген (inline) болуы мүмкін.

Класс құрылымы бойынша жасалатын ұсыныстар.

Әдетте класс қолданушы типі сияқты құрамына жасырын (private) мүше

- деректердің саны және келесі әдістері кірмейді:
- класс объектілерін инициализациялаудың әдістерін анықтайтын конструкторлар және қажет болғанда динамикалық жадыны резирверлеуді қамтамасыз ететін конструкторлар;
- көшіріп алу конструкторы (егер класс динамикалық жады үшін қолданылса қажет болады);
- деструктор (егер класс динамикалық жады үшін қолданылса қажет болады);
- класс қасиеттерін жүзеге асыратын әдістер жиыны (бұл кезде кластың private – мүше – деректерінің мәндерін қайтаратын әдістер өрістер мәндерін өзгерте алмайтындығын көрсететін const модификаторымен сипатталуы керек);
- класта қажет етілетін объекттерді салыстыруға, өзіне алуға, арифметикалық және басқа әрекеттерді орындауға мүмкіндік беретін операциялар жиыны;
- қателер туралы хабарламалар үшін қолданылатын ерекшеліктерді өңдеу құрылғылары.

Класты жобалаған кезде келесі сұрақтар өте маңызды болып табылады.

Кластың мүше - деректер құрылымын және олардың тиесілігін қалай анықтайды? Класс әдістері мен тиесілігін қалай анықтайды?

Егер берілген класс әдістермен ғана қолданылса, онда оны жапқан жөн болады. Егер берілген класс пен оның туындыларында қатынау қажет болса, онда оны қорғау керек. Мүше - деректерді ешқашан ашпаған жөн, себебі класс әлсіз болып қалады, ал бұл программалық кодты сенімсіз етеді.

Класс пен қолданушы кластың интерфейсті әдістері мен деструкторлары, конструкуторларының барлық түрлері көмегімен байланыса алады, сондықтан оларды кез келген программалық ортадан рұқсат етілген болуын жасау керек. Көмекші интерфейсті емес, егер олар тек осы класта, немесе қорғау (protected), егер олар осы және одан да туынды кластарда керек болса әдістерді жабық (private) ету керек.

Бақылау сұрақтары:

1) Виртуалды функция дегеніміз не?
2) Қандай функциялар виртуалды бола алмайды?
3) Қалай виртуалды функциялар динамикалық полиморфизмді жүзеге асыруға көмектеседі?
4) Абстрактылы класс дегеніміз не?
5) Полиморфты класс дегеніміз не?

ДӘРІС №10. ӘРЕКЕТТЕР АЙМАҚТАРЫ МЕН АТАУЛАР КЕҢІСТІКТЕРІ

Дәрістің мақсаты: әрекеттер аймақтары мен атаулар кеңістіктерін ашып көрсету.

Дәрістің мазмұны: блок. Функция прототипі. Функция. Файл. Класс. Атаулар кеңістігі.

Әрбір программалық объекттің оның түрі мен орнын анықтайтын әрекет ететін аймағы мен өміршең уақыты бар. Әрекеттер аймағының келесідей түрлері бар:

- блок;
- функция прототипі;
- функция;
- файл;
- программалық жобаның барлық файлдарын қамтитын шеңберіндегі файлдар тобы (әрекеттердің глобальды аймағы);
- класс;
- атаулар кеңістігі (әрекеттердің глобалды аймағының бөлігі).

Программалық объекттің әрекеттер аймағының барлық бес категориясын қысқаша қарастырып өтейік.

Блок. Блок ішінде анықталған объект локалды болып саналады. Осындай обьекттің әрекеттесуінің аймағы анықталу нүктесінде анықталады және блок аяғында аяқталады. Сақтау класы обьектісінің (автоматты) өмір сүру уақыты оның анықталу мерзімінен басталады және блок жұмысы біткеннен кейін аяқталады. Static (статикалық) спецификаторы бар блок объектісі өз мәнін блоктың аяқталуынан кейін сақтайды, ал оның өмір сүру уақыты бағдарламаның орындалу уақытымен сәйкес келеді.

Функция прототипі. Функцияның прототиптер (хабарламалар) параметрлер тізімінде көрсетілген идентификаторлардың әрекеттер аймағы ретінде тек функция прототипі бар. Сондықтан функция параметрлерінің туынды идентификаторларын қолдануға болады және олардың мүлдем жіберіп қоюға болады.

Функция. Функция блогында анықталған бағдарламалық объектілер қалыпты блокта сияқты әрекеттер аймағы және өмір уақытына иеленеді. Мән бойынша берілетін функция параметрлері әрекеттер аймағы ретінде бүкіл функциясы бар және өмір уақыты - функциясының орындалу уақыты.

Сілтеме бойынша берілетін функция параметрлері функция шақырылуына сәйкес аргументтермен анықталатын өмір уақытымен әрекеттер аймағына, әрине, функция блогы да кірмейді.

Файл. Атаулар кеңістігі немесе класс, функция, кез келген блоктан тыс static сақтау класының сипаттауышын қолдану көмегімен анықталған программалық объект анықтау нүктесінде басталатын және файл соңында

аяқталатын әрекеттер аймағына ие болып табылады. Егер құрамында идентификаторы бірдей болып табылатын қайта анықталған программалық объект болмаса, онда әрекеттер аймағына енгізілген (ішкі) блоктар қосылады. Егер енгізілген блокта бірдей идентификатор, қайта анықталған объект бар болса, онда бұл жағдайда сыртқы объект енгізілген блокта көрінбейді. Оған, егер глобальды болса (атаулар кеңістігіне немесе кластан, блоктан тыс анықталған), «::» көріну аймағына өтуді рұқсат ету операциясының көмегімен қатынасуға болады. Мұндай объекттің өмір уақыты максималды және программаны орындау уақытына сәйкес келеді.

Программалық жобаның (әрекеттердің глобалды аймағы) барлық файлдарын шегіне қосатын файлдар тобы. Extern (сыртқы) класының сақтау класының сипаттаушысын қолдану көмегімен басқа файлдардағы және жарияланған, атаулар кеңістігі немесе класс, функция, блоктан тыс жобаның файлдарының біреуінде анықталған программалық объекттің әрбір осындай файлдарын жариялау немесе анықтау нүктесінде басталатын және файл соңында аяқталатын әрекеттер аймағына ие. Егер құрамында бірдей идентификаторы бар программалық объекттің өмір уақыты максималды және оны орындау уақытымен сай келсе, онда әрекеттер аймағына енгізілген (ішкі) блоктары қосылады.

Класс. Статикалық класс мүшелерінен басқа обьект кластарының мүшелері әрекеттер аймағын иеленеді. Бұл олар класс ішінде ғана көрінетінін білдіреді. Объект кластар мүшелерінің өмір уақыты – объект класын құру мерзімінен бастап оның бұзылу мерзіміне дейінгі уақытта анықталады. Берілген класс құрамын сипаттау, кәдімгі статикалық айнымалылар мен функциялардан айырмашылығы сияқты көп файлдық жобаның барлық файлдарында қолжетімді деректердің статикалық мүшелері және әдістер глобальды болып табылады.

Деректердің статикалық мүшелері кәдімгі глобалды айнымалылар сияқты жоба файлдарының біреуінде міндетті түрде анықталуы тиіс. Деректердің статикалық мүшелері және функция-мүшелер бір объекттіні құрғанға дейін де рұқсат етілген болады. Оларға «::» операторы арқылы қатынасуға болады.

Атаулар кеңістігі. C++ тілі name space операторы арқылы атаулардың әрекеттер аймағын глобаьлды бөлігі сияқты нақты түрде беруге мүмкіндік береді. Әрбір әрекеттер аймағында атаулар кеңістігі деп аталатындарды айырады. Атаулар кеңістігі – идентификатор әмбебап болып табылатын аймақ. Түрлі атаулар кеңістігінде идентификаторлар бірдей болуы мүмкін, себебіі сілтемелер шешуші бағдарламадағы идентификатор контексті бойынша жүзеге асырылады, мысалы:

```
struct Node
{ int Node; int I;
} Node;
```

Бұл жағдайда қарсылықтар жоқ, себебі тип атаулары, құрылым айнымалылары мен өрістері атаулардың түрлі кеңістіктеріне қатысты болады.

C++ тілінде атаулар кеңістігінің төрт түрі анықталған, олардың әрбіреуінің шектерінде идентификаторлар әмбебап болуы керек:

- қолданушымен анықталған типтер, функциялар, айнымалылар (объекттер) идентификаторларға қатысты атаулар кеңістігі және әрекеттердің бір

аймақтық шектеріндегі константалардың аталып өтуі. Функциялар идентификатоларынан басқаларының барлығы енгізілген блоктарда қайта анықталуы мүмкін;
- бірлестіктер, кластар, құрылымдар, атап өтулер типтерінің атауларын түзетін кеңістік. Бұл атаулар кеңістігінде әрбір осындай идентификатор бір әрекеттер аймағындағы шектерде әмбебап болуы керек;
- атаулардың бөлек атаулары әрбір кластың мүшелерін құрады. Класс мүшесінің аты класс ішінде әмбебап болуы керек, бірақ басқа кластардың мүшелерінің атауларымен бірдей болуы мүмкін;
- белгілер бөлек атаулар кеңістігін түзеді.

Атаулар кеңістігі. Атаулар кеңістігі (аталған аймақ) анықтамалар, хабарламаларды логикалық түрде топтастыру үшін және оларға қатынауды шектеу үшін қызмет етеді. Бағдарлама өлшемі неғұрлым үлкен болған сайын соғұрлым аталған аймақтарды қолдану өзекті. Түрлі атаулар кеңістігінің көмегімен атаулар қақтығысы мен сәйкес келу мүмкіндіктерінсіз басқа бағдарламалаушымен жазылған кодтан бір бағдарламалаушымен жазылған кодты анықтауға болады.
Атаулар кеңістігінің жариялануы (аталған аймақ) келесі форматтарға ие:

Namespace [аймақ _ аты] {/ *анықтамасы және хабарламасы */….}

Атаулардың бір кеңістігі бір рет жариялануы мүмкін емес, сонымен келешек хабарламалар алдыңғылардың кеңейтулері ретінде қарастырылады. Сөйтіп, атаулар кеңістігі бір файлдың шектерінен тыс өзгеруі және хабарлануы мүмкін.

```
Namespace demo
{int I=1;                          // объектті анықтау
int k=0;                          // объектті анықтау void func 1(int);        // функция прототипі void func2(int r){…}  // функция анықтамасы
}
// demo атаулар кеңістігі namespace demo
{
// int I=2 ;                  дұрыс емес - екі анықтама
void func1(double);                // дұрыс – функция прототипі (қайта жүктеу) void func2 (int);                    // дұрыс - функция прототипі
}
```

Атаулар кеңістігінің хабарламасына тек хабарламаларды логикалық түрде орналастыру болады, ал кейіннен олар аймақ аты мен «**::**» көріну аймағының қатынау операторы көмегімен анықталады, мысалы:

```
void demo:: func1 (int n) {…}
```

Мұндай әдіс интерфейстің бөлінуі мен жүзеге асырылуын қамтамасыз етеді (сөйтіп, атаулар кеңісігінің жаңа атын хабарлауға болмайды).

Егер атау жиі өз кеңістігінен тыс қолданылса, онда оны using операторы

арқылы мүмкін етуге болады:

Using demo **::** i ;

Одан кейін аймақтың нақты белгіленуінсіз (i) атауы қолданылуы мүмкін. Егер қандай да бір аймақтан барлық атауларды мүмкін ету қажет болса, using namespace операторы қолданылады:

using namespace demo ;

using және using namespace операторларын басқа аймақтан анықтамалар мен хабарламалардың мүмкін болуын жасау үшін аталған аймақтың хабарламасының ішінде де қолданылады.

Бақылау сұрақтары:

1) Атаулар кеңістігін қолдану міндетті ме?
2) Using және using namespace қолданудың арасында қандай айырмашылық бар?
3) Атаулардың аталған кеңістіктері дегеніміз не және олар не үшін керек?
4) using қызметші сөзін қолданбай атаулар кеңістігінде хабарланған идентификаторларды қолдануға бола ма?
5) std атауларының стандартты кеңістігі дегеніміз не?

ДӘРІС №11. СТАНДАРТТЫ КІТАПХАНА ҚҰРЫЛҒЫЛАРЫМЕН С++ ТІЛІНДЕГІ ЕНГІЗУ-ШЫҒАРУ.

Дәрістің мақсаты: С++ тілінің стандартты кітапханасындағы енгізу- шығару ағындары кластарын түсіндіру.

Дәрістің мазмұны: контейнерлі кластар, алгоритмдер және итераторлар кіретін стандартты кітапханасының бөлігі, үлгілердің стандартты кітапханасы.Тілдің операторларынан басқа кез келген программа құрудың интегрирленген ортасына қосылатын түрлі кітапханалардың құрылғылары қолданылады. С++ тілінің стандартты кітапханасын екі бөлікке бөлуге болады:

- С++ тілінің кітапханасынан мұрагерлік константалар, типтер, макростар, функциялар;
- стандартты кластар және С++ тілінің басқа құрылғылары.

С++ тілінің стандартты кластарын келесі топтарға бөлуге болады:

- сыртқы құрылғылар мен жедел жады арасындағы, сонымен қатар жедел жады шектерінде деректер ағынын басқаруға арналған ағынды кластар (дисктер, пернетақта, экран, монитор);
- символдық жолдары бар жұмыстың қателерінен қорғалған және қолайлы жолдық класс;
- контейнерлі кластар, алгоритмдер және итераторлар. Контейнерлі кластар деректерді сақтаудың кең таралған танымал құрылымдарымен: тізімдер, векторларды, көпмүшеліктерді және тағы басқа сақтауларды жүзеге асырады;
- стандартты кітапхана құрамына компоненттерді түрлендіретін және қолданатын алгоритмдер жатады. Итераторлар контейнерлі кластар элементтеріне унифицирленген қатынауды қамтамасыз етеді;
- жылжымалы нүктелі массивтер мен кешенді деректерді тиімді өңдеуге арналған математикалық кластар;
- типтер идентификациясы мен қателердің объекттіге бағытталған өңдеуге арналған диагностикалық кластар;
- жадыны динамикалық тарату, локальды ерекшеліктерге бейімделу және тағы басқалар үшін қалған кластар.

Алгоритмдер және итераторлар кіретін стандартты кітапханасының бөлігін үлгілердің стандартты кітапханасы деп аталатын (Standart Template Library,STL) контейнерлі кластар.

С++ тілінің стандартты кітапханасының ағынды кластарын қолдануға негізделген С++ тілінің енгізу-шығару құралдарын қарастырайық.

Ағын - қоректену көзінен қабылданушыға деректерді тасмалдауға қатысты түсінік. С++ тілінің ағындары деректер типінің стандартты және анықталған қолданушылармен сенімді жұмыспен қамтамасыз етеді, сонымен қатар бірегей және синтаксисімен түсінікті.

Ағыннан деректерді оқу оларды ағыннан алу деп аталады, ағынға шығару - ағынға деректерді қосу немесе орналастыру. Ағын байттар тізбегі ретінде анықталады және алмасу орындалатын нақты қондырғыға тәуелсіз болады.

Алмасу бағытына қарай ағындар кіріс (деректер енгізіледі), шығыс (деректер шығарылады) және екі бағытты деп бөлінеді.

Ағын жұмыс істейтін қондырғы түрлі бойынша ағындарды стандарты, файлдық, жолдық деп бөлуге болады.

Стандартты ағындар деректерді пернетақталардан дисплей экранына беруге арналған файлдық ағындар - магнитті дискідегі ақпаратпен алмасу үшін, ал жолдық ағындар - символдардың масивтерімен жұмыс істеу үшін қолданылады.

Ағындарды қолдану үшін С++ тілінің кітапханасына ios және streambur екі базалық класс негізінде құрылған кластар иерархиясы кіреді. Ios класынан енгізу – шығаруға арналған өріс пен әдістер кіреді. Streambur класы ағындардың буферизациясы мен оның физикалық қондырғылармен олардың әсерлесуін қамтамасыз етеді. Бұл кластардан stream класы кіріс ағындар мен ostream класы – шығыс үшін мұрагерленеді.

Бұл кластар екі бағытты ағындарды жүзеге асыратын iostream класы үшін базалық болып табылады. Кейін кластар иерархиясында файлдық және жолдық ағындар орналасқан.

Си тілінің енгізу-шығару стандартты функциясымен салыстырғанда ағындардың негізгі артықшылығы типтерді бақылау болып табылады, сонымен қатар кеңейтілуі, яғни қолданушымен анықталған типтермен жұмыс істеу мүмкіндігі болып табылады. С++ тілі енгізу-шығаруды жүзеге асыруға араналған құрылғылардың кең жиынын береді. С++ тіліндегі енгізу-шығару бұл - iostream класының “<<” және “>>” операцияларды қайта кең қолдану.

Енгізілген типтердің енгізу-шығаруды (стандартты) сәйкес кластар және объектілермен, соның ішінде шарт алдын ала анықталған объектілерден қолдау табады: cin, cout, cerr және clog.

Мысалы, cout«x; операторы шығару үшін iostream класының cout объектісіне x айнымалысының мәнін жібереді. Қайта жүктелген operator<< әдісі ол үшін шақырылған iostream объектісіне сілтемені қайтарады. Сондықтан шығару операциясының негізінде тағы бір шығару операциясын қолдануға болады, яғни шығарудың бірнеше операциясын бір тізбекке қосуға болады.

include<iostream.h>
int x; cout«″ =″ « x«″ \n″;

″ «″ операциясында солдан оңға қарай есептеу тәртібі бар екендігін ескеріп, мұндай жазба келесі түрде интерпретирленеді:

((cout. operator «(″ x=″)) operator « (x)). Operator « (“\n);

Берілген аргументтің типіне тәуелді operator<< әдісінің белгілі бір данасы шақырылады.

С++ тілінде шығуға ұқсас ену анықталады. Енгізілген стандартты тип үшін ″«″ операциясы қайта жүктелген анықталған iostream класы байланысты. Сонымен бірге operator<< әдісі iostream класс объектісіне сілтемені қайтарады, және енгізу кезінде енгізу операцияларын біріктіруге болатындығын білдіреді, мысалы, үш аргументті:

// cin .operator» (x)) . operator» (y) . operator» (z) ;

Енгізу және шығарудың нақты операциясын таңдау енгізілген немесе шығарылған деректердің әрбір типімен анықталады.

Енгізілген типтермен қатар C++ тілі жаңа типтерді (қолданушымен анықталған типтер) анықтауға мүмкіндік береді.

Ол үшін қайта жүктелетін функцияны қолданумен бірге әрбір қолданушы типі үшін “<<” және “>>” операциясын қайта жүктеу керек. Қайта жүктейтін әдіс “<<” немесе “>>” операциясы үшін міндетті түрде класқа дос болуы керек, бірақ кластың мүшесі болмау керек, себебі мұнда операциялардың операндалары түрлі кластан алынған.

Енгізу-шығару операцияларын қайта жүктеу бөліктерін көрсететін мысалды келтірейік:

```
Class Complex                    // класс кешенді сан
{ int real, image; public:
Complex ( int re, int im) {real=re; image=im;}
friend ostream & operator« (ostream &,const Complex &); friend istream & operator» (istream &, Complex &);
};
ostream & operator« (ostream & obj_out, const Complex &val_out)
{ return obj_out « val_out.real «″i″ « val_out.image «endl;}
// obj_out- шығарылатын объект , val_out шығарылатын мән Енгізу үшін ″>>″ операциясын қайта жүктеу.
istream & operator » (istream &obj_in, Complex &val_in)
{ obj_in» val_in.real>>val_in.image;
if(!obj_in) { cout «″ \n енгізудегі қате ″ « endl;exit(1);} return 0;
}
int main (void)
{Complex obj(12,21); cout«″ объект мәні: «″obj;
cout«″\n кешенді мәнді енгізуіңіз : ″ ; cin »obj; cout«″\n объект мәні: obj=“« obj;
return 0;
}
```

Ағынды кластарда форматтауды үш әдіспен: манипуляторлар көмегімен (ең қолайлысы), форматтау жалауларымен және форматтау әдістері көмегімен орындауға болады.

Ағынды кластар құрамына дәлдігі әртүрлі, белгілі бір сандар жүйесіне мәндерді енгізу немесе шығаруға мүмкіндік беретін құралдар кіреді.

Шығарылатын мәндер шығару өрісінің сол немесе оң жақ соңына қысуға болады. Форматталған енгізу-шығару құрылғылары енгізу-шығару форматының басқа да бөлшектерін басқаруға мүмкіндік береді. Берілген мүмкіндіктерді жүзге асыруға, анағұрлым қолайлы әдістерінің бірі манипуляторларды қолдану болып табылады.

Манипуляторлар, форматталған енгізу-шығаруды кодтауды жеңілдететін, қайта жүктелетін әдістерді қолданады. Мапипуляторларды - “<<” және “>>” операциялары көмегімен енгізу-шығару тізбегіне енгізуге болады. Манипуляторлар, параметрленген болады, яғни бір аргументі, бар функцияны шақыру түрінде жазылады, және қарапайым яғни манипулятор атынан кейін аргументі бар дөңгелек жақшалар болмайды. Параметрленген манипуляторларды қолдану үшін бағдарламаға <iomanip.h> файлын қосу керек.

Бақылау сұрақтары:

1) cerr мен clog арасында қандай айырмашылықтар бар?
2) Енгізудің қайта жүктелген операторы дегеніміз не және ол қалай жұмыс істейді?
3) Шығарудың қайта жүктелген операторы дегеніміз не және ол қалай жұмыс істейді?
4) Шығарудың қайта жүктелген операторы қандай мәнді қайтарады?

ДӘРІС №12. ФАЙЛДЫҚ ЕНГІЗУ-ШЫҒАРУ

Дәрістің мақсаты: файлдық енгізу-шығару кластарын қарастыру.

Дәрістің мазмұны: istream, ostream, iostream, ofstream, fstream кластарының объектісі.

Файлдық және консольды енгізу-шығару өте тығыз байланысты. Нақты түрде файлдық енгізу-шығару консольды енгізу-шығару сияқты сол кластар иерахиясынан қолдау табады.

Программа өз жұмысын бастаған кезде, төрт ағын автоматты түрде инициализацияланады (жүктелінеді) және оларға сәйкес стандарттты файлдармен (қондырғылармен) байланысады – бұл cin, cout, cerr және clog. Егер басқа ағындарды инициялизациялау және кейбір файлдармен байланыстыру керек болса, онда мұны айқын түрде істеу керек.

Кез келген инициализацияланатын файлдық ағын енгізу ағыны немесе шығару ағыны немесе енгізу-шығару ағыны болу мүмкін. Енгізу ағыны бұл istream класынан туындаған, istream класының объектісі. Шығару ағыны – бұл ostream класынан туындаған, ofstream класының объектісі. Енгізу-шығару ағыны - бұл iostream класынан туындаған, fstream класының объектісі. Осы кластардың әрбіреуінде, класс объектісін сәйкес файлмен біріктіру жұмысын орындайтын конструкторы бар. Файлдармен жұмыс істеу үшін кластар, ағынды кластардан қайта жүктелген операцияларды ″«″және ″»″, жалауларды және форматтау әдістерін, манипуляторларды, ағындар күйін тексеру құрылғыларын және тағы басқаны мұрагерленеді.

Программада файлдарды қолдану келесі әрекеттерді тұжырымдайды:

1) Ағынды құру.
2) Ағынды ашу және оны файлмен байланыстыру.
3) Ағынмен алмасу (енгізу-шығару).
4) Файлды жабу.
5) Ағынды жою.

Параметрлері жоқ конструкторлар объектті файлмен байланыстырмай құрады. Параметрлері бар конструкторлар объект құрады, берілген режимде көрсетілген аты бар файлды ашады және файлды объектімен байланыстырады. Егер үнсіздік бойынша файлды ашу режимінің мәні программист үшін қолайсыз болса, онда операция арқылы, ios класында анықталған, биттік маскалардан құралған басқа мәнін орнатуға болады.

Ағынды құрған соң, оны файлмен байланыстыру әдістерінің бірі, ағынды кластардың барлық файлдарының мүшесі болып табылатын, open ( ) функциясы болып табылады. Егер open ( ) функциясының орындалуы қатемен аяқталса, онда бульдік өрнекте ағын false мәніне тең болады. Файлды жабу үшін қайтарылатын мәні мен параметрлері жоқ close ( ) функциясы колданылады.

Программалаудың негізгі, мынадай аспектілерін бейнелейтін бір тәжірибеде маңызды мысалды қарастырайық: ағынды енгізу-шығару; туынды класс конструкторынан, базалық класс конструкторына параметрлерді берген кезде,

класс үлгілерін қолдану; динамикалық жадыда орналастырылған екі өлшемді массивпен жұмыс. Келтірілетін программмада аспектілердің тек көрсетілген ерекшеліктері ғана қарастырылады, ал бақылау түрлері ескерілмеді.

```
// Файлдарды ашу/жабу үшін IOFile класын анықтау Class IOFile : public fstream
{ public : void open_f(char *filename, int mode ){ open (filename, mode ); void close
_f(char * filename){ close ( );}
};
// матрицалармен жұмыс істеуге арналған, базалық кластар үлгісі template<class T>
                                        // T-элементітің жалпы типі
class Matrix_B                  // класс-«Матрица» - базалық класс
{ protected:
T**array;                       // жолға көрсеткіштер массивінің көрсеткіші int rows,
columns;
public: Matrix_B (int n, int m);                        // конструктор
Matrix_B (const Matrix_B <T>&copy);                     // көшіру конструкторы
~Matrix_B (void);  // деструктор
void ReadMatrix (char*fp);                  // файлдан матрица элементтерін оқу
};
template<classT> Matrix_B<T>:: Matrix_B(int n, int m)
{
array=new T*[n]; // матрицаға сай жадының динамикалық бөлінуі for(int i=0; i<0; i+
+) array[i]=new T[m];
for(i=0; i<n; i++)
for(int j=0; j<m; j++) array[i][j]=(T)o; rows=n; columns=m;
return;
}
//                                  көшіру                                  конструкторы
template<classT>Matrix_B<T>::Matrix_B(constMatrix_B<T>&copy)
{  array=new  T*[copy.rows];  for(int  i=0;  i<copy.rows;  i++)  array[i]=new
T[copy.columns]; for( i=0; i<copy.rows; i++) for(int j=0; j<copy.columns; j++)
   array[i][j]=copy.array[i][j]; rows=copy.rows; columns=copy.columns;
}
template<class T>Matrix_B<T>::~Matrix_B(void)                   // деструктор
{ for(int i=0; i<rows; i++)             delete [] array[i]; delete [] array;
}
// берілген файлдан матрица элементтерін оқу template<classT>void
Matrix_B<T>::ReadMatrix(char*fp)
{ IOFile fin;               fin.open_f(fp,ios::in); for(int i=0; i<rows; i++)
for(int j=0; j< columns; j++) fin>>array[i][j]; fin.close_f(fp);
return;
}
// матрицалармен жұмыс істеу үшін туынды класс үлгісін анықтау template<classT>
class Matrix_D:public Matrix_B<T>
{ public: Matrix_D(int nrow, int ncol);
```

```
void PrintMatrix(IOFile &);                    // матрицаны басып шығару
//қосу, меншіктеу және индексациялау амалдарының жүктелуі операцияларын жүктеу
Matrix_D<T>operator+(const Matrix_D<T>&); Matrix_D<T>2operator=(const Matrix_D<T>&); T*& operator [ ] (int index);
// туынды класс конструкторы template<classT>
Matrix_D<T>: : Matrix _D (int nrow, int ncol): Matrix_B<T>( nrow, ncol){
}
// Қосу операцияларын қайта жүктеу template <class T>
Matrix_D<T> Matrix_D<T>: : operator + (const Matrix_D<T>&ref)
{ Matrix_D<T> result (rows, columns); // бос объектті құру for(int i=0; i<copy.rows; i++)
for(int j=0; j<copy.columns; j++)
result .array[i][j]=array[i][j]+ref.array[i][j]; return result;
}
// меншіктеу операциясын қайта жүктеу template<class T>
Matrix_D<T>& Matrix_D<T>: : operator = (const Matrix_D<T>&ref)
{
for(int i=0; i<rows; i++) for(int j=0; j<columns; j++) array[i][j]=ref.array[i][j];
rows=ref.rows; columns=ref.columns; return*this;
}
// матрицаны басып шығару template<classT>
void Matrix_D<T>: : PrintMatrix(IOFile &fout)
{
for(int i=0; i<rows; i++)
{for(int j=0; j<columns; j++) fout « array[i][j] « ;
fout « endl;
}
return;
}
// идексация операциясын қайта жүктеу template<classT>
T*& class Matrix_D<T>: : operator [ ] (int index)
{ return array[index];
}
// программаны тестеу
int main(void)                    //сәттілік кезінде, main  o  қайтарады
{ IOFile fout; // ″ matrix.out ″ файлына жазуға арналған объект; fout.open_f(“matrix.out”, ios::out);
// объект массивтерді құрамыз, оларды файлдан толтырамыз және басып шығарамыз
Matrix_D<int> a1(2,2), a2(2,2), a3(2,2);
```

```
a1.ReadMatrix( ″matrix.in ″ ); a2 . ReadMatrix( ″matrix.in ″); fout « ″\na1:  ″« endl; a1. PrintMatrix( fout) ;
fout « ″\na2: ″« endl; a2. PrintMatrix( fout) ; fout « ″\na3:  ″« endl; a3. PrintMatrix( fout);
//Қайта жүктелген [] - операцияларды тестеу
fout « ″\n қайта жүктелген [] - операцияларды тестеу″
″\n int k=a1[1][1];” «  endl ;
int k=a1[1][1]; fout « ″ \n k=” « k «endl;
// Қарапайым мәндерді тестеу
fout « ″\n Қарапайым мәндерді тестеу \n  ″
″a3=a1+a2;”«  endl;
″a3=a1+a2;
fout « ″\n a3= ”« endl; a3 PrintMatrix(fout);
// Шығару файлын жабу fout.close_f( ″matrix.out ″); return 0;
}
```

Бақылау сұрақтары:

1) Егер scanf ( ) - fscanf ( ), printf-fprintf-sprintf ( ) жиындары жақсы жұмыс істесе Енгізу-шығару ағындары не үшін құрылады?
2) ignore ( ) әдісін қашан қолдану керек?
3) Шығарудың қайта жүктелген операторы қандай мәнді қайтарады?
4) Бүтін сандарды шығарғанда, үнсіздік бойынша шығарудың өрісінің қандай ені қабылданады?
5) ios: : ate ашудың қарапайым режимі нені қамтамасыз етеді?

ДӘРІС №13. ЕРЕКШЕ ЖАҒДАЙЛАРДЫ ӨҢДЕУ

Дәрістің мақсаты: ерекше жағдайларды өңдеудің есептеу үдерісін түсіндіру.

Дәрістің мазмұны: өңдеудің жалпы механизмі және ерекшеліктер синтаксисі.

Ерекше жағдай немесе ерекшелік – бұл түрлі себептердің нәтижесінде туындайтын күтпеген немесе авариялық жағдайдың, оқиғаның пайда болуы. Мысалы, бөлінетін ресурстардың жетіспеушілігінен. Ерекше жағдайларды өңдеу құрылғылары көбінесе программа жұмысы кезінде туатын қателерді өңдеу үшін қолданылады. Сондықтан программа жұмысы кезінде туатын қате ерекше жағдайдың жеке түрі болып табылады. Ерекшеліктерді өңдеу пайда болған ерекшелікті тексеру (жеке жағдайдағы қате), бағдарламаның жұмысқа қабілетін қалпына келтіру және оның орындалуын жалғастыруға мүмкіндік береді.

Ерекше жағдайларды өңдеу есептеу процесін логикалық түрде екі бөлікке бөлуге мүмкіндік береді: авариялық жағдайды табу және оны өңдеу. С++ тілінің ерекшеліктерді өңдеу құралдарының басқа артықшылығы қате туралы хабарламаны беру үшін оның өңделуінің орнына қайтарылатын мәнді, параметрлерді немесе глобальды айнымалыларды қолдануды талап етпейді. Сондықтан функциялар интерфейсі таратылмайды.

Өңдеудің жалпы механизмі және ерекшеліктер синтаксисі.

Ерекше жағдайды өңдеу үш шешу сөз арқылы жүзеге асырылады: try, catch және throw.

Қате туындай алатын, бағдарламалық кодтың фрагменті, байқап көру деп аударылатын try-қызметші сөзі алдында жазылған, құрама оператор болып табылатын, бақыланатын блокқа кіру керек .

Егер қате try блогында бар болса, онда ол қоздырылынып (throw), содан кейін ұсталынып (catch) және өңделінеді.

Блок try-дан шақырылатын функцияларда, ерекше жағдайды туғыза алады. Кез келген ерекше жағдай, catch нұсқасымен ұстап алынуы керек, ал ол, ерекше жағдайды қоздыратын try блогынан кейін орналасу керек. try блогымен catch нұсқасының бірнешеуі байланыста бола алады. Онда қандай catch нұсқасы қолданылатыны ерекше жағдайына байланысты, қалған try блогының нұсқалары ескерілмейді. Программаны құратын кластары типтермен бірге кез келген типтерді қолдануға болады. Негізінен ерекше жағдайлар ретінде, жиі түрде класс типтері қолданылады.

throw нұсқасының негізгі формасы: throw ерекше_жағдай;

Нұсқа throw try блогының ішінде, немесе осы блок шақыратын (тура немесе жанама) кез келген функцияның ішінде орындалуы керек. Мұндағы ерекше жағдай – бұл throw ерекше жағдайымен қоздырылатын нұсқау.

Егер сәйкес catch нұсқасы үшін жоқ ерекше жағдайы қондырылса, онда программаның қалыпты емес аяқталуы болуы мүмкін. Standart С ++ стандартына

сәйкес өңделмейтін ереше жағдайдың қондырылуы terminate ( ) стандартты кітапханалық функцияның шақырылуына алып келеді. Үнсіздік бойынша бағдарламаны аяқтау үшін terminate функциясы abort ( ) функциясын шақырады. Ерекше жағдайды өңдеудің жай мысалы:

```
#include<iostream.h> int main(void)
{ cout «″ басы \n″;
try{                    // try блогының басы cout «″ try блогының ішінде \n″; throw
10                      // қатені қоздыру
cout «″ Бұл нұсқа орындалмайды! cout″;
} catch (int i)          //қатені ұстап алу
cout «″ қате номері қамтылған :  «″ i cout « endl;
}
cout «″ соңы ″; return 0;
}
```

catch нұсқауын орындағанан кейін, бағдарламаны басқару одан кейінгі келесі нұсқау беріледі. Оған қарамастан, әдетте catch блогы exit ( ), abort( ) немесе қандай да бір функцияны шақырумен бағдарламаны мәжбүр етіп аяқтайды.

Егер осы нұсқау try блогынан шақырылатын функцияға кірсе, онда ерекше жағдай try блогына кірмейтін нұсқамен де қоздырылуы мүмкін.

try блогын функцияның ішінде орналастыруға болады. Мұндай жағдайда, функцияға әрбір енген сайын ерекше жағдай өңдеушісі қайта орнатылады.

Кейбір жағдайларда жүйені барлық ерекше жағдайлардың типіне тәуелсіз, ұстап алатындай етіп баптау керек. Мұны өте оңай істеуге болады. Ол үшін catch нұсқасының келесі формасын қолданыңыз:

```
catch ( …) { //барлық ерекше жағдайларды өңдеу }
```

Мұндағы көпнүкте мәндердің кез келген типіне сәйкес келеді.

try блогынан шақырылатын функция үшін ерекше жағдай типтерінің санын шектеуге болады, оны осындай функция қоздыра алады. Нақтысында функцияға қандай қандай да бір ерекше жағдайды қоздыруға тыйым салуға болады. Ол үшін функцияның анықтамасына throw кілттік сөзін қосу керек:

```
Нәтиже_типі функция _аты (аргументтер_тізімі) throw (типтер_ тізімі)
{…}
```

Мұнда типтер тізімі өрісінде, функция қоздыра алатын, ерекше жағдайлардың мәндер типтері үтірмен бөлініп келтірілген. Басқа типті ерекше жағдайды қоздыруды байқап көргенде, бағдарламаның авариялық түрде аяқталуына алып келеді. Қоздыратын ерекше жағдайлар типтерін шекте, функцияның try блогынан шақырғаннан кейін мүмкін болатындығын түсіну керек. Яғни, функция ішінде try блогы ерекше жағдайдың кез келген типін қоздыра алады, себебі ол осы функция ішінде ұсталынады.

Ерекше жағдайды қайта қоздыруға болады. Мұның мағынасы, әрқайсысы ерекше жағдайды өзінің аспектісінен өңдейтін бірнеше процедураға, ерекше жағдайды өңдеуге мүмкіндік беру болып табылады. Ерекше жағдай, осы блоктан шақырылатын кез келген функцияда немесе catch блогының ішінде ғана қайта қоздырылады.

Бір типті ерекше жағдайды қайта өңдеу мысалы:

```
#include<iostream.h> void Exception ( )
{ try { // char * типті ерекше жағдайды қоздыру throw ″ сәлем ″;
}
// char* типті ерекше жағдайды ұстап алу catch (char*)
{ cout « ″Exception ( ) функциясының ішіндегі char*ұстап алу \n″;
throw;                // char* типті ерекше жағдайды қайта өңдеу, бірақ енді  Exception (
) функциясының ішінде емес.
}
}
int main(void)
{ cout «″ Басы \n″; try{   Exception ( );
catch (char*){ cout « ″соңы main ( ) функциясының ішіндегі char*ұстап алу\n″;}
cout «″ Соңы ″; return 0;
}
```

Бағдарлама экранға мынаны шығарады:

```
Басы
Exception ( ) функциясының ішіндегі char*ды ұстап алу main ( ) функциясының
ішіндегі char*ды ұстап алу Соңы.
```

New операторының қазіргі уақыттағы спецификациясына сәйкес, ол жадыдағы орын бөлу сәтсіз болған кезде ерекше жағдайды қоздырады. Жадыдан осындай сұраныс үшін, new операциясынан басқа стандартты құрылғылар, операция-функциялар operator new ( ) және operator new[ ] көмегімен енгізілетін қайта жүктелген операциялар болып табылады. Үнсіздік бойынша, егер new операциясы жадыдан қажет бөлігін бөле алмаса, онда ол нөлдік мәнді (NULL) қайтарады және қабаттас xalloc типті ерекшелікті құрады. xalloc типі - бұл except.h тақырыптық файлында анықталған класс.

Бақылау сұрақтары:

1) Ерекшелік дегеніміз не?
2) try блогы және catch операторы не үшін керек?
3) Ерекшелік қандай ақпаратты сақтайды?
4) Функциялар, глобальды айнымалылар және ерекшелік механизмінің қолдау кластары.
5) Ерекшеліктегі конструкторлар мен деструкторлар.

ДӘРІС №14. ДИНАМИКАЛЫҚ ИДЕНТИФИКАЦИЯ ЖӘНЕ ТИПТЕРДІ КЕЛТІРУ

Дәрістің мақсаты: динамикалық идентификация және типтерді келтіру түсініктерін беру.

Дәрістің мазмұны: const_cast, static _cast, dynamic_cast, reinterpret_cast операциялары.

Полиморфизмді қолдайтын тілдерде компиляция кезінде объекттің типі белгісіз болу жағдайлары мүмкін, себебі бағдарламаның орындалуына дейін объекттің нақты табиғаты анықталмаған. C++ -те полиморфизм кластар иерархиясы, виртуалды функциялар мен базалық класс көрсеткіштері арқылы жүзеге асырылады. Осындай байланыс кезінде базалық класс көрсеткіші, базалық класс объектісіне сілтеуге немесе осы базалық кластан туылған кез келген туынды объектісіне сілтеу үшін қолданылады. Әрбір уақыт мезетінде, базалық класс көрсеткіші сілтеме жасай алатын объект типін алдын ала анықтау мүмкіндігі бола бермейді. Мұндай жағдайларда объект типін анықтау бағдарлама орындалуы кезінде орындалуы керек, ал ол үшін типтің динамикалық идентификациясының механизмі қызмет етеді.

Объекттің типі туралы ақпаратты typeid (объект) операторы арқылы алады, ол объекттің объектті типін сипаттайтын type-info тип объектіге сілтеме қайтарады. type-info класында келесі ашық мүшелер анықталған:

```
Bool operator == (const type_info& объект);          // типтерді салыстыру Bool
operator! = (const type_info& объект);               // типтерді салыстыру Bool
before (const type_info& объект);
Const char*name ( );                                 // объект атына нұсқауыш
```

typeid операторы түрлі объект типтерін алуға мүмкіндік беруіне қарамастан, егер оның аргументі ретінде полиморфты базалық кластың көрсеткіші берілсе, онда ол анағұрлым пайдалы болады. Мұндай жағдайда, оператор автоматты түрде, көрсеткіш сілтеме жасайтын нақты объект типін қайтарады. Сөйтіп, typeid операторы көмегімен, программаманың орындалуы кезінде, базалық класс нұсқауыш сілтеме жасайтын объект типін анықтауға болады. Сонымен қатар бұл сілтемелерге де қатысты болып табылады. Егер typeid операторының аргументі ретінде, полиморфты класс объектісіне сілтеме берілсе, онда оператор, сілтемесі бар объекттің нақты типін қайтарады. typeid операторын полиморфты емес класқа қолданғанда, базалық типтің нұсқауышын немесе сілтемесін алады.

Типтің динамикалық идентификациясы әрқашан қолдана берілмейді, бірақ мәндердің полиморфты типтерімен жұмыс істегенде, ол әртүрлі жағдайларда өңделетін объектілер типтерін анықтауға мүмкіндік береді.

// typeid операторын қолдану мысалы:

```
# include<iostream.h>
```

```
# include<typeinfo.h> class Base
{ vertual void f ( ) { };                // класты полиморфты етеміз
};
class Der1 : public Base{…}; class Der2 : public Base {…}; int main(void)
{ int i;
Base *p,baseob; Der1 d1; Der2 d2;
cout «″ \n i айнымалысының типі -бұл ″  « tepeid (i). name( )  « endl;
// полиморфты типтерді өңдеу p=& baseob;
cout «″ p көрсеткіші тип объектісіне сілтеме жасайды ″; cout «typeid (*p ) name ( )
«endl;
p=&d1;
cout «″p көрсеткіші тип объектісіне сілтеме жасайды ″«typeid (*p ) name ( );
p=&d2;
cout «″\n p көрсеткіші тип объектісіне сілтеме жасайды ″ «typeid (*p ) name ();
return 0 ;
}
```

Бағдарлама экранға келесі ақпаратты шығарады:

```
i айнымалысының типі  - int
p нұсқауышы Base типті объектіге сілтеме жасайды
p нұсқауышы Der1 типті объектіге  сілтеме жасайды ″; p нұсқауышы Der2  типті
объектіге  сілтеме жасайды ″;
```

Объекттер функцияларға сілтеме бойынша берілетін жағдайда typeid операторының нақты қасиеті объекттің нақты типін қайтарады, мысалы:

```
oid func ( Base & ref)
{ cout «″ ref –сілтеме объектіге мына түрдегі «typeid (ret  ). name ( ) ;}
```

d1, d2 объектілері үшін шақыру кезінде олардың дұрыс идентификациясы орындалады:

```
func (d1) және func (d2 ) сәйкесінше.
```

Typeid операторының басты артықшылығы компиляция кезінде белгісіз объекттер типтерін идентификациялау мүмкіндігі болып табылады.

Экранда түрлі геометриялық фигураларды салу үшін арналған кластар иерархиясы анықтайтын бағдарлама мысалын қарастырайық. Иерархия басында төрт класты мұрагерленетін Shape абстрактты класы орналасқан: Line, Square, Rectangle және NullShape. Функция generator( ) объектіні туғызады және оған көрсеткішті қайтарады. Онда қандай объект құрылатынын, және rand ( ) - кездейсоқ сандық генераторы анықтайды. main ( ) функциясында, NullShape типті объекттерден басқа, экранда түрлі типті туындалған объектілер қортындысы жүзеге асырылған. Объекттер кездейсоқ туатындықтан, алдын ала, келесі қандай

объект құрылатыны белгілі болмайды. Нәтижесінде, құрылатын объектінің типін анықтау үшін типтің динамикалық идентификациясы талап етіледі.

```
# include<iostream.h> # include<cstdlib.h>
# include<typeinfo.h> class Shape
{ public : virtual void example( )=0;}; class Rectangle:public Shape
{public:void example( ){cout«″ Тікбұрыш\n″;} }; class Triangle :public Shape
{ public:void example ( ) { cout«″ \n Үшбұрыш \n ″;} }; class line : public Shape
{ public: void example( ) {cout «″\n Сызық\n″; } }; class NullShape
{ public :void example( ){ } };
// Shape класынан туындылар фабрикасы Shape *generator ( )
{ switch (rand ( ) %4)
{ case o: return new Line ; case 1: return new Rectangle; case 2:return new Triangle; case3:return new NullShape;
}
return Null;
} int main (void)
{ int i; Shape *p; for (i=0;i <10;i++)
{ p=generator( );                    // келесі объектіні құру cout «typeid (*p ) name «endl
// объектіні салады, егер ол NullShape типті емес болса if (typeid (*p )! =typeid (NullShape)) p--example( );
} return 0;
}
```

Typeid операторы үлгі-кластармен жұмыс істей алады. С++ тілінде жазылған бағдарламаны орындағанда типтер түрлері орындалады: нақты және нақты емес. Типтердің нақты түрленулері әрқашан программист қалауымен орындалады және ол үшін С++ тілінде мыналар бар:

- Си тілінен мұрагерленген типті келтіру операциясы;
- С++ тіліне енгізілген функционалды жазба формасы;
- const_ cast, dynamic_cast, reinterpret_cast және static_cast операциялары.

Соңғы операциялар типтердің түрленулерінің анағұрлым сенімді операциялары болып табылады, және оларды барлық қажет жағдайларда қолданған жөн.

const_cast операциясы. Берілген операция const модификаторының әрекеттерін нөлге айналдыру үшін қолданылады:

```
const int i;
int *p=const_cast<int*>(&i);                    // жай көрсеткішті аламыз Жалпы жағдайда
```

бұл операция келесі форматқа ие болады: const_cast <T> (V),

Мұндағы Т типі V типі (бұл әдетегі көрсеткіш) сияқты болу керек. Операция Т типті нәтижені қайтарады. Бұл операцияның қажеттілігі функцияны жобалағанда const сияқты өзгермейтін параметрлерді сипаттау міндетті емес болумен шатастырылған, бірақ мұны жасау ұсынылады. С++ ережелері константты көрсеткішті жай көрсеткіш орнына беруге рұқсат етпейді. Сондықтан const_cast

операциясы осы шектеуді айналып өту үшін енгізілген. Әрине, мұндай жағдайда функция берілетін көрсеткіш сілтеме жасайтын мәнді өзгертпеу керек.
Компиляция кезінде (тақырып, ортаға жаз) типтердің түрленуі (static _cast операциясы) static _cast операциясы компиляторымен орындалады және онсыз жұмыс істеуге болмайтын кезде ғана қолданылады. static_cast операциясын қолданудың мысалын көрсеткіштің қажет типіне void* көрсеткішін түрлендіру деп атауға болады, бүтін типін есептеп шығу типіне келтіру және туыс типтердің басқа да түрленуі деп те атауға болады.

Программаны орындау кезіндегі типтердің түрленуі (dynamic_cast) dynamic_ cast операциясы әдетте полиморфты объектілер үшін және виртуалды базалық кластардың түрленуін төмендету үшін қолданады. Бұл кезде, операция программаны орындау кезінде типтерді идентификациялау (Run-Time Type Information, RTTI) механизмін қолданады.

Бақылау сұрақтары:

1) typeid операторы көмегімен орындауға болатын опрацияларды сипаттаңыз.
2) Қандай объекттің типін typeid операторымен анықтайды?
3) const_cast типті түрлендіру операциясы не үшін керек?
4) dynamic_ cast типіті түрлендіру операциясы не үшін керек?
5) static _cast және reinterpret_cast операциялары не үшін керек?

ДӘРІС №15. С++ ТІЛІНІҢ СТАНДАРТТЫ КІТАПХАНАСЫ. ЖАЛПЫЛАНҒАН ПРОГРАММАЛАУ

Дәрістің мақсаты: С++ тілінің стандартты кітапханасы кластарының жұмысын ұғындыру.

Дәрістің мазмұны: контейнерлі кластар. Итераторлар мен функциональды объектілер.

Алгоритмдер.

С++ тілінің стандартизация үдерісі 1989 жылы басталып және 1998 жылы тіл стандартын басып шығарумен аяқтады.(Informating Technology- Programming Language-C++, нөмірі ISO/ IEC 14882-1998).

С++ тілінің ережелері мен синтаксисінің қатаң , құрылған анықтамалары тілге үйретуді, бағдарламаны жазуды, сонымен бірге олардың басқа платформаларға бейімделуін жеңілдетеді.

С++ тілінің стандартты кластарын былай бөлуге болады: ағынды кластар, жолдық кластар, контейнерлі кластар, алгоритмдер және итераторлар, математикалық, диагностикалық және қалған кластар.

Контейнерлі кластардан, алгоритмдерден және итераторлардан құралған стандартты кітапхана бөлігін үлгілердің стандартты кітапханасы деп атайды. (Standard Temlate Library, STL). Кітапханаға қазіргі және тиімді алгоритмдерді қолданып объекттер жиынымен (коллекциялармен) жұмыс істеуге арналған белгіленген құралдар кіреді. С++ тілінің стандартты кітапханасымен оған негізделген жалпыланған технологиясын қолдану, программистерге жұмыс принциптерін қарастырмай, мәндер мен алгоритмдер құрылымдары аймағында жаңа өндірулерді қолдануға мүмкіндік беріледі.

Контейнерлі кластар.

Контейнер – бұл белгілі бір түрде ұйымдастырылған басқа объектілер жиыны құрайтын объект. Контейнерлер анық типтің объект коллекцияларын басқаруға арналған. Контейнерлер мысалдары массивтер (векторлар және ассоциативті массивтер) және тізімдер (жекеленген тізімдер, кезектер, стектер) болып табылады. Контейнерге объектілерді қосуға болады және оларды одан жоюға болады. Контейнермен жұмыс, стандартты кітапханадағы контейнерлі кластар көмегімен қолдау табады. Бұл мүмкіндік класс үлгілері арқылы жүргізіледі. Контейнерлерді қолдану программалар сенімділігін жоғарлатуға, олардың тасымалдануымен бір мезгілдегі уақыттың кемуі мен өндіру құны бар әмбебаптылыққа мүмкіндік береді.

STL контейнерлі класы кезекті және ассоциативті деп бөлінеді. Кезекті контейнерлер үздіксіз кезектілігі түріндегі бір типті объектілердің соңғы санын анықтауда қамтамасыз етеді және келесі түрлері бар:

- векторлар (vector);
- екі жақты кезектер немесе басқаша айтқанда деректер (deque);
- тізімдер (list);
- стегтер (stack);
- кезектер (quere);

- п риоритеттері бар кезектер (priority-quere).

Ассоциативті кезектер сұрыптаудың белгілі критериі бойынша оның мәніне тәуелді объект(элемент) позициясы бар сұрыпталған коллекцияларды береді. Ассоциативті контейнерлер кілт бойынша деректермен тез қатынауды қамтамасыз етеді және балансталған ағаштар негізінде құрылған. Ассоциативті контейнерлердің келесі түрлері бар:

- сөздіктер (map);
- көшірмелері бар сөздіктер (multimap);
- жиындар (set);
- көшірмелері бар жиындар (multiset);
- биттік жиындар (bitset).

Түрлі контейнерлі кластардың ортақ қасиеттері көп және бұл өте қолайлы. Контейнерлі кластардың стандартталған интерфейсі бар. Бұл түрлі контейнерлі кластардағы бір атты деректер мүшелер мен операциялардың мағынасы бірдей екендігін білдіреді және контейнерлердің барлық типтеріне қолданылады.

Итераторлар мен функциональды объектілер

Итераторлар ұйымдастыру әдістері мен әрекеттер типіне тәуелсіз деректердің әрбір типіне қатынау және кезекті қарастыру құрылғылары болып табылады. Сөйтіп, итератор жалпыланған итератор болып табылады, және итератор мен көрсеткіш семантикасы бірдей STL –да итераторлар контейнерлі кластар, ағындар мен буферлі ағындармен жұмыс істеу үшін қолданылады.

Деректер түрлі түрде ұйымдастырылуы мүмкін (массив, тізім, ағаш және тағы басқа) әрбір кезектілік түрі үшін белгілі операциялар жиынын қолдайтын итератордың өз типі талап етіледі (кіріс, шығыс,тура, екі бағытты, және кезекті қатынау). Итераторлы адаптерлер бар, мыналарды қолдайтын алгоритмдерді орындауды қамтамасыз ететін арнайы итераторлар:

- элементтері кері тәртіпте мөлшерден тыс жинау (кері итераторлар);
- енгізу режимі (енгізу итераторлары);
- деректер ағынымен жұмыс (ағынды итераторлар).

Функционалды объект деп функцияны шақыру операциясы орындалған класс типі бар объектті атайды. Жиі түрде функционалды объекттер өңдеу әдістері мен оларды салыстырудың қолданушы контейнерлерін беруге арналған стандартты алгогритмдер параметрлері ретінде қолданылады. Параметр ретінде функционалды объектілер қолданылатын алгоритмдерде функцияға нұсқауышты қолдануға болады. Bool мәнін қайтаратын функциональды объектілер бар және предикаттар деп аталады. Предикат деп bool мәнін қайтаратын жай функцияны айтады.

Контейнерлердің өзі үлкен мәндерге иеленбейді. Пайдалы болу үлкен контейнер негізгі операциялармен жабдықталуы керек. Стандартты кітапхана контейнер қолданушысынан талап етілетін көптеген кең таралған операциялардың орындалуына арналған алгоритмдерді ұсынады. Осындай алгоритмдер саны алпыс шамасында.

Әрбір алгоритм функция үлгісі түрінде немесе функциялардың үлгілерінің жиыны түрінде жүзеге асырылған. Соның арқасында алгоритм кезекшіліктің әр түрімен жұмыс істей алады және түрлі типтердің деректерімен (жалпыланған

пролграммалаудың технологиясының концепциясы). Қолданушының нақты талаптарының алгоритмін баптау үшін функционалды объекттер қолданылады. Стандартты кітапхананың басқа құрылғылары сияқты стандартты алгоритмдерін қолдану программисті кезекшілікті өңдеу циклдерін құжаттау, түзету, жазудан айырады. Бұл бағдарламадағы қателерді кемітеді, өндіру уақытын қысқартады және анағұрлым оқылатын және жинақты етеді.

STL барлық алгоритмдерді бес категорияға бөледі:

- кезекшілігі бар модиицирленбейтін операциялар;
- кезекшілігі бар модиицирленетін операциялар;
- кезекшілікті сұрыптау алгоритмдері;
- көпшілікпен пирамидалармен жұмыс істеу алгоритмдері;
- жалпыланған сандық алгоритмдер.

Параметрлер ретінде алгоритмдерге өңделетін кезектіліктің басы мен соңын анықтайтын итераторлар беріледі. Итераторлар түрлері берілген алгоритм қолданылуы мүмкін контейнерлер типін анықтайды. Мысалы: сұрыптау алгоритмі sort ( ) еркін қатынау итераторларын қолдануды талап етеді. Сондықтан ол list контейнерлерімен жұмыс істей алмайды. Алгоритмдер кезекшілік шегінен шығуды тексермейді.

Бақылау сұрақтары:

1) Жалпыланған бағдарламалаудың технологиясының концепциясы?
2) Контейнерлер дегеніміз не, олар не үшін керек?
3) Итераторлар не үшін арналған?
4) Қандай операциялар итератордың кез келген типі үшін мүмкін болып табылады?
5) STL алгоритмдерінің негізгі категорияларын атап өтіңіз.

№1 ЗЕРТХАНАЛЫҚ ЖҰМЫС.

Есептелетін және есептелмейтін түріндегі қарапайым типтік есептер

Жұмыс мақсаты: Студенттерді енгізу – шығару операторларын пайдланып, С++ тілінде қарапайым программалар жазуға үйрету.

Студенттерге қойылатын талап: Кез келген студент берілген тапсырмаларға программалар жазып, ЭЕМ-де есептеп, нәтижесін алуы тиіс.

Жұмыстың орындалу тәртібі:

1. Берілген жұмыстың әдістемелік нұсқауымен танысу.
2. Берілген бақылау сұрақтарына жауап беру.
3. Жаттығуларды талдау.
4. Компьютерден нәтижені алу.

ӘДІСТЕМЕЛІК НҰСҚАУЛАР

“Берілгендерге операция жүргізу“ түсінігі объект (объектілер) барын және операция белгілерін пайдалануды қарастырады. Көбінесе меншіктеу операциясы қолданылады, оның белгісі мына түрде “=“ болады.

Мысалы:

f=125;
f=f+1;
k=a=b;

Бір объект қатысуымен орындалатын операция унарлық, ал екі объектімен -бинарлық деп аталады. Типтік арифметикалық операциялар (“+”-қосу, ”-” -айыру, ”*”-көбейту, “/”-бөлу) бинарлық болып табылады. С++ тілінде оң (++) және теріс (- -) объектілердің бірлік өсімшесінің унарлық операциялары пайдаланылады.

Мысалы:

sum=a+b++; (апостериорлық өсуі)
sum=a+ ++b; (априорлық өсуі)

Биттарға (екілік цифрлармен) операция жүргізу де, бинарлық және унарлық болып бөлінеді.

Бинарлық операция:

солға жылжыту	a=b<<c;
оңға жылжыту	a=b>>c;
ЖӘНЕ	a=b&c;
НЕМЕСЕ	b|c;
НЕМЕСЕ-ні болдырмау	a=b ^ c;

Унарлық операция:

ЖОқ	a=~b.

Меншіктеу операторы

Бұл базалық команда (оператор) жоғары деңгейлі барлық программалау тілдерінде келесі тІрде болады:

<аты> = <өрнек>;

Бұл команданың әртүрлілігі болып келесі табылады:

<аты>=<аты><белгі><өрнек>;

С++ тілінде соңғы жазбаны мына түрде беруге болады:

<аты><белгі>=<өрнек>;

Жоғарыдағы келтірілген командаларда “аты“- айнымалының аты, “белгі“- операция белгісі, “өрнек“- айнымалылар аттарының жиынтығы және (немесе) операциялар белгісімен байланысты тұрақтылар.

Мысалдар:

1) a=a+b a+=b;
2) a=a-b a-=b;

3) a=a*b a*=b;
4) a=a%b a%=b;
5) a=a>>b a>>=b;
6) a=a&b a&=b;
7) a=a|b a|=b;
8) a=a^b a^=b.

С++ тілінде арифметикалық және логикалық операциялармен қатар, қатынас және логикалық операциялары пайдаланады. Егер соңғы екі опреациялар нәтижесінде (қатынас және логикалық) жауабы- “жалған“, онда нәтиженің мәні нольге тең. Ал егер де жауабы “шын“ болса, онда жауабы нольден өзгеше болады. С++ тілінде логикалық берілгендер (bollean) жоқ.

қатынас операцияларына мысалдар:

a>b a>=b a<b
a<=b a==b a!=b

Логикалық операциялардың белгілері:

a&&b - ЖәНЕ операциясы;
a||b - НЕМЕСЕ операциясы;
!a - ЖОқ операциясы.

Бұл операциялардың ерешелігі, егер осы әрекеттерді орындамастан бұрын, нәтижесі белгілі болатын болса, онда логикалық өрнектердің есептелінуі тоқтатылады.

Берілгендерді енгізу және шығару

Көп жағдайда, берілгендерді өңдеу ғана керек емес, сонымен қатар оларды енгізу және шығару. С++ тілінде енгізу үшін біраз стандарттық функциялар бар: бірі берілгендерді клавиатурадан енгізеді, қалғандары берілгендерді файлдан алуға мүмкіндік береді. Олардың қатарына *scanf () және read ()* функцияларын жатқызуға болады. Берілгендерді шығаруды жүзеге асыратын негізгі функцияларға *printf(), puts() және putchar()* жатады.

Printf() функциясы

Printf() функциясының жалпы түрі:

printf() (форматтар қатары, объект, объект,..,объект);

Мысалы:

printf() (”қорытындысы %d\n”,y);

мұндағы %d - бүтін санды көрсету спецификасы; y - шығару объектісі. Шығару объектісі - айнымалы, тұрақты, өрнек немесе функция көрсеткіші.

Берілгендерді көрсетудің келесі алаңдардың спецификасын пайдалану мүмкін:

%d - бүтін сан;
%u - белгісіз бүтін сан;
%ld - ұзын бүтін сан;
%e - мантисса түріндегі нақты сан;
%p - көрсеткіштің мәні;
%c - С++мвол;
%s - С++мволдар қатары;
%x - оналтылық түрдегі бүтін сан;

Алаңның өлшемі үнсіздік бойынша немесе “ %“ белгісі және әріптің арасына санды көрсету арқылы.

“\ n“ қос С++мволдар басқару тізбегі деп аталады. Мұндай тізбектердің қызметі:

\n -келесі жолға көшу;
\t - келесі зонаға көшу;
\f - келесі бетке көшу;
\b -бір позицияға қайту;
\\ - “\“ белгісін шығару;

\xhhh- С++мволды шығару, бір, екі немесе үш оналтылық С++мволдармен көрсетілген ASCII-коды.

Puts() және putchar() функциялары

Puts() және putchar() функциялары, printf() функциясы С++яқты, информацияны экранға шығару үшін қолданылады. Puts функциясы экранға С++мволдық қатарды шығару үшін пайдаланады. С++мволдық жол, тұрақты түрінде, С++мволдарды тырнақшаға алу жолымен анықталынуы мүмкін. *Puts()* функциясы экран дисплейіне жолды шығарады, содан кейін экран алаңының жаңа жолына көшуін орындайды.

Мысалы:

```
#include <stdio.h>
main()
{ puts («Turbo C жүйесі»);}
```

Ескере кететін жай, «\ n» пайдаланбайды, өйткені жаңа жолға көшу автоматты орындалады.

Putchar() функциясы экранға жалғыз С++мволды, келесілерін жаңа жолға көшірмейтіндей шығарады.

Мысалы:

```
Putchar(symb);
```

Scanf() функциясы

Интерактивті енгізу үшін пайдаланады. Функцияның жазылу түрі:

scanf(форматтар қатары, адрес, адрес,..,адрес);

Форматтар қатарында, printf() функцияның форматтар қатарындағы пайдаланылатын спецификациясының көптеген түрі пайдаланылады, атап айтқанда, %d, %f, %c және %s - бүтін сандарды, нақты сандарды, сәйкес С++мволдар және жолдық С++мволдарды көрсететін алаңдарды С++паттайды.

Scanf() функциясы printf() функциясына қарағанда, шығару тізімінде берілгендерді емес, олардың адрестерін көрсетуін қажет етеді.

*Мысалы: (*a және b жадысында “ұяшығында“ екі бүтін санды енгіз*):*

```
scanf(«%d%d»,&a,&b);
```

«&» операция белгісі scanf() функциясына a және b жадысының ұяшықтар адресін беруін көрсетеді. Scanf() функциясын енгізу үшін пайдаланған кезде, кезекті С++мвол пробел болған кезде, жолды енгізу аяқталынады.

Мысалы (С++мволдық жолды енгіз):

```
#include<stdio.h>
main()
{ char im[12];
        printf(«Сіздің атыңыз:»);
        scanf(«%s»,im);
        printf(«Есім енгізілді:%s\n»,im);}
```

Gets(), getch() функциялары

Gets() функциясы пробелі бар, С++мволдық жолды енгізуге мүмкіндік береді.

Мысалы:

```
#include<stdio.h>
main()
{ char famim[35];
        printf(«Сіздің аты-жөніңіз:»);
        gets(famims);
        printf(«ФИО:%s\n»,famim);}
```

Gets() функциясын пайдаланған кезде, «Енгізу» пернесі басылғанға дейін енгізуді жалғастырамыз.

Getch() функциясы клавиатурадан жалғыз С++мволды, экранға бейнелеуінсіз енгізу үшін пайдаланады. Бұл функция параметр (аргумент) тапсырмасын қажет етпейді және char типті мәнді қабылдайды.

Мысалы (программа фрагменті):

```
char ch;
ch= getch();
```

Бір варианттың орындалуы

№1 тапсырма *a және b екі нақты сандар берілген. Олардың қосындысын, айырымын және көбейтіндісін алу керек.*

Шешімі:

```
#include<stdio.h>
main()
{       float a, b, c, d, e;
          printf(«Екі санды енгіз:»);
          scanf(«%e%e »,&a, &b);
          c=a+b;
          d=a-b;
          e=a*b;
          printf(«қосындысы c=%e\n Айырымы d=%e\n Көбейтіндісі e=%e\n »,c, d, e);
}
```

Нәтижесі:

```
Екі санды енгіз: 45 5
қосындысы c=50
Айырымы d=40
Көбейтіндісі e=225
```

№2 тапсырма. Кубтың қабырғасының ұзындығы берілген. Кубтың көлемін табу керек.

Шешімі:

```
 #include<stdio.h>
main()
{       int a, V;
        printf(«қабырғаның ұзындығын енгіз:»);
        scanf(«%d »,&a);
        V=a*a*a;
        printf(«Кубтың ауданы =%d\n»,V);

}
```

Нәтижесі:

```
қабырғаның ұзындығын енгіз: 3
Кубтың ауданы = 27
```

Тапсырма нұсқаулары

1.
2.
3.
4.
5.
6.
7.
8.
9.
10.
11.

12.
13.
14.
15.
16. f(x)=9,2 cosx-
17. f(x)= 12,4sin-8,3 cos1,2x
18. f(x)= sinxcosxsin(x+1,4)+5,14
19. f(x)= cosxsin(2x-1)+4,29
20. f(x)= 2sinxsin(2x-1,5)cos(2x+1,5)-6

Бақылау сұрақтары

1. Берілгендерді енгізу және шығару операторлары.
2. Логикалық және қатынас операциялар
3. Тұрақты, өрнек, меншіктеу туралы түсінік.
4. Инкремента және декремента туралы түсінік. Мысал келтіріңіз.
5. Математикалық функциялар.

№2, 3 ЗЕРТХАНАЛЫҚ ЖҰМЫС.

С++ тілінде шартты операторды қолданып программа құру.

Жұмыс мақсаты: Студенттерді практика жүзінде басқару конструкцияларын және басқа да операторларын пайдаланып, С++ тілінде программалар жазуға үйрету.

Студенттерге қойылатын талап: Кез келген студент берілген тапсырмаларға программалар жазып, ЭЕМ-де есептеп, нәтижесін алуы тиіс.

Жұмыстың орындалу тәртібі:

1. Берілген жұмыстың әдістемелік нұсқауымен танысу.
2. Берілген бақылау сұрақтарына жауап беру.
3. Жаттығуларды талдау.
4. Компьютерден нәтижені алу.

ӘДІСТЕМЕЛІК НҰСҚАУЛАР

Программа дегеніміз - есепті шешу алгоритмін программалау тілдерінің жабдықтарын пайдаланып С++паттау. Кез келген алгоритм тізбектелген, тармақталған және қайталау процестерінен тұрады. Бірінен соң бірі орындалатын амалдар тізбегі - тізбектелген болып табылады. Егер көптеген есептерді шешкенде қойылған шарттың орындалуы немесе орындалмауына байланысты шешу жолдарын таңдау- тармақталу, ал амалдар тізбегін қайталап орындайтын болсақ, кайталау процесін пайдаланамыз.

IF операторы

Бұл команданы (операторды) екі түрде пайдалану мүмкін:

а) if (өрнек)

1-ші оператор;

else

2-ші оператор;

б) if (өрнек)

1-ші оператор;

Егер өрнек (кейбір шарттар) нольге тең болмаса, онда 1-ші оператор орындалады. Оператор жай немесе құрама болуы мүмкін. құрама оператор - фигуралық жақшаның {} ішіне кіретін операторлар (командалар) жиынтығы. Жақшаның ішіндегі әрбір оператор нүктелі үтірмен аяқталады.

Мысалы:

```
#include<stdio.h>
main()
{       float a, b, res;
        printf(«Екі санды енгіз:»);
        scanf(«%f%f »,&a, &b);
        if (b= =0)
        printf(«a/b қатынасы анықталмаған\n »);
                else
        { res = a/b;
        printf («a/b тең%f\n », res);}}
```

SWITCH таңдау операторы

С++ тілінде көп жақты таңдау жасайтын орнатылған оператор бар. Оның негізгі формасы мына түрде болады:

```
switch (өрнек)
        { case constant1:
                операторлардың тізбегі
                break;
          case constant2:
                операторлардың тізбегі
                break;
                ...
          case constantN:
                операторлардың тізбегі
                break;
          default
                операторлардың тізбегі
}
```

Алдымен switch негізгі сөзінен кейін, жақшаның ішіндегі тұрған өрнектер есептелінеді. Содан кейін есептелінетін өрнектің мәніне сәйкес метка табылмағанша, белгілер тізімі (case constant және т.б.) қарастырылады. қос нүктеден кейінгі операторлардың тізбегі

жалғастырылып орындалады. Егер де өрнектің мәні switch операторының ешбір белгісіне сәйкес келмесе, онда default негізгі сөзінен кейінгі операторлар тізбегі орындалады.

```
#include<stdio.h>
main()
{       char ch;
        printf(«Қазақ алфавитінің бас әріптерін енгіз:»);
        ch = getchar ();
        if (ch>=’А’&&’ß’)
        switch (ch)
        {       case ‘А’:
                        printf(«Ахметов\n »);
                case ‘Б’:
                        printf(«Бокеев\n »);
                case ‘В’:
                        printf(«Валашов\n »);
                case ‘Г’:
                        printf(«Газизов\n »);
                default:
  printf («Дастанов, Зейінқұлов және басқалар\n »);
        }
   else printf («Қазақ алфавитінің бас әріптерін енгізуіңіз керек еді\n»);
Циклдің әртүрлі операторлары бар программалармен жұмыс орындау.        }
```

Бір варианттың орындалуы

№1 тапсырма. *a және b екі нақты сандары берілген. Max (a,b) алуымыз керек.*

Шешімі:

```
#include<stdio.h>
main()
{       float a, b, max;
        printf («Екі санды енгізіңіз:»);
        scanf («%e%e »,&a,&b);
        if (a>b) max =a; else max=b;
        printf(«Екі санның ең үлкені= %e\n»,max);
}
```

Нәтижесі:

```
Екі санды енгізіңіз: 45 89
Екі санның ең үлкені= 89
```

№2 тапсырма. *x, y және z нақты сандары берілген. max (x+y+z, xyz) есептеу керек.*

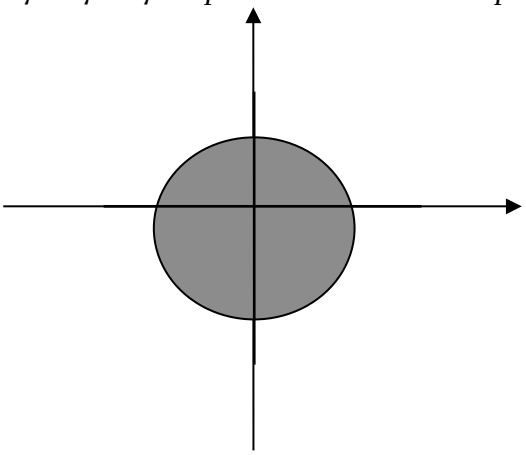
Шешімі:

```
#include<stdio.h>
main()
{       float x, y, z, a, b, max;
        printf («°ш санды енгізіңіз:»);
        scanf («%f%f%f »,&x,&y,&z);
        a=x+y+z; b=x*y*z;
        if (a>b) max =a; else max=b;
        printf(«Екі санның ең үлкені= %f\n»,max);
}
```

Нәтижесі:

```
Үш санды енгізіңіз: 3 4 5
Екі санның ең үлкені= 60
```

№3 тапсырма. *x және y екі нақты сандары берілген. X,Y координаттарымен берілген нүкте, жазықтықтың штрихталған бөлігіне кіретіндігін анықтау.*



Шешімі:

```
#include<stdio.h>
main()
{       float x, y;
        printf («Екі санды енгізіңіз:»);
        scanf («%e%e »,&a,&b);
        if (x*x+y*y<1) printf («Нүкте кесіндіге жатады»);
                        else printf («Нүкте кесіндіге жатпайды»);
}
```

Нәтижесі:

Екі санды енгізіңіз: 0.5 0.5
Нүкте кесіндіге жатады

Тапсырма нұсқаулары

1.
2.
3.
4.
5.
6.
7.
8.
9.
10.
11.
12.
13.
14.
15.

Бақылау сұрақтар

1. Тармақталған оператордың жазылу үлгісі.
2. Құрама шарт
3. Таңдау операторы. Switch операторы

№4 ЗЕРТХАНАЛЫҚ ЖҰМЫС.

С++ тілінде есептерді шығару үшін цикл операторын пайдалану.

Жұмыс мақсаты: Студенттерді практика жүзінде пайдаланушының қарапайым функцияларын құруды ұйымдастыру және басқа да операторларын пайдаланып, С++ ++тілінде программалар жазуға үйрету.

Студенттерге қойылатын талап: Кез келген студент берілген тапсырмаларға программалар жазып, ЭЕМ-де есептеп, нәтижесін алуы тиіс.

Жұмыстың орындалу тәртібі:

1. Берілген жұмыстың әдістемелік нұсқауымен танысу.
2. Берілген бақылау сұрақтарына жауап беру.
3. Жаттығуларды талдау.
4. Компьютерден нәтижені алу.

ӘДІСТЕМЕЛІК НҰСҚАУЛАР

Циклді ұйымдастыру операторлары. Қайталау есептеу процестері - while..., for... немесе do... while... операторлары бойынша орындалады.

while *(өрнек)*
оператор;

Біршама әмбебап оператор болып табылады. әзір өрнек нольге тең болмаса, оператордың (жай немесе құрама) әрекетін қайталап орындайды.

Мысалы 1:

```
#include<stdio.h>
main()
{       int dlina;
        dlina =0;
        puts («Жолды енгізіңіз, содан соң <Кқ>пернесін басыңыз»);
        while (getchar() !='\n')
                dlina ++
          printf(«\n Жолдың ұзындығы тең %d С++мволға \n»,dlina);
}
```

Мысалы 2:

```
#include<stdio.h>
main()
{       char *text;
        int ind;
        text = «СИ - кәсіби программистердің тілі. »);
        ind =1;
        while (ind <=5)
          { printf(«Жол %2d: %s»,ind, text);
            ind ++;}
}
```

СИ тілінде санауыш бойынша циклді ұйымдастыру үшін, арнайы оператор пайдаланады:

for (*1-өрнек; 2-өрнек; 3-өрнек;****)***
оператор;

мұндағы *“1-өрнек“* - цикл параметрінің бастапқы мәнін орнатады, *“2-өрнек“* - операцияның орындалуының шартының жалғасын анықтайды, *“3-өрнек“* - параметрдің модификациясының ережесін береді.

Ескерте кететін жай, әрбір өрнек үтір арқылы бөлінген, бірнешеуден құралуы мүмкін. Мұндай өрнектер топтық деп аталынады.

Мысалы:

```
#include<stdio.h>
main()
{       char *text;
        int vverh, vnis;
        text = «қадам »;
        ind =1;
        for (vverh =1, vnis=5; vverh<=5; vverh++, vnis --)
                printf(« %s: %2d\t %2d\n», text, vverh, vnis);
}
```

Циклді ***do*** *оператор* ***while*** *(өрнек);*

әзір өрнекте берілген шарт орындалғанша, “*операторомен*“ С++патталған әрекет қайталанады.

Мысалы:

```
#include<stdio.h>
#include<conio.h>
main()
{       float a, b, res;
        do {
                puts («\n Екі санды енгізіңіз:»);
        scanf («%f%f »,&a,&b);
        if (b= =0)
                puts («Бөлуге болмайды \n »);
        else
                {res = a/b;
                printf(«Нәтижесі: %f\n»,res);
};
        puts («q пернесін басу жұмысты аяқтайды»);}
while (getch() ! =’q’);}
```

while және do...while циклдерінің айырмашылығы, do...while циклін пайдаланған жағдайда, оның құрамына кіретін операторлар міндетті түрде бір рет орындалады.

Бір варианттың орындалуы

№1 тапсырма. *Қай сан цифрлар қатарында макС++мальды көп кездесетінін анықтау.*

Шешімі:

```
#include<stdio.h>
#include<conio.h>
char
input[50]=”9725222284490276284358712222222220687525914510289 6
main()
{       int i, l, sifra, simbol, kol[10], maximum;
        for (l=0; l<10; l++)
                kol[l]=0;
        for (l=0;(simbol=input[1]!=’\0’;l++)
                {switch(simbol)
                        {case’1’:kol[1]++;continue;
                        case’2’:kol[2]++;continue;
```

```
                    case’3’:kol[3]++;continue;
                    case’4’:kol[4]++;continue;
                    case’5’:kol[5]++;continue;
                    case’6’:kol[6]++;continue;
                    case’7’:kol[7]++;continue;
                    case’8’:kol[8]++;continue;
                    case’9’:kol[9]++;continue;
                    case’0’:kol[0]++;continue;
                    default:  continue;
                    }        }
maximum=kol[1];sifra=1;
for(l=2;l<10;l++)
        if (maximum<kol[1]) { maximum=kol[1];sifra=1;};
clrskr();
printf(“Көп қайталанатын сан %d-%d рет”,sifra, maximum);
if ((maximum==2)!!( maximum==3)‖ ( maximum==3)) printf(“a”);
printf(“|n”);     }
```

Нәтижесі:

Көп қайталанатын сан 2 - 18 рет

<u>Тапсырма нұсқаулары:</u>

n және x берілген кезде, №1 -ден №10 тапсырмаға дейін өрнекті есептеу:

1. 2.
3. 4.
5. 6.
7. 8.
9. 10.

x мәні берілгенде және ε ($\varepsilon=10^{-6}$) болғанда №11 –ші №20 тапсырмаға дейін қосындыны есептеу:

11. 12.
13. 14.
15. 16.
17. 18.
19. 20.

Бақылау сұрақтар

1.Циклді ұйымдастыру операторлары.
2. Цикл-Әзір, Цикл-Кейін, Цикл-Үшін операторлары
3. Олардың жазылу үлгісі. Мысал келтіріңіз.
4. Итерациялық цикл дегеніміз не?

№5-ЗЕРТХАНАЛЫҚ ЖҰМЫС.

Мәтіндік ақпаратпен және массивтермен жұмыс

<u>Жұмыс мақсаты:</u> Бір өлшемді массив түрінде берілген деректерді өңдеу және элементтерінің мәндерін белгілі бір заңдылықтарға байланысты алгоритмдерін түзіп программалауды үйрену.

<u>Студенттерге қойылатын талап:</u> Кез келген студент берілген тапсырмаларға программалар жазып, ЭЕМ-де есептеп, нәтижесін алуы тиіс.

Жұмыстың орындалу тәртібі:

1. С++ тілінің массивпен жұмыс істейтін операторларымен танысу
2. Берілген жұмысқа әдістемелік нұсқаумен танысу
3. Келтірілген мысалдарды талдау
4. Жеке тапсырмадағы есептерді шешудің алгоритмдері мен программасын түзу

5. Программаны ЭЕМ - де орындау
6. Бақылау сұрақтарына жауап беру

Қысқаша теориялық бөлім

ЭЕМ арқылы шешілетін көптеген есептер математикалық мағынасы бойынша біріктірілген немесе бір бірімен мағынасы бір деректердің жиынын құрайтын көлемді информацияны өңдеуден тұрады. Кеңістікте нүктенің орнын анықтайтын координаторлар, сызықты теңдеулер С++стемасын анықтайтын коэффиценттер матрицасы, функцияның кез келген нүктедегі мәндері, көпмүшеліктердің коэффиценттері осындай біртұтас деректер жиынын құрайды. Оларды бір өлшемді немесе екі өлшемді массив элементтері түрінде қарастыру есептеуді оңайлатады.

Жоғары математикада бір және екі өлшемді массивтерді **вектор** және **матрица** деп анықтайды. Сонымен **массив** дегеніміз - әр-қайсысына индекс деп аталатын бүтін сан сәйкес қойылатын бір тектес деректердің реттелген жиынтығы. Массив элементі индексті айнымалы болып табылады. С++ тілінде программалау негізінде пайдаланылатын айнымалылар қарапайым және индексті айнымалылар болып екіге бөлінеді. Массивтің әр бір элементіне оның индексі арқылы байланыс жасалынады. Мысалы a_2 бір өлшемді А массивінің екінші элементі, a_i - ші элементі, элементтерді өңдеу барысында оның индексі цикл параметрі қызметін атқарады.

ЭЕМ арқылы шешілетін көптеген есептер математикалық мағынасы бойынша біріктірілген немесе бір-бірімен мағынасы ортақ деректердің жиынын құрайтын көлемді информацияны өңдеуден тұрады.

Кеңістікте нүктенің орнын анықтайтын координаталар, сызықты теңдеулер С++стемасын анықтайтын коэффициенттер матрицасы, функцияның кез келген нүктедегі мәндер, көпмүшеліктердің коэффициенттері осындай біртұтас деректер жиынын құрайды. Оларды сызықты немесе тік таблицалар түрінде қарастыру есептеуді оңайлатады.

Сызықты таблицаны құрайтын мәндер қағазға жазылған кезде жолға немесе бағанаға орналастырылады. Таблицаның әрбір мәніне немесе элементіне оның нөмірі сәйкес болады және керісінше, реттік нөмері берілген кезде, таблицаның қай элементі жайлы әңгіме болып отырғаны бірден айқындалады. Мысалы, метеорологиялық станцияда температураның өзгерісі әр сағат сайын өлшенеді де тәулік бойғы өлшеу нәтижелері таблицаға жазылады

өлшеу уақыты,сағ	0	1	2	3	.... ..	22	23
температура	17	16	15,5	14	.... ..	18	17,5

Осы сызықтық таблицада 0 ден бастап 23 ке дейін нөмірленген 24 элемент бар. Мысалы таблицаның екінші элементінің мәні 17.

Жоғары математикада таблицалық шамаларды вектор және матрица деп анықтайды. Ал программалау курсында таблица түрінде берілген шамаларды **массив** деп атайды. Сонымен **массив** дегеніміз - әрқайсысына индекс деп аталатын бүтін сан сәйкес қойылған біртектес алғашқы деректердің реттелген жиынтығы. Массивтің аты,өлшемі болады. Массив элементіне индекс арқылы байланыс жасалынады, яғни массивтің әрбір элементі индекс бойынша анықталады.

Массив аты айнымалы шаманы анықтау ережесі бойынша анықталады, яғни, оның иденфикаторы болады. Мысалы, A,B,C т.с.с. **Массивпен жұмыс** дегеніміз оның элементтерін өңдеуден, оларға амал қолданудан тұрады.Қайсы элемент екенін көрсету үшін массивтің атына оның реттік нөмірін индекс етіп көрсетсе жеткілікті, сондықтан массив элементтері индексті айнымалылар арқылы белгіленеді.

Индекс ретінде тұрақтыларды, айнымалыларды сондай-ақ индексті өрнек деп аталатын арифметикалық өрнектерді (мысалы, a_{k+1},c_{i-2}) қолдануға болады. Бір ескере кететін жағдай, айнымалылар мен арифметикалық өрнектің құрамындағы айнымалылар алдын ала айқындалған болуы тиіс.

Индекс элементтің массивтегі орнын анықтайды, ал индекстердің саны массивтің өлшемін анықтайды: бір өлшемді, екі өлшемді т.б. Бір өлшемді массив-сызықты таблица. Оның элементі бір индекспен белгіленеді: a_0, a_2, a_i бір өлшемді А массивінің сәйкес 0-ші,2-ші,і-ші элементтері.

Сонымен массивтің нақты элементін көрсету үшін массив аты мен оның индексін көрсету керек.

Бір өлшемді массив мына түріндегі операторлармен С++патталады:

Элементтер типі массив аты [элементтер саны**];**

Мысалы:

```
#include<stdio.h>
#degine KO 3
main()
{  int mas[KO], i;
                    int mas[KO], i;
                    mas[0]=123;
                    mas[1]=456;
                    mas[2]=1990;
                    printf(«Айнымалылардың адрестері:»);
                    for (i=0;i<KO;i++)
                    printf(«\nАйнымалылардың мәні:»);
                    for (i=0;i<KO;i++)
                    printf(«%5d»,mas[i]);
                    printf(«\n»); }
```

Бір өлшемді массивті қолданып,қарапайым есептер шешу мысалдары.

1-мысал. Элементтері $к=x^2$ формуласы бойынша есептелетін 11 элементтен тұратын К массивін анықтау керек, мұндағы х=2; 2,1; 2,2;.....2,9; 3

Шешімі. Егер массив деген сөзді ескермейтін болсақ, бұл есеп өзімізге таныс h=0,1 адымымен өзгеретін [2; 3] аралығындағы Өрбір х үшін $K=X^2$ функциясының мәнін есептеп, таблица түрінде х,у -ті баспаға беру есебі. Қойылған есептің бар айырмашылығы - мұнда і есепшісі (счетчик) енгізілуі керек. Ол К-ны есептеудің әрбір санын есептеу, сонымен қатар, массивтің келесі элементінің нөмірін белгілеу қызметін атқарады.

Бұл алгоритм - қайталану құбылысты алгоритм.К массивінің элементтерін y_i-ді анықтау барысында цикл параметірінің қызметін Х жәй айнымалысы атқарады, себебі Х-тің алғашқы және соңғы мәні, өзгеру адымы белгілі. К бір өлшемді массиві анықталған соң баспаға беру үшін 0-ден 11-ге дейін өзгеретін і-ші айнымалысы (индекс) арқылы тағы да бір параметрлі циклдық құбылыс ұйымдастырылған.

Массивтің әрбір элементін есептеген соң дереу баспаға беріп, алгоритмде бір ғана циклдық құбылыспен шектелуге де болар еді. Бірақ, біздің мақсатымыз алдымен К массивін анықтау болғандықтан, оларды жауапқа беруді өз алдына жеке мәселе ретінде қарастырдық, себебі әрбір элемент мәні ЭЕМ жадында ұяшықтарда сақталып тұрады.

```
# include <stdio.h>
# include <conio.h>
# include <stdlib.h>
# include <math.h>
const int M=10;
void main ( )
{ float y[M],x; int i;
clrscr();
x=2;
i=0;
  printf ("x  y\n");
```

```
 do
 {
 y[i]=pow(x,2);
printf ("%2.3f%2.3f",x,y[i]);
printf ("\n");
x+=0.1;
i++;
}
while (x<=3);
}
```

2-мысал. A[1:13] бір өлшемді массивінің элементтерінің қосындысын есептеу керек

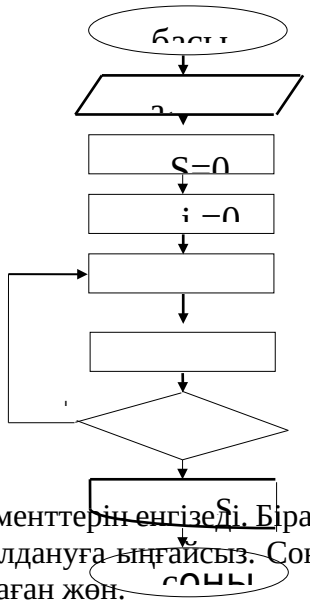
$$\sum_{i=1}^{13} ai$$

Шешімі. $S=a_1+a_2+a_3+.....+a_{13}=$,мұндағы і-қосылғыштың нөмірі, a_i-і-ші нөмірлі қосылғыш, қосындыны есептеу үшін қолданылатын реккурентті формуланы анықтау үшін алғашқы қосынды 0-ге тең деп аламыз, яғни $S_0=0$

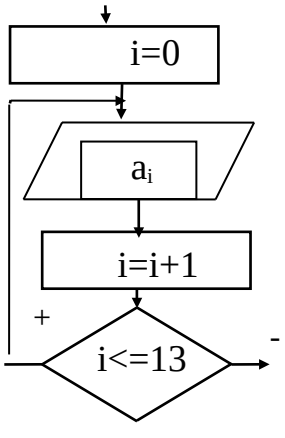
Сонда $S_1=S_0+a_1$, $S_2=S_1+a_2$, $S_3=S_2+a_3$, ... , $S_{13}=S_{12}+a_{13}$

Қосындыны есептеу алгоритмі-қайталану құбылысты алгоритм болады,цикл денесі қайталану орындалғанда қосылғыштың нөмірі 1-ге артып отырады, яғни $S_i=S_{i-1}+a_i$

Қорыта келсек, қосындыны есептеудің рекуррентті формуласын $S=S+a_i$ түрінде жазуға болады. Бұл формуланың мағынасы: алгоритмнің алдыңғы сатысында есептелген қосындының мәніне a_i -кезекті і-ші қосылғышты қосып, нәтижесін сол S айнымалысына меншіктеу.Қосындының алғашқы мәні S=0, себебі $S=S+a_i$ формуласы алғаш рет орындалғанда S мәні бірінші элементтің өзіне тең болуы керек. Бұл алгоритмде 0-ден 13-ке дейін өзгеретін і айнымалысы цикл параметрі қызметін атқарады. Қосынды нәтижесі цикл аяқталған соң баспаға беріледі. Келтірілген алгоритмнің схемасын сызып көрсетейік.



2-блок массив элементтерін енгізеді. Бірақ енгізудің бұл түрі енгізілетін деректердің саны көп болған жағдайда қолдануға ыңғайсыз. Сондықтан, массив элементтерін енгізуді циклдық құбылыс арқылы орындаған жөн.



Ескерту: А массивіндегі элементтердің саны 13 шартты түрде алынған. 13 емес, 113, 1000, т.с.с элементтен тұратын массив қарастыруға да болар еді. Қанша элемент болса да қарастырылған алгоритмнің құрылысында өзгеріс болмайды, элементтердің саны алгоритмнің ұзындығына әсер етеді.

C++ тіліндегі программасы:

```
# include <stdio.h>
# include <conio.h>
# include <stdlib.h>
# include <math.h>
void main ( )
{const int M=13;
int A[M]; int s,i;
clrscr();
randomize();
for(i=0;i<13;i++)
        {A[i]=random(100)-50;
        printf("%4d",A[i]);
        }
s=0;
for(i=0;i<13;i++)
s+=A[i];
printf("\n%d",s);
}
```

3-мысал. 10 элементтен тұратын бір өлшемді массив берілген. Элементтерінің көбейтіндісін есептеп, баспаға беру керек.

Шешімі. Көбейтіндіні есептеу рекуррентті формуласын қорытып шығару үшін, қосындыны есептеуден өзгешелігі, көбейтіндінің алғашқы мәні 1-ге тең деп есептелінеді,яғни $F_0=1$. Сонда ,

$F_1= F_0 * A_1$

$F_2= F_1 * A_2$

.....

$F_{10}= F_9 * A_{10}$

Олай болса рекуррентті формула $F=F * a_i$ түрінде болады, мұндағы айнымалы i=1-ден 10-ға дейін өзгереді, яғни i=1,10. Сондықтан і-цикл параметрі болады. Көбейтіндіні есептеудің параметрлі қайталану қубылысты алгоритмінің программасын келтірейік.

C++ тіліндегі программасы:

```
# include <stdio.h>
# include <conio.h>
# include <stdlib.h>
# include <math.h>
void main ( )
{const int M=10;
int A[M]; int p,i;
clrscr();
randomize();
```

```
for(i=0;i<10;i++)
{A[i]=random(100)-50;
printf("%4d",A[i]);
}
p=0;
for(i=0;i<13;i++)
p*=A[i];
printf("\n%d",p);
}
```

4-мысал. A[1:100] массивінің оң элементтерінің қосындысын есептеу керек.

Шешімі: Массивтің барлық элементтерін есептеу $S=S+a_i$ рекуррентті формуласын 0-ден 100-ге дейін өзгеретін і-дің мәндері үшін қайталап орындайтын циклдік құбылыс нәтижесінде атқарылатыны бізге мәлім, және алғашқы қосынды S=0.Енді тек оң мәнді элементтердің қосындысын есептеу керек болғандықтан, қосу формуласына дейін $a_i>0$ шартының орындалуын тексеру керек ($a_i>0$ шартының қойылу себебі барлық 100 элементті тексеру қажет). Егер элемент осы шартты қанағаттандырса, қосу формуласын орындау керек те, кері жағдайда келесі элементті тексеруге көшу керек. Бұл алгоритм күрделі қайталану құбылысты алгоритм, тармақталу қайталанады.

Ескерту: Егер қосындымен қатар массивтегі оң элементтердің санын анықтау қажет болса, к айнымалысы арқылы берілген оң элементтердің санын есептейтін есептегіш енгізуге болады. Оның алғашқы мәні 0-ге тең деп есептеледі де, әрбір оң элемент табылғанда мәні 1 ге артып отырады, яғни, $a_i>0$ болса к=k+1 болады.

C++ тіліндегі программасы:

```
# include <stdio.h>
# include <conio.h>
# include <stdlib.h>
# include <math.h>
void main ( )
{const int M=100;
int A[M]; int s,i;
clrscr();
randomize();
for(i=0;i<M;i++)
{A[i]=random(100)-50;
printf("%4d",A[i]);
}
s=0;
for(i=0;i<M;i++)
{if (A[i]>0)
s+=A[i];
}
printf("\n%d",s);
}
```

5-мысал $X(x_1,x_2,....x_{40})$ массивінің ең кіші элементін және оның реттік нөмірін анықтау керек.

Шешімі.

М арқылы массивтің ең кіші элементін белгілейік. Массивтің бірінші элементін ең кіші элемент ретінде алайық, яғни $M=x_1$. Онда ең кіші элементтің нөмірін сақтап отыратын К айнымалысына 1 меншіктеледі, яғни К=1. Алғашқы элементтің мәнін өзімен салыстырудың ешқандай мағынасы жоқ болғандықтан, М айнымалысының мәнін массивтің екінші элементінен бастап қалған барлық элементімен салыстырып шығу керек: егер М мәні

салыстырған элемент мәнінен үлкен болса М-нің мәнін сол элемент мәнімен ауыстырып (яғни салыстырылып тұрған элемент М-нен кіші), К айнымалысына оның индексін меншіктеу керек, кері жағдайда М, К шамаларын өзгеріссіз қалдыру керек. Бұл әрекет қайталау құбылысы арқылы іске асырылады. Алгоритм-күрделі: қайталану құрамына тармақталу құбылысы еніп тұр.

С++ тіліндегі программасы:

```
# include <stdio.h>
# include <conio.h>
# include <stdlib.h>
# include <math.h>
void main ( )
{const int M=40;
int A[M]; int min,k,i;
clrscr();
randomize();
for(i=0;i<M;i++)
{A[i]=random(100)-50;
printf("%4d",A[i]);
}
min =A[0];
k=1;
for(i=1;i<M;i++)
        {if (A[i]< min)
                { min =A[i];
                 k=i;
        }
                }

printf("\n%d  %d", min,k+1);
}
```

Бақылау сұрақтары

1. Массив ұғымын қалай түсінуге болады?
2. Массивті С++паттау, мүмкін болатын диапозондағы мәндер?
3. Массивті енгізу және шығару. Кездейсоқ сандар генераторын пайдаланып инициализациялау.
4. Бір өлшемді массивтерді өңдеу барысында цикл ұйымдастыру міндетті түрде қажет пе?

Тапсырмалар

1 вариант

1.Бір айда әр күнде жауған жауын мөлшері бір өлшемді массив арқылы берілген. Бір айда түскен жалпы жауын мөлшерін анықтаңыз.

2.Берілген Х(21) бір өлшемді массивінің алғашқы 10 элементін мәндерінің өсуі бойынша, соңғы 10-ын кемуі бойынша реттеп, Х массивін баспаға беріңіз.

2 вариант

1.Бір айда әр тәуліктегі орта температуралар бір өлшемді массив арқылы берілген. Қанша күнде температура айлық орта температурадан төмен болған?

2.Берілген А(15) массивін элементтерінің кемуі бойынша реттеп, ең үлкен және ең кіші элементтерінің коС++нустарының қосындысын табыңыз, түрленген А массивін баспаға беріңіз.

3 вариант

1.Бір айдағы әр тәуліктегі орта температура бір өлшемді массив түрінде берілген. Қанша күнде температура 0-ден төмен болған?

2.Берілген А(13) массивінің ең үлкен элементіне дейінгілерін өсуі бойынша, кейінгілерін мәндерінің кемуі бойынша реттеп баспаға беріңіз.

4 вариант

1.Кластағы оқушылардың бой ұзындықтары бір өлшемді массив түрінде берілген (қыздардың бой ұзындығы «+», ұлдардыкі «-» таңбаларымен). Ұлдардың орта бой ұзындығын табыңыз.
2.Берілген А(12) массивін мәндерінің өсуі бойынша реттеп, ең үлкен және ең кіші мәндерін, А реттелген массивін баспаға беріқіз.

5 вариант

1.Бір айда әр тәулікте түскен жауын мөлшері және ауаның температурасы бір өлшемді массивтері арқылы берілген. Жауынның қанша мөлшері қар түрінде, қаншасы жаңбыр түрінде түскенін анықтақыз. (Ауаның температурасы 0-ден жоғары болса, жаңбыр жауғаны)

2.Бір өлшемді А(11) массивін мәндерінің өсуі бойынша реттеп, ортаңғы бес элементтерінің қосындысын баспаға беріңіз.

6 вариант

1. 101 тобы студенттерінің физика пәнінен тоқсандық бағалары бір өлшемді массив түрінде берілген. Топтың физикадан орта үлгерімін анықтаңыз.
2. Берілген А(10) массивін мәндерінің өсуі бойынша реттеп, 5-ші, 9-шы және соңғы элементтерінің қосындысының коС++нусын баспаға беріңіз.

7 вариант

1.Тәуліктегі әр сағатта өлшенген температуралар белгілі ең жоғарғы температураны және уақытты(сағатты) анықтаңыз.
2.Берілген А(15) массивін ең кіші элементке дейінгілерін кемуі бойынша, кейінгілерін мәндерінің өсуі бойынша реттеп, баспаға беріңіз.

8 вариант

1.Бір тәулікте әр сағатта өлшенген ауа температуралары бір өлшемді массив арқылы берілген. Қай сағатта ең төмен температура болған және қанша градус.
2.Х(20) массиві берілген. Массив элементтерін мәндерінің өсуі бойынша реттеп, ең үлкен және ең кіші элементтерінің қосындысын табыңыз.

9 вариант

1.Класс оқушыларының математика пәнінен қорытынды бағалары бір өлшемді массив арқылы берілген. 5 алған оқушылардың санын анықтаңыз.
2.В(30) массивінің элементтерінің тангенстерінен тұратын Х(30) бір өлшемді массивін анықтап, соңғысын элементтерінің кемуі бойынша реттеп баспаға беріңіз.

10 вариант

1.Бір айда әр тәуліктегі орта температуралар белгілі. Қанша рет (күнде) температура айлық орта температурадан жоғары болады.
2.Берілген А(10) массивінің соңғы 7 элементін мәндерінік өсуі бойынша реттеп, А массивінің ең үлкен элементі қанша рет кездесетінін анықтаңыз, реттелген А массивін баспаға беріңіз.

11 вариант

1.101 топ студенттерінің тізімі және философия пәнінен қорытынды бағалары А$ және В бір өлшемді массивтер арқылы берілген. Бестік алған студенттердің тізімін баспаға беріңіз.
2.Берілген А(21) массивінің алғашқы 11 элементін өсуі бойынша, қалғандарын кемуі бойынша реттеп, шеткі мүшелерінің квадраттарының қосындысын есептеңіз.

12 вариант

1. "Жоғары математика" кафедрасының қызметкерлерінің тізімі және туған жылдары бір өлшемді A$ және B массивтері арқылы берілген. 1950 жылдан бұрын туылғандар тізімін баспаға беру керек.

2.Берілген A(21) массивінің алғашқы 7 элементін кемуі бойынша реттеп, A массивінің ең кіші элементі қанша рет кездесетінін анықтаңыз, A массивін баспаға беріңіз.

13 вариант

1.Сайыста түскен 10 спортсменнің жарыс қорытындылары бір өлшемді массив арқылы берілген. Жеңімпаздық нөмірін және оның көрсеткішін баспаға беріңіз.

2.Берілген A(15) массиві бойынша $в_i=\sin a_i$ формуласы бойынша B(15) массивін анықтап, баспаға беріңіз. (мұндағы $a_{i+1}>=a_i$ барлық i=1,15 үшін)

14 вариант

1.Абитуриенттердің тізімі және алғашқы емтиханнан бағалары бір өлшемді A$ және B массивтері арқылы берілген .Екінші емтиханға жіберілгендердің тізімін баспаға беру керек.

2.Берілген Z(19) массивін элементтерінің өсу ретімен реттеп, ең үлкен және ең кіші элементтері қанша рет кездесетінін анықтаңыз.

15 вариант

1.Кафедра қызметкерлерінің тізімі және туған жылдары A$ және B массивтері арқылы берілген. Қанша адам 1996 жылы 45-ке толатынын анықтаңыз.

2.Берілген B(15) массивінің ең үлкен элементіне дейінгілерін квадраттарымен ауыстырып,кейінгілерін мәндерінің кемуі бойынша реттеп, B массивін баспаға беріңіз.

16 вариант

1.Класс оқушыларының химия пәнінен тоқсандық бағалары массив арқылы берілген.Үлгермейтін оқушылардың санын баспаға беріңіз.

2.Берілген B(25) массивінің соңғы 13 элементін мәндерінің кемуі бойынша реттеп,барлық элементтерінің қосындысын және B(25) массивін баспаға беріңіз.

17 вариант

1.Тәулікте әр сағатта өлшенген ауа температурасы бір өлшемді массив арқылы берілген. Тәуліктегі орта температура қандай.

2.Берілген B(35) массивінің орташа 15 элементін мәндерінің өсуі бойынша реттеп, B массивінің арифметикалық ортасын табыңыз.

18 вариант

1.Студенттің бір пәннен 10 бағасы бір өлшемді массив арқылы берілген. Оның орта үлгерімі қандай.

2.Берілген A(30) массивінің алғашқы теріс элементке дейінгілерін(кем дегенде бір теріс элемент бар) өсуі бойынша реттеп, қалғандарын 2 еселеп, A массивін баспаға беріңіз.

19 вариант

1. 10 спортшының 100 м-ге жүгіруден көрсеткіштері бір өлшемді массив арқылы берілген. Қанша желаяқ жоғарғы көрсеткіш көрсеткен?

2.Берілген A(9) массивін мәндерінің өсуі бойынша реттеп. Ең үлкен элементін және реттелген A массивін баспаға беріңіз.

20 вариант

1. Тараз **қ**аласынан 12 қалаға дейінгі ара қашықтықтар бір өлшемді массив арқылы берілген. Таразға ең жақын қала қандай қашықтықта?

2.Берілген B(11) массивінің элементтерін өсуі бойынша реттеп, ең үлкен және ең кіші элементтерінің мәндерін ауыстырып баспаға беріңіз.

21 вариант

1.Абитуренттердің алғашқы емтиханнан алған бағалары бір өлшемді массив. Келесі емтиханға қанша адам өткенін анықтаңыз.
2.Берілген B(25) массивінің ең үлкен 5 элементінің қосындысын және көбейтіндісін баспаға беріңіз.

22 вариант

1.Бір тәулікте әр сағатта өлшенген температура белгілі. Қанша рет температура тәуліктік орта температурадан төмен болған?
2.Берілген A(15) массивінің ең кіші 3 элементінің арифметикалық ортасын табыңыз.

23 вариант

1.Класс оқушыларының туған жылдары бір өлшемді массив арқылы берілген. 1998 жылы 15-ке қанша бала толған?
2.Берілген X(20) массивін мәндерінің өсуі бойынша реттеп, алғашқы 10-шы және соңғы элементтерінің арифметикалық орташасын баспаға беріңіз

24 вариант

1.Бір айда әр тәуліктегі температуралар бір өлшемді массив арқылы берілген. Ең төменгі және ең жоғарғы температура қай күндері болған?
2. A(24) массиві берілген. Оның алғашқы теріс элементіне дейінгілерін (кем дегенде бір теріс элемент бар) өсу ретімен, қалғандарын кему ретімен реттеп, A массивін баспаға беріңіз.

25 вариант

1.Бір тәулікте 30 рет ауаның температурасы өлшенген, ол бір өлшемді массив түрінде берілген. Тәуліктегі орта температура қандай?
2.Берілген A(20) бір өлшемді массивінің ең үлкен 5 элементінің қосындысын және геометриялық ортасын табыңыз.

№6-ЗЕРТХАНАЛЫҚ ЖҰМЫС.

Пайдаланушының қарапайым функцияларын құру.

Жұмыс мақсаты: Жаңа типтерді өңдеу және оларды программаларда пайдаланудағы ерекшеліктер

Студенттерге қойылатын талап*:* Кез келген студент берілген тапсырмаларға программалар жазып, ЭЕМ-де есептеп, нәтижесін алуы тиіс.

Жұмыстың орындалу тәртібі:

1. С++ тілінің пайдаланушының қарапайым функцияларны құру функцияларымен танысу
2. Берілген жұмысқа әдістемелік нұсқаумен танысу
3. Келтірілген мысалдарды талдау
4. Жеке тапсырмадағы есептерді шешудің алгоритмдері мен программасын түзу
5. Программаны ЭЕМ - де орындау
6. Бақылау сұрақтарына жауап беру

ӘДІСТЕМЕЛІК НҰСҚАУ

Пайдаланушы анықтайтын берілгендер типі екі әдіспен пайдалануға беріледі. Бірінші тәсіл - жаңа берілгендер типін енгізу. Екінші тәсіл - әртүрлі модификаторлы типтерді (type modifiers) қолдану. Модификаторлы типтер негізгі базалық типтерге қолдануы мүмкін. Мұндай модификаторлар 4 типті:

- ену (доступ) модификаторы (const және voliatile)
- жадының модификаторлы класы (auto, static, extern және register).
- функцияның Borland C++ модификаторлы типі.
- жадының Borland C++ модификаторлы моделдері

Типтің модификаторын пайдалану келесі түрде орындалады:

модификатор_типі типтің_спецификаторы <айнымалылар _тізімі>

яғни, модификатор типтің атының алдында қойылады.

Const модификаторы C++ тілінде C++ ++ тілі шыққаннан соң шықты. Const модификаторымен хабарланған айнымалылар, программаның жұмыс істеу барысында өзгертілмейді. Const модификаторы көмегімен типтес, аттас тұрақтылар құралады.

Хабарлау мысалдары:

const float radius = 15.5;
const char yes = ‘y’;

const типінің ең қажетті колданылуы - функция аргументтерін өзгертулерден сақтайды. Егер функцияның аргументі болып, айнымалыға көрсеткіш табылатын болса, онда бұл айнымалының мәні функцияны шақыру кезінде өзгертілуі мүмкін. Егер де бұл айнымалыны const ретінде C++паттасақ, онда функция бұл айнымалының мәнін өзгерте алмайды.

Жолдың кодталу мысалы:

```
# include <stdoi.h>
void code (const char *str);
main (void)
{ code (“Это тест”);
return 0; }
void code (const char *str)
{ while (*str) printf (“%c”,(*str++)+1);}
```

Бұл айнымалыны өзгерткен кезде, мысалы, операциондық жүйенің жүйелік функциясын,voliatile модификаторы пайдаланамыз. Мысалы, компьютердің жүйелік уақыты бар глобальдық айнымалы.

Const және volatile модификаторлары бір уақытта пайдалану мүмкін:

Const volatile unsigned char*port=0x30;

Бұл хабарлама қажет болады, егер процессор портының 0x30 мәні тыс(извне) өзгертілген деп жорамалдасақ.

Borland C++ тән келесі спецификалық модификатордың түрі - функция типінің модификаторы. Бұл модификаторлар MS DOS операциондық жүйесінің программалау жүйесінде қажетті болады. Олар үшеу: pascal, cdecl және interrupt.

pascal, cdecl модификаторлары компиляторға, функция параметрлерін беру, C++ және Паскаль тілдеріне сәйкес ұйымдастырылады. Оған қажеттілік C++ тілінде жазылған программаларды, Паскаль тіліндегі программалармен бірге пайдалану кезінде туады.

```
Pascal factorial (register int n);
        { register int i=1;
                for (;n;n--) i*=n;
        return i;}
```

Егер де сіз, біреуінен басқа барлық функцияларды, Паскаль тіліндегідей параметрлерді беруді компиляциялау C++яқты, онда интегралдық ортада сәйкес опцияны орнатады. Функция үшін, параметрлерді C++ тіліндегідей беру үшін, осы функцияның типінің алдында cdecl модификаторын орнату керек.

interrupt модификаторы, компиляторға функция осы модификатормен үзілісті өңдеуді іске асыратындығын хабарлайды. Бұл модификатор резиденттік программаларды құруда пайдаланады.

Бізге танымал C++ тіліндегі берілгендер типінен басқа, 5 берілгендер типін құруға мүмкіндігі бар:

- құрылымдар (structure),
- біріктіру (union),
- саналатын тип (enumeration),
- өрістер биті (bit fields),
- 5-ші мүмкіндігі - typdef операторының көмегімен бар тип үшін жаңа ат құру.

Бір варианттың орындалуы

№1 тапсырма. Клавиатурадан енгізілетін, ASCII екілік кодын С++мволын экранға шығару.

Шешімі:

```
#include <stdio.h>
 #include <conio.h>
/*Биттер өрісін және біріктіруді пайдалану*/
struct byte
{       int b1:1;
        int b2:1;
        int b3:1;
        int b4:1;
        int b5:1;
        int b6:1;
        int b7:1;
        int b8:1;
};                      /* құрылым анықталды - биттік өріс*/
};
union bits
{       char ch;
        struct byte bit;
}u;             /*Біріктіру анықталды*/
void decode (union bits b);    /*функция прототипі*/
main (void)
{       do
        {       u.ch=getche();
                printf(“.”);
                decode (u);
        }       while (u.ch! =’q’);
        return 0;
        }
void decode (union bits b)
{       if (b.bit.b8) printf (“1”); else printf (“0”);
        if (b.bit.b7) printf (“1”); else printf (“0”);
        if (b.bit.b6) printf (“1”); else printf (“0”);
        if (b.bit.b5) printf (“1”); else printf (“0”);
        if (b.bit.b4) printf (“1”); else printf (“0”);
        if (b.bit.b3) printf (“1”); else printf (“0”);
        if (b.bit.b2) printf (“1”); else printf (“0”);
        if (b.bit.b1) printf (“1”); else printf (“0”);
        printf(“\n”);           }
```

Нәтижесі:
r: 01110010
a: 01100001
d: 01100100
i: 01101001
o: 01101111
q: 01110001

№7-ЗЕРТХАНАЛЫҚ ЖҰМЫС.

Массивтер және көрсеткіштер. Көпөлшемді массивтер

Жұмыс мақсаты: Студенттерді массивпен жұмыс істеудің әртүрлі алгоритмдерін пайдаланып, С++ тілінде программалар жазуға үйрету.

Студенттерге қойылатын талап: Кез келген студент берілген тапсырмаларға программалар жазып, ЭЕМ-де есептеп, нәтижесін алуы тиіс.

Жұмыстың орындалу тәртібі:

1. С++ тілінің массивпен жұмыс істейтін операторларымен танысу
2. Берілген жұмысқа әдістемелік нұсқаумен танысу
3. Келтірілген мысалдарды талдау
4. Жеке тапсырмадағы есептерді шешудің алгоритмдері мен программасын түзу
5. Программаны ЭЕМ - де орындау
6. Бақылау сұрақтарына жауап беру

ӘДІСТЕМЕЛІК НҰСҚАУ

Көп өлшемді массив келесі үлгі бойынша С++патталады:

массив атының типі [1-өлшем] [2-өлшем**]...**[N-өлшем**];**

Мысалы:

A(3,4) матрицасын B(4,2) матрицасына көбейту.

```
#include<stdio.h>
main()
{  int a[3][4]={{5,2,-4,13}, {4,9,0,6}, {7,-1,8,3}};
   int b[4][2]={{23,-6}, {92,-53},{-6,0}, {12,3}};
   int c[3][2] ,i,j,k;
         for (i=0;i<3;i++)
            {for (j=0;j<2;j++)
                  {c[i][j]=0;
                  for (k=0;k<4;k++)
                        c[i][j]=a[i][k]*b[k][j];}}
         for (i=0;i<3;i++)
         for (j=0;j<2;j++)
                  {printf(«c[%d][%d]=%d»,i,j,c[i][j]);
                        printf(«\n»); }}
```

Бір варианттың орындалуы

№1 тапсырма. *n ретті квадраттық матрицаны алу:*

1	*0*	*0*	*0*	*0*	*n*
0	*2*	*0*	*0*	*n-1*	*0*
0	*0*	....	....	*0*	*0*
....	....	....	....	....	....
0	*0*	....	....	*0*	*0*
0	*2*	*0*	*0*	*n-1*	*0*
1	*0*	*0*	*0*	*0*	*n*

Шешімі:

```
#include<stdio.h>
main()
{  const int n=9;
   int i,j,a[n][n];
         for (i=0;i<n;i++)
          for (j=0;j<n;j++)
             a[i][j]=0;
          for (i=0;i<n;i++)
             {a[i][j]=i+1;a[i][n-1-i]=n-;}
         for (i=0;i<n;i++)
           {for (j=0;j<n;j++)
                  printf(«%2d»,a[i][j]);
               printf(«\n»); }
         return 0;}
```

Нәтижесі:

1	*0*	*0*	*0*	*0*	*0*	*0*	*0*	*9*
0	*2*	*0*	*0*	*0*	*0*	*0*	*8*	*0*
0	*0*	*3*	*0*	*0*	*0*	*7*	*0*	*0*
0	*0*	*0*	*4*	*0*	*6*	*0*	*0*	*0*
0	*0*	*0*	*0*	*5*	*0*	*0*	*0*	*0*
0	*0*	*0*	*4*	*0*	*6*	*0*	*0*	*0*
0	*0*	*3*	*0*	*0*	*0*	*7*	*0*	*0*

0	*2*	*0*	*0*	*0*	*0*	*0*	*8*	*0*
1	*0*	*0*	*0*	*0*	*0*	*0*	*0*	*9*

№2 тапсырма. *A(9,9) массиві берілген. Ең үлкен элементтің табу керек.*

Шешімі:

```
#include <stdio.h>
main()
{  const int n=3;
  int i,j,a[n][n],max;
  printf (“Массивтің элементтерін енгіз”);
  for (i=0;i<n;i++)
    for (j=0;j<n;j++)
        {printf(«a[%d] [%d]=»,i,j);
         scanf(“%d”,&a[i][j];}
  max=a[1][1];
  for (i=0;i<n;i++)
    for (j=i;j<n;j++)
        if (a[i][j]>max) max= a[i][j];
  printf(«Ең үлкен элемент=%2d»,max);
  printf(«\n»);  return 0;}
```

Нәтижесі:

```
Массивтің элементтерін енгіз
a[0][0]=5
a[0][1]=4
a[0][2]=2
a[1][0]=3
a[1][1]=9
a[1][2]=2
a[2][0]=7
a[2][1]=6
a[2][2]=1
Ең үлкен элемент=9
```

Тапсырма варианттары:

1 вариант

1. A(K,L) массивінің жұп элементтерінің қосындысын табу керек.
2. D(5,5) массивінің диагональдан жоғары орналасқан элементтерінің арифметикалық ортасын табу керек.

2 вариант

1. B(4,5) массивінің берілген A және C сандарына тең элементтерін анықтау керек.
2. A(3,5) массивінің әр бағана элементтерінің қосындысы мен санын анықтау керек.

3 **вариант**

1.A(5,5) массивінің тақ элементтерінің көбейтіндісін табу керек.
2. B(4,4) массивінің диагональдан төмен орналасқан элементтерінің санын анықтау керек.

4 вариант

1. Z(5,6) массивінің 3-ке бөлінетін сандарының арифметикалық ортасын табу керек.
2. S(5,5) массивінің әр қатарының теріс элементтерінің қосындысын тауып баспаға беру керек.

5 вариант

1.A(6,4) массивінен берілген C санынан модулі бойынша артық болатын (($a_{i\ j}$(>C) B массивін анықтау керек.
2. P(3,5) массивінің әр қатарының оң элементтерінің санын анықтау керек.

6 вариант

1. A(5,6) массиві берілген. B<(a_{ij}(<C шартын қанағаттандыратын элементтердің қосындысын есептеу керек.
2. Z(5,5) массивінің диагональдан жоғары және нольдік элементтерінің көбейтіндісін есептеу керек.

7 вариант

1. B(L,N) массивінің 5-ке тең элементтерін санайтын алгоритм түзу керек.
2. C(4,4) массивінің әр бағанасының 1-ге тең элементтерінің геометриялық ортасын есептеу керек.

8 вариант

1. C(K,L) массивінің теріс және тақ элементтерінің арифметикалық ортасын табу керек.
2. A(6,4) массивінің әр қатарының оң және 5-ке тең элементтерінің санын анықтау керек.

9 вариант

1. A (10,5) массивінің (a_{ij}(>c шартын қанағаттандыратын элементтерінің арифметикалық ортасын анықтау керек.
2. C(5,5) массивінің диагональ бойында орналасқан элементтерінен B(5) массивін құру керек.

10 вариант

1.A(4,5) массивінің a_{ij}>10 шартын қанағаттандыратын элементтерін 100-бен алмастыру керек.
2. B(4,5) массивінің әр бағанасының жұп элементтерінің санын анықтау керек.

11 вариант

1. Z(4,5) массивінің теріс элементтерін 10 санымен алмастыру керек.
2.A(5,5) массивінің әр бағанасының үлкен элементтерін тауып, оларды диагональ элементтерімен алмастыру қажет.

12 вариант

1. A(4,6) массивінің тақ және оң элементтерінің қосындысын табу керек.
2. K(L,N) массивінің әр қатарының кіші элементтерінің геометриялық ортасын табу керек.

13 вариант

1. A(3,4) массивінің 5 пен 7 санының аралығында жатқан элементтерінің көбейтіндісін табу керек.
2. C(5,5) массивінің диагональ элементтерінің ең үлкені және бағаналардың кіші элементтерінің көбейтіндісін табу керек.

№8 ЗЕРТХАНАЛЫҚ ЖҰМЫС.

Функциялар және көрсеткіштер

Жұмыс мақсаты: Студенттер C++ тілінде рекурсивті функцияларды және параметрлі айнымалы санды функцияларды программаларда пайдаланып, олардың пайдаланудағы ерекшеліктерін оқып - үйрену

Студенттерге қойылатын талап: Кез келген студент берілген тапсырмаларға программалар жазып, ЭЕМ-де есептеп, нәтижесін алуы тиіс.

Жұмыстың орындалу тәртібі:

1. функцияны анықтауды және жариялау ерекшеліктерімен танысу
2. Берілген жұмысқа әдістемелік нұсқаумен танысу
3. Келтірілген мысалдарды талдау

4. Жеке тапсырмадағы есептерді шешудің алгоритмдері мен программасын түзу
5. Программаны ЭЕМ - де орындау
6. Бақылау сұрақтарына жауап беру

ӘДІСТЕМЕЛІК НҰСҚАУ

Егер кейбір операциялар жиынтығын бірнеше рет орындауға (бастапқы берілгендерді жиі жаңадан енгізу) тура келсе, онда ішкі программа деп аталатын программалық модуль қалыптастыруға болады. С++ тілінде ішкі программаның бір ғана түрі -функция -қолданылады.

Ішкі программа -функциямен жұмыс істеу нәтижесінде, оның атымен байланысты кейбір мәндер қалыптасады. Сондықтан ішкі программаны анықтаған кезде, оның атының алдыңда, көрсеткіштің типі қойылады. Кейде, функцияны пайдаланудың логикасы, функцияның атымен байланысқан мәнді қалыптасуын қажет етпейді. Сонда типтің көрсеткіші ретінде void қызметші сөзі пайдаланады.

С++ тілінде функцияның анықталуын (хабарлануын) және оның С++патталуын білуіміз керек. Егер функцияның бары туралы хабарланса, онда қалған функциялар (main деп аталатын бас функцияны қоса) функцияда хабарлануы мІмкін. Бірақ та функцияны хабарлау жеткіліксіз. Оның толық мәтінің С++паттауымыз керек.

Мысалы:

```
#include <stdio.h>
#include <conio.h>
/*Функцияның анықталуы */
void inab (float*u1, float*u2 );
float result (float chi, float zna);
void outres (float res);
const float INFINI=3/4e+38;
/* Бас модульдің мәтіні */
main()
{       float a,b,res;
        do { inab(&a, &b); /* a және b енгізу */
                res = result (a,b);
        outres (res);   /* res шығару*/
puts (“Жұмысты жалғастыру үшін Q пернесін басыңыз”);
        } while (getch () ! =’q’);
}
/* inab функциясын С++паттау
void inab (float *u1, float *u2)
{       puts (“Екі санда енгізіңіз:”);
        scanf (“%f%f”,u1,u2);
}
/* result функциясын С++паттау
float rsult (float chi, float zna)
{       if (zna= =0.0)
                return (INFINI);
        else return (chi / zna);}
/* outres функциясын С++паттау
void outres (float res)
{       if (res= =INFINI)
                printf(“Нәтиже анықталмаған\n”);
        else
                printf (“Нәтиже: %f\n”,res);}
/* Программаның тексті аяқталды */
```

Программаның басында пайдаланатын функциялардың анықталынуы берілген. Анықталуында параметрлердің типі және саны көрсетілген, бұл алдағы уақытта функцияны шақыру кезінде, әртүрлі қателерді (негізгі параметрлердің типімен санының сәйкес келмеуі) бақылауды қамтамасыз етеді. Содан кейін тұрақтылар С++патталады. Жалпы тұрақты функциядан тыс С++патталғандықтан, барлық функцияларға *“көрінің“* тұр. Келесіде, әрдайым main деп аталатын бас функцияның тексті (С++патталуы) келеді. Программада мұндай функция - жалғыз жӘне main функциясына назар аударып орындалады. Программаның текстінде main() кез келген жерінде орналасуы мүмкін. қарастырылған программада main() кейін үш функция С++патталған: берілгендерді енгізу, нәтижені есептеу және оны шығару.

Inab функциясының орындалуының нәтижесі, оның атымен байланысты, бұл нәтиже пайдаланбайды, void типінің берілгені ретінде жүреді. Функцияның параметрлері, берілгендерді енгізетін орындарының адрестері болып табылады. Сондықтан кіретін параметрлер, float типті берілгендер С++яқты анықталмайды. Ал, олар осы берілгендерге көрсеткіш түрінде анықталынады.

Result () функциясы бастапқы берілгендер ретінде сандық мәндерді және float типті нәтижені шығарады. Функцияның атымен байланысты нәтижені беру үшін, return операторы пайдаланады.

Outres () функциясы оның атымен байланысқан нәтижені шақыру “нүктесіне“ қайтармайды. Сондықтан бұл функция void типті болады.

Функцияның анықталуы

Программада пайдаланатын функция анықталуы тиіс; басқаша айтқанда, олар “хабарланған“ болуы керек. Функцияның анықталуының екі тәсілі бар: классикалық және қазіргі «современный». Классикалықтәсіл функцияның атын және нәтижені шақыру нүктесіне қайтаратын типті ғана хабарлайды. Жоғарыда қарастырылған программада:

```
void inab ();
float result ();
void outres ();
```

Мұнда параметрлердің саны, типтері туралы ешқандай информация жоқ.

“Қазіргі заманғы“ тәсіл прототип деп аталатын хабарлануды пайдаланады. “Прототип“ термині функцияны хабарлау кезінде, параметрлер туралы информацияны қосу үшін пайдаланады. қарастырылған программада прототиптер, әрбір функцияның бірінші қатарын дәлме-дәл қайталайды. Жалпы жағдайда прототип мына форматқа бағынады:

функция-аты типі (*инф-парам-1, ..., инф-парам-і,...****);***

мұндағы инф-парам-і- і-ші параметр туралы информация

Прототипті параметр туралы информация мына түрде көрсетіледі:

1. тип
2. параметр-атының типі

3. варианттың пайдалану параметрлері өзгеруі мүмкін екендігін білдіреді. Прототиптерді қолдану, параметрлердің тапсырмаларының корректілі қойылмағандығына, компиляторға бақылау жауапкершілігін жүктейді.

Функцияның С++патталуы

С++ тіліндегі программалар - функциялар жиынтығы. Функцияны С++паттаудың классикалық үлгісі келесі схемаға бағынады:

```
функция-атының типі (параметрлер тізімі)
        параметрлерді С++паттау;
{ локальдық С++паттау;
        ....
        команда;
        ....
        команда;        }
```

“Жаңаша“ үлгі параметрлерді С++паттауды жай жақшаның ішіне ауыстырды:

функция-атының типі (инф-парам-1,...,инф-парам-і,...)

{ локальдық С++паттау;

....

команда;

....

команда; }

Мұндағы инф-парам-і -параметр туралы информация (параметр типі және аты). Бұл прототипке ұқсас: әрбір параметр туралы информация бар. Прототиптен айырмашылығы тақырыптың жазылуы “;” бітпейді.

Тұрақтылардың, берілгендер типінің және функцияның ішіндегі айнымалылардың С++патталуы (main ішіндегі жазылғанды қоса) тек осы функцияның аралығында ғана орындалады.

С++ тілі бір функцияның екіншісіне енуін жібермейді.

Рекурсивтік функциялар

С++ тілінде функциялар өз-өздерін шақыра алады. Функция рекурС++вті деп аталады, егер функция денесіндегі операторда сол функцияның шақыруы бар болса. РекурС++втік функцияның классикалықмысалы ретінде,

N! =1*2*3*4*...N санының факториалын есептеу жатады.

```
factorial ()
{       int a;
        if (n ==1) return 1;
        a= factorial (n-1)*n;
        return a;
}
```

РекурС++втік функцияда функцияны шақыру функцияның жаңа көшірмесін құрмайды, ол жадыда жаңа локальдық айнымалылардың және параметрлердің көшірмесін құрайды. РекурС++втік функциядан шығуды қарастыру керек, онда жүйенің “тежеліп”(“зависание“) қалуы мүмкін.

Бір варианттың орындалуы

№1 тапсырма. a,b,c нақты сандары берілген.

$$f = \frac{\max(a, a+b) + \max(a, b+c)}{1 + \max(a+b*c, 1.15)}$$ алу керек.

Шешімі:

```
#include <stdio.h>
float max (float a, float b);
main()
{       float a,b, c,z;
        printf (“Енгіз a,b,c\n”);
        scanf (“%f%f%f”,&a,&b,&c);
        z=(max(a,a+b)+max(a,b+c))/(1+a+b*c,1.15));
        printf (“Нәтиже=%f”,z);
        printf (“\n”); }
float max (float a, float b)
{       float m;
        if (a<b) m=a;
        else m=b;
        return m;       }
```

Нәтиже:

Енгіз a,b,c

6 8 4

Нәтиже=0.666667

№2 тапсырма. s,t нақты сандары берілген. f(t,-2s,1.17)+f(2.2,t,s-t) алу керек, мұндағы

$$f(a,b,c) = \frac{2a - b - \sin c}{5 + |c|}$$

Шешімі:

```
#include <stdio.h>
#include <math.h>
#include <conio.h>
/* Функцияның анықталуы */
void inst (float *a, float *b);
float result (float a, float b, float c);
        /* Бас модульдің тексті  */
main()
{       float s,t,m,res;
                /* s және t енгіз */
        inst (&s, &t);
        res=result(t,-2*s,1.17)+result(2.2,t,s-t);
                /* res шығару */
        printf ("Нәтиже:%f\n",res);
        return 0;
}       /*  inst функцияның С++патталуы */
void inst (float *a, float *b)
{       puts ("Екі санды енгізіңіз:");
        scanf ("%f%f",a,b);
}       /* result функцияның С++патталуы */
float result(float a, float b, float c)
{       float j;
        j=(2*a-b-sin(c))/(5+abs(c));
        return j;
}
```

Нәтиже:

```
Екі санды енгізіңіз:
5  7
Нәтиже: 3.605013
```

Бақылау сұрақтары

1 Функцияны анықтау және жариялау

2. Рекурсивті функциялар қызметі. Мысал келтіріңіз

№9-ЗЕРТХАНАЛЫҚ ЖҰМЫС.

Құрылымдарды анықтау және қолдану.

Жұмыс мақсаты: Құрылымдар туралы теориялық мәліметтерді игеріп, программа құруға машықтану

Студенттерге қойылатын талап*:* Кез келген студент берілген тапсырмаларға программалар жазып, ЭЕМ-де есептеп, нәтижесін алуы тиіс.

Жұмыстың орындалу тәртібі:

1. С++ тілінің құрылыммен жұмыс істейтін операторларымен танысу
2. Берілген жұмысқа әдістемелік нұсқаумен танысу
3. Келтірілген мысалдарды талдау

4. Жеке тапсырмадағы есептерді шешудің алгоритмдері мен программасын түзу
5. Программаны ЭЕМ - де орындау
6. Бақылау сұрақтарына жауап беру

ӘДІСТЕМЕЛІК НҰСҚАУ

Құрылым әртүрлі типті, бірнеше айнымалыларды біріктіреді. құрылыммен біріктірілген айнымалылар мүшелер, элементтер немесе құрылымның өрісі деп аталады.

Мысалы:

```
struct student
{       char name [30];
        int kurs;
        char group[3];
        int stip;};
```

struct student типтің нақты айнымалыларын жариялау үшін, *struct student stud1, stud2* -деп жазсақ болады.

құрылымның нақты элементіне ену “нүкте”(dot) операциясының көмегімен іске асырылады.

Мысалы:

streру (stud1.name, “С++доров П.Р.”);

немесе

printf (“%s”,stud2.group);

құрылымдар, басқа типті айнымалылар тәрізді құрылымды массивке біріктірілуі мүмкін. құрылымды массивті жариялау үшін, алдымен құрылымның шаблонын беру керек (бізде student шаблоны бар), ал содан кейін массивті хабарлау орындалады:

struct student study [200];

Массивтің 25-ші элементіне kurs өрісіне ену үшін:

study [24].kurs

Әрбір жеке типтерге ену

С++ тілі байттағы немесе сөздегі бір немесе бірнеше биттерге енуді қамтамасыз етді. Олардың өзінің артықшылығы бар. Бір тәсілі- бұл биттер өрісі (bit fields). Биттер өрісі - құрылымның мүшелерінің арнайы
типтері. Мұнда әрбір элемент қанша биттен тұратындығын анықтайды. Осындай құрылымның негізгі түрі төменде көрсетілген:

```
struct құрылым_аты
  {типі аты1: бит_ұзындығы;
  типі аты2: бит_ұзындығы;
  ....
  типі атыN: бит_ұзындығы;};
```

құрылымда типтер :int, unsigned немесе signed болуы мүмкін.

құрылым ұзындығы әрдайым сегізге кратна. Егер, былай деп көрсетсек,

```
struct onebit
        { unsigned one_bit: 1;
        } obj;
```

онда obj айнымалысы үшін 8 бит бөлінеді, бірақ бірінші ғана пайдаланады.

Біріктіру

С++ тілінде әртүрлі типті бірнеше айнымалыларды жадыда орналастыру үшін тағы бір тип анықталды. Бұл біріктіру. Біріктіру, құрылым С++яқты хабарланады, мысалы:

```
union u
{       int i;
        char ch;
```

long int 1; };

Бұл біріктіру қандай да бір айнымалыны бермейді. Ол біріктіру шаблонын береді. Айнымалыны былай хабарлауға болады:

union u alpha, beta;

Жоғарыда көрсетілген мысалда, alpha айнымалыға жадыда 4 байт орын бөлінеді. Ал негізінен і элемент 2 байт, ch элемент - 1 байт және 1 элемент -4 байт қажет етеді. қалған айнымалылар жадының сол жерінде орналасады. Біріктіру элементтерін пайдалану С++нтакС++сі құрылымдағыдай:

u.ch='5';

Саналатын тип

Саналатын тип (enumeration) -бүтін тұрақтылардың аттарымен берілген жиын. Саналатын тип, осы типтің барлық айнымалыларының жіберілетін барлық мәндерін анықтайды. Типтің хабарлануы келесі түрде болады:

enum типтің_аты { аттар_тізімі }айнымалылар тізімі;

Айнымалылар тізімі бос болуы мүмкін.

Мысалы:

enum seasons {win, spr, sum, aut};
enum seasons s;

Бұл типтің негізгі түсініктемесінің кілті болып, win, spr, sum, aut әрбір аттары бүтін шаманы көрсетеді. Егер бұл шамалар басқаша анықталмаса, онда үнсіздік бойынша олар сәйкес нөл, бір, екі және үшке тең болады.

prinf ("%d%d",win, aut);

операторы экранға 0 және 3 сандарын береді. Типті хабарлау уақтысында бір немесе бірнеше С++мволдарға басқа мәндерді меншіктеуге болады. мысалы:

enum value {one=1, two, three, ten=10, thousand=1000, next};

Енді мәндерді баспаға шығарғанда,

printf ("%d%d%d%d%d\n",one,two,ten,thousand, next};

онда экранда 1 2 10 1000 1001 сандары экранда шығады, яғни әрбір келесі С++мвол алдыңғымен салыстырғанда,бірге өседі.

Саналатын типті айнымалылармен келесі операцияларды жүргізуге болады:

- enum типті айнымалыға, сол типті басқа айнымалыны меншіктеу;
- тең немесе теңсіздікті тексеру мақсатымен салыстыруды жүргізу;
- enum типті (i=win-aut) тұрақтылармен арифметикалық операциялар.

enum типті айнымалыларға арифметикалық операциялар және ++ операциясын және - - операциясын пайдалануға болмайды.

Typedef -типтердің атын өзгерту

С++ тілінде бар берілгендер типтеріне жаңа ат беруге мүмкіндігі бар. Ол үшін typedef қызметші сөзі қолданады. Бұл жағдайда жаңа берілгендер типі құрылмайды.

Мысалы:

typedef char SYMBOL;
typedef unsigned UNSIGN;
typedef float real;

құрылымды қолдану арқылы *typedef* операторын пайдалану жиі кездеседі:

```
typedef struct st_tag
{       char name[30];
        int kurs;
        char group [3];
        int stip;
} STUDENT;
```

Айнымалыны анықтау үшін, struct st_tag avar; пайдалануға болады және де STUDENT avar пайдалануға да болады.

Бір варианттың орындалуы

№1 тапсырма. Құрылымды - функция параметрі ретінде пайдалану.

Шешімі:

```
#include <stdio.h>
struct stru
        {int x;
        char e;
        };
void f(struct stru param); /*прототип*/
main (void)
{       struct stru arg;
        arg.x=1;
        arg.e='2';
        f(arg);
        return 0;       }
void f(struct stru param)
{   printf("Программа нәтижесі:%d%c\n",param.x,param.y);    }
```

Нәтижесі:

Программа нәтижесі: 1 2

№10-ЗЕРТХАНАЛЫҚ ЖҰМЫС.

Деректерді сұрыптау және іздеу.

Жұмыстың мақсаты: Тізбектілерді өңдеудің негізгі алгоритмдерін қарастырып, программа құруға үйрену

Жұмыстың орындалу тәртібі:

1. Берілген жұмысқа әдістемелік нұсқаумен танысу
2. Келтірілген мысалдарды талдау
3. Жеке тапсырмадағы есептерді шешудің алгоритмдері мен программасын түзу
4. Программаны ЭЕМ - де орындау
5. Бақылау сұрақтарына жауап беру

ӘДІСТЕМЕЛІК НҰСҚАУ

Алгоритмен жұмыс істегенде көбінесе элементтерді реттеу класы қолданады. Бұндай жұмыс реттеу класы деп аталады және есеп шығарған кезде кейбір әдістері қолданылады.

Қарапайым таңдау бойынша сұрыптау – бұл сұрыптаудың қарапайым алгоритмі. Сұрыпталатын массивте ең кіші элемент массивтің басында орналасады. Жұмыс жанында тізімдермен айқын ретте тізім элементтерінің ауыстырып қою қажеттілігі өте жиі көрінеді . Сондай мақсаттағы тізімдерді сұрыптау деп аталады және оның шешіміне арналған әртүрлі әдістерді қолданады.Соларды қарап шығайық.

Жай таңдау арқылы реттеу

Бұл – реттеудің ең қарапайым реттеу алгоритмі болып саналады.. Реттелетін массивтен ең кіші элемент ізделініп, оны массивтің бірінші элементімен алмастырамыз. Келесі кезекте қалған массив өздігінше массив болып қарастырылады. Өз кезегінде қысқартылған массивтен ең кіші элемент табылып, оны бірінші элементтің орнына қоямыз. Бұл әрекет массив бір элементке дейін қысқартылғанға дейін қайталанады.

Мысалы. 142, 23, 97, 19, 2, 4 массив элементтері берілген. Оларды реттейік.

```
#include <stdio.h>
#include <conio.h>
#include <stdlib.h>
void main()
{ const int N = 6;
        intmas[N];
        int i,j, i_min,j_min, min;
        randomize();
        clrscr();
        for (i = 0; i<N; i++)
        { mas[i]=random(100)-50;
        printf("%4d",mas[i]);}
        printf("\n");
         for (i=0; i<N; i++)
        { min = mas[i]; i_min = i;
  for (j = i+1; j<N; j++)             //қысқартылған массивте таңдау
        if(mas[j] < min) {min = mas[j]; j_min=j;}
        mas[j_min] = mas[i]; mas[i] = min; }
        for (i = 0; i<N; i++)
        printf("%4d",mas[i]);
        printf("\n");}
```

Көпіршікті сұрыптау

Көпіршікті сұрыптау өзінің қарапайым нұсқауында жай орындалады. Сондықтан оны элементтерді оптималдау үшін қолданады.Барлық көпіркшікті сұрыптаудың ерекшелігі-массив элементтерінің арасындағы алмасу.Массивтің екі көрші арасында орындалады.Көпіршікті сұрыптау келесіден тұрады:тұрақты, бүтін сандар берілген. (Қарапайым түрі)

В тобындағы элементтерді өз орнынп қойып шыққанда B=b тобындағы элементтерді шығарып алу үшін келесі формуланы қолданамыз кез-келген үшін.1<=i<=n элемент K^(1)=”k(i+1)”

Алгоритмді сұрыптау кезінде В тізімінен,В үздіксіз айырбастау кезінде жанында тұрған элемент ретке сәйкес. Сұрыптаудың үздіксіз қарапайымы көршілес тұрған элементтерді ауыстыру кезінде, қарағанда көпіршікті сұрыптауды анықтайды.МакС++малды элементтер тізімнің соңында тұрады.

Мысалы:

B=<20.-5.10.8.7>,

B1=<-5.10.8.7.20> 1 көрініс

B2=<-5.8.7.10.20> 2 көрініс
B3=<-5.7.8.10.20> 3 көрініс

Келесі мысалда бір өлшемді массивті реттеу класымен (n индексінен m индексіне дейін) өсу ретін байқаймыз.

Төмендегі Bubble функциясы массив көпіршікті сұрыптау әдісімен көрсетілген.

```
/*Көпіршікті сұрыптау әдісімен
Float bubble ( float a , int m , int n )
{
Char is =1;
Int i ;
Float c ;
While ( is )
{ is =0;
For ( i = m +1; i <= n ; i ++) if ( a [ i ]< a [ i -1]){ c =" a [ i ];" a [ i ]=" a [ i -1];" a [ i -1]=" c ;" is ="1;"}} return ( a );}
```

Көпіршікті сұрыптау көптеген жағдайда Q=(n-m)+(n-m) қажетті жадты қажет етпейді.

Мысалы. 142, 23, 97, 19, 2, 4 массив элементтерін көпіршікті әдіспен реттеу

```
#include <stdio.h>
#include <conio.h>
#include <stdlib.h>
void main()
{ const int N=6; int mas[N]; int i, j, wrk;
  randomize(); clrscr();
  for (i=0; i<N; i++)
          { mas[i]=random(100)-50;
            printf("%4d", mas[i]);}
            printf("\n");
            for(i=0; i<N-l; i++)
             for(j=0; j<N-l; j++)
               if(mas[j]< mas[j+1])
                              { wrk=mas[j];
                                mas[j]=mas[j+l];
                                mas[j+1]=wrk;}
          for (i=0; i<N; i++)
            printf("%4d", mas[i]);
           printf("\n");}
```

Тәжірибелік сабақ тапсырмалары:

Тапсырмалар

1 вариант

1.Берілген X(21) бір өлшемді массивінің алғашқы 10 элементін мәндерінің өсуі бойынша, соңғы 10-ын кемуі бойынша реттеп, X массивін баспаға беріңіз.

2 вариант

1.Берілген A(15) массивін элементтерінің кемуі бойынша реттеп, ең үлкен және ең кіші элементтерінің коС++нустарының қосындысын табыңыз, түрленген A массивін баспаға берікіз.

3 вариант

1. Берілген A(13) массивінің ең үлкен элементіне дейінгілерін өсуі бойынша, кейінгілерін мәндерінің кемуі бойынша реттеп баспаға беріңіз.

4 вариант

1. Берілген А(12) массивін мәндерінің өсуі бойынша реттеп, ең үлкен және ең кіші мәндерін, А реттелген массивін баспаға беріңіз.

5 вариант

1. Бір өлшемді А(11) массивін мәндерінің өсуі бойынша реттеп, ортаңғы бес элементтерінің қосындысын баспаға беріңіз.

6 вариант

1. Берілген А(10) массивін мәндерінің өсуі бойынша реттеп, 5-ші, 9-шы және соңғы элементтерінің қосындысының коС++нусын баспаға беріңіз.

7 вариант

1.Берілген А(15) массивін ең кіші элементке дейінгілерін кемуі бойынша, кейінгілерін мәндерінің өсуі бойынша реттеп, баспаға беріңіз.

8 вариант

1.Х(20) массиві берілген. Массив элементтерін мәндерінің өсуі бойынша реттеп, ең үлкен және ең кіші элементтерінің қосындысын табыңыз.

9 вариант

1. В(30) массивінің элементтерінің тангенстерінен тұратын Х(30) бір өлшемді массивін анықтап, соңғысын элементтерінің кемуі бойынша реттеп баспаға беріңіз.

10 вариант

1. Берілген А(10) массивінің соңғы 7 элементін мәндерінік өсуі бойынша реттеп, А массивінің ең үлкен элементі қанша рет кездесетінін анықтаңыз, реттелген А массивін баспаға беріңіз.

Бақылау сұрақтар:

1.Тізбектіліктерді өңдеудің негізгі алгоритмдері: көру, іздеу, сұрыптау ерекшеліктері.

2. Реттелген тізбектердің қосылуын атаңыз, мысал келтіріңіз

№11 -ЗЕРТХАНАЛЫҚ ЖҰМЫС.

Мәліметтердің типтік динамикалық құрылымын программалау

***Жұмыс мақсаты*:** Файлдан және клавиатурадан форматталған берілгендерді енгізу және нәтижені экранға және файлға шығаруды оқып- үйрену.

Жұмыстың орындалу тәртібі:

1. С++ тілінің файлдарымен жұмыс істейтін операторларымен танысу
2. Берілген жұмысқа әдістемелік нұсқаумен танысу
3. Келтірілген мысалдарды талдау
4. Жеке тапсырмадағы есептерді шешудің алгоритмдері мен программасын түзу
5. Программаны ЭЕМ - де орындау
6. Бақылау сұрақтарына жауап беру

ӘДІСТЕМЕЛІК НҰСҚАУ

С++ тіліндегі енгізу-шығару операциялар кітапханалық функциялар көмегімен ұйымдастырылады. С++-дің стандарттық кітапханасында енгізу/шығару функциялары файлдардан берілгендерді оқуды немесе оларды енгізу құрылғысынан алуды және файлдарға берілгендерді жазуды немесе оларды әртүрлі құрылғыларға енгізуге мүмкіндік береді.

Енгізу/шығару функциялары үш класқа бөлінеді:

1. Жоғарғы деңгейдегі енгізу/шығару (“ағын” түсінігін пайдалану арқылы).
2. Консольдық терминал үшін енгізу/шығару
3. Төменгі деңгейдегі енгзу/шығару(“дескриптор” түсінігін пайдалану арқылы).

Мысалы 1. А дискісіндегі phone.txt файлын досыңыздың фамилиясын, телефон номерімен толықтыруыңыз керек. Егер дискіде файл жоқ болса, онда программа оны құру керек. Файлда деректер элементы (аты, фамилиясы, телефоны) жеке қатарда орналасуы қажет.

```
// Фамилияны, атын, телефон номерін файлға толықтырады. #include "stdio.h"
#include "conio.h"
#define FNAME "a:phone.txt\0" // файл аты
void main()
{
char fname[20] = FNAME;
FILE *out; // сандар файлы
char fam[15]; // фамилиясы
char name[15]; // аты
char tel[9]; // телефон номері
puts("\n Телефон анықтамасын толықтыру");
// (t) мәтінін қосу режимінде (a) файл мәтінін ашу
// Егер бұл атпен файл жоқ болса, онда ол құрылады
if ((out = fopen(fname, "at")) == NULL)
{
printf("Толықтыру үшін файлды ашу қателігі");
getch();
return;
}
// пайдаланушыдан мәліметтерді аламыз
printf("Фамилиясы ->");
scanf("%s", &fam) ;
printf("Аты ->");
scanf("%s", &name);
printf("Телефоны ->");
scanf("%s", &tel);
//оларды файлға жазамыз
fprintf(out,"%s %s %s", fam, name, tel);
puts("Ақпарат қосылды");
fclose(out); // файлды жабу
printf("\n\n Аяқтау үшін  <Enter> - ді басыңыз\n");
getch();
}
```

Тапсырма варианттары:

1. Компоненттері нақты сандар болып табылатын f файл берілген. Табу керек: а) f файлының компоненттерінің қосындысын; б) файлдың соңғы компонентін.
2. Компоненттері нақты сандар болып табылатын f файл берілген. Табу керек: а) жұп нөмерлі компоненттердің ең кішісін анықтау; б) файл компонентінің бірінші және соңғы айырымын анықтау.

3. Компоненттері нақты сандар болып табылатын f_1, f_2, f_3 , f_4, f_5 файлдары берілген. Файлдардың арасында келесі түрде компоненттер алмасуын ұйымдастыру: f_1 файлының компоненты f_3 файлына жазылады, f_2 файлы - f_4-ке, f_3 - f_5, f_4 - f_1, f_5 - f_2. h көмекші файлын пайдалануға болады.

4. Компоненттері бүтін мәнді f файлы берілген. F файлының жұп сандарын g файлына, ал тақ сандарын h файлына жазу керек.

5. f C++мвольды файл берілген. G файлына f файлының мәндерін кері ретпен жазу керек.

6. Компоненттері нақты мәнді f файлы берілген. Табу керек: файлдың соңғы және бастапқы компонентінің осындысын.

7. Компоненттері бүтін мәнді f файлы берілген. Табу керек: компоненттердің ең үлкенін анықтау.

8. Компоненттері нақты мәнді x файлы берілген. Табу керек: компоненттердің көбейтіндісін.

9. Компоненттері нақты мәнді f файлы берілген. F файлдың тақ сандарын g файлына, ал жұп компоненттеріни h файлына.

10. Компоненттері оң таңбалы f файлы берілген. Табу керек: а) жұп номерлі компоненттердің геометриялық ортасын; б) бірінші және соңғы файл компонентінің қосындысын

11. Жақсы көретін 4 шумақ өлең жолдары бар текстік файл құру. Құрылған файлды қарап шығу. 2-ші куплетпен файлды толықтыру.

12. Текстік файл құру. Онда қатарлар санын есептеу.

13. **Фам.dat** файлында білімгерлер туралы мәлімет бар: фамилиясы, орта балл. Жаңа файл құру керек және онда тек **Фам.dat** файлындағы білімгерлердің фамилиясы қажет.

14**. PEOP.dat** файлында адамдар туралы мәлімет бар: фамилиясы, жасы, жынысы. Қыздарды және ұлдардың фамилиясы екі әр түрлі текстік файлда болу керек.

15. Текстік файл құру керек. Қатар бойынша оның номерін алдына қойып баспаға беру.

Бақылау сұрақтары

1. Файлдармен жұмыс істеуге арналған негізгі функциялар
2. файлды бағыттау функциясы
3. файлдарды байт бойынша өңдеу

№12-ЗЕРТХАНАЛЫҚ ЖҰМЫС.

Графикалық құрастыруларды программалау. Мәліметтер құрылымын өңдейтін есептерді программалау

***Жұмыс мақсаты*:** Объектілері бар үш программаны қарастыру, осы мысалдарды талқылай отыра, объектілермен жұмыс істеуді оқып -үйрену.

Жұмыстың орындалу тәртібі:

1. C++ ++ тілінің графикамен жұмыс істейтін функцияларымен танысу

2. Берілген жұмысқа әдістемелік нұсқаумен танысу
3. Келтірілген мысалдарды талдау
4. Жеке тапсырмадағы есептерді шешудің алгоритмдері мен программасын түзу
5. Программаны ЭЕМ - де орындау
6. Бақылау сұрақтарына жауап беру

ӘДІСТЕМЕЛІК НҰСҚАУ

Экранда сонымен қатар графикалық режимді орнатуға болады. Осы адаптерге сәйкес, сіздің компьютеріңізде графикалық адаптердің бірі және монитор орнатылған. Графикалық режимде экранды басқару функция жиындарымен орындалады, олардың прототиптері GRAPHICS.H бас файлында орналасқан. Ол жерде тұрақтылар және макростар жарияланған. GRAPHICS.H файлы С тілінің препроцессора #INCLUDE директивасының көмегімен қосылуы керек.

Графикалық режимнен шығып, текстік режимге көшу үшін, мына функцияны пайдалану керек:

void far closegraph (void);

және

void far restorecrtmode (void);

closegraph ( ) функциясы программа текстік режимде жұмыс істеуін жалғастыратын болса, пайдаланады. Бұл функция графикалық функциялар пайдаланатын жадыны босатады және initgraph () функциясын шақырғанға дейінгі, текстік режимді орнатады. Егер программа жұмысын бітірсе, онда restorecrtmode ( ) функциясын пайдалануға болады. Initgraph () функциясын алғашқы шақырғандағы, текстік режимде видеоадаптерді орнатады.

Сіздің компьютеріңізде орнатылған графикалық видеоадаптердің типі, графикалық режимде қанша түстер саны және қандай түстер пайдаланатындығын анықтайды. CGA және EGA адаптерлерінің арасында біршама айырмашылық бар. Алдыңғы таблицада түстер саны келтірілген.

Фонның түсін орнату мына функция бойынша:

void far setbkcolor (int color);

ал палитраны өзгерту мына функция бойынша:

void far setpalette (int index, int color);

бұл функция CGA видеоадаптерінде қолданылмайды.

Графикалық экранды пиксельдер массиві деп қарастырылады. әрбір пиксель экрандағы бір нүктеге сәйкес келеді және өзінің түсі болады. Экран нүктесінде (x,y) координаттарымен пиксельдің түсін орнату үшін, мына функцияның көмегімен орындалады:

void far putpixel (int x, int y, int color);

Негізгі line() және circle()«сурет салатын» функциялар болып табылады. Олардың прототиптері-

void far line (int x, int y, int x1, int y1);

void far circle (int x, int y, int radius);

line () функциясы экранда (x,y) координаталы нүктеден (x1,y1) координаталы нүктеге дейінгі, ағымды түспен түзу сызықты сызады.

Circle () функциясы экранда ортасы (x,y) координаталы нүктеде, радиусы radius (өлшем бірлігі - пиксель) бойынша және ағымды түспен дөңгелекті салады. °нсіздік бойынша ағымды түс -ақ болады. Ағымды түсті, яғни сызық салынатын түсті, setcolor () функциясын прототипі арқылы өзгертуге болады.

void far setcolor (int color);

Басқа да «сурет салатын» функцияларға мыналар жатады:

arc() дөңгелектің доғасын салады;

drawpoly көпбұрыштың контурын салады;

ellipse() эллипсті салады;

lineral ағымды нүктеден, салыстырмалы арақашықтықпен берілген нүктенің сызығын салады;

lineto() ағымды нүктеден, (x,y) координаталы нүктеге дейінгі сызықты салады;
mofeto() ағымды нүктені, (x,y) координаталы нүктеге орналастырады;
rectangle() төртбұрыш салады;
setspestratio() үнсіздік бойынша орнатылған сығылу коэффициентін өзгертеді;
setlinestyle() сызықтың стилін және енін орнатады.

Прототипті С++паттау мысалы ретінде, rectangle() функциясының прототипін келтіреміз:

void far rectangle(int left, int top, int right, int bottom);

Тұйық контурды бояу Ішін floodfill () функциясы қолданылады, берілген шаблон бойынша, беілген түспен облысты бояйды. Оның прототипі-

void far floodfill (int x, inty int bordecolor);

мұндағы x және y -контур ішіндегі нүктенің координаттары, bordecolor -контур түсі.

void far setfillstyle (int pattern, int color);

функциясы бойынша толтырудың түсі мен шаблонын орнатады.

Сонымен қатар, контурды сызатын және контурдың ішіндегі облысты бояйтын функциялар жиыны бар:

bar() толтырылған төртбұрыш,
bar3d() толтырылған бағана,
fillelipse() толтырылған эллипс,
pieslice() доғаның толтырылған секторы,
sector() эллипстік сектордың толтырылуы.

Borland C++ жүйесінің графикалық функциясының кейбір функциясын атап өтейік:

setfillpattern() пайдаланушы анықтайтын, шаблонды береді;
getfillpattern() толтырудың шаблонының типін қайтарады;
getfillsetting() шаблон және толтыру түсі туралы информацияны қайтарады;
getlinesetting() ағымды стиль, сызықтың түсі және ені туралы информацияны қайтарады;
getpixel() (x,y) нүктесінде пиксельдің түсі туралы хабарлайды.

Графикалық режимде текстi экранға шығару үшін, төмендегі функциялар пайдаланады.

Outtext() ағымды позициядан бастап, экранға жолды шығарады.
Outtextxy() берілген позициядан бастап, экранға жолды шығарады;
settextjustify() тексті түзетудің режимін орнатады;
setusercharsize() текстің үлкейтудің коэффи-циентін, шрифтін, стилін орнатады;
textheight() пиксель бойынша жолдың биіктігін қайтарады;
textwith() пиксель бойынша жолдың енін қайтарады;

Программаның жұмыс істеу уақтысында, графикалық операцияларды орындау барысында қателер пайда болуы мүмкін. Программалардың жұмыс істеуін қауіпсіздендіру үшін, қателерді өңдеуде, келесі функциялар пайдаланады:

graphresult() орындалудың қате кодын, соңғы графикалық операция үшін қайтарады;
grapherrormsg() берілген қате коды арқылы қате жолы туралы хабарламаны қайтарады.

Borland C++ графикалық кітапхананы пайдаланудың программасына мысал:

№1 Мысалы

```
# include<graphics.h>
/*Borland C++ графикалық функцияларды пайдалану*/
main (void)
    {int driver, mode, errorcode;
    register int i;
    driver= DETECT;
    initgraph (&driver, &mode, "");
    errorcode=gaphresult();
    if (errorcode != grOK)    /*егер қате болса */
    { printf ("қате: s\n",grapherrormsg (errorcode));
printf ("Тоқтату үшін кез келген пернені басыңыз\n");
```

```
        getch();
        exit(1);   /*программаның жұмысының аяқталуы */
        }
        rectangle (0,0,639,349);
        setcolor (RED);
        line (0,0,639,349);
        line (0,349,639,0);
        setfillstyle (5, GREEN);
        getch();
        clesrviewport();
        for (i=3;i<38;i+=3)
                circle (50,50,i);
        getch();
        restorecrtmode();
        return 0;}
```

Тағы бір мысал - палитрамен жұмыс істейтін функцияларды пайдалану.

№2. Мысалы

```
      # include<graphics.h>
      # include<conio.h>
      # include<stdio.h>
      # include<dos.h>
      void main (void)
        {int gd=DETECT,gm;
        initgraph (&gd,&gm, "F:\\BC31\\BGI");
        struct palettetype save , palette;
        getpalette (&save);
        getpalette (&palette);
        for (int i=o; i<10; i++)
                {setfillstyle (SOLOID_FILL, 6+i);
                setcolor (6+i);
fillellipse(getmaxx()/2,getmaxe()/2,(10-i)*10,(10-i)*10);
                }
        while (! Kbhit())
                {int temp=palette.colors[6];
                for (int i=7; i<16; i++)   //Shift palette
                        palette.colors[i-1]=palette.colors[i];
                palette.colors[15]=temp;
                setallpalette(&palette);
                delay(100);              }
        getch();
        closegraph();  }
```

Бір варианттың орындалуы

№1 тапсырма. Бұл программа кеме суретін салып оны қозғалысқа түсіреді.

```
          #  include  <graphics.h>
          #  include  <stdio.h>
          #  include  <conio.h>
          #  include  <math.h>
          #  include  <dos.h>
          #  define  dx  5
          #  define  dy  5
```

```
void  ship (int  x, int  y, int  color);
{
        setcolor (color);
        moveto (x, y);
        lineto (x, y-2*dy);
        lineto (x+10*dx, y-2*dy);
        lineto (x+17*dx, y-3*dy);
        lineto (x+14*dx, y);
        lineto (x, y);
        moveto (x+3*dx, y-2*dy);
        lineto (x+4*dx, y-3*dy);
        lineto (x+4*dx, y-4*dy);
        lineto (x+13*dx, y-4*dy);
        lineto (x+13, y-3*dy);
        line (x+5*dx, y-3*dy, x+9*dx, y-3*dy);
        rectangle (x+8*dx, y-4*dy, x+11*dx, y-5*dy);
        rectangle (x+7*dx, y-4*dy, x+8*dy, y-7*dy);
        circle (x+12*dx, y-2*dy, dx/2);
        circle (x+14*dx, y-2*dy, dx/2);

        line (x+10*dx, y-5*dy, x+10*dx, y-10*dy);
        moveto (x+17*dx, y-3*dy);
        lineto (x+10*dx, y-10*dy);
        lineto (x, y-2*dy);
}
#  define PATHTODRIVER " \\c:\\borlandc\\bgi\\"
void main (void)
{
        int   x, y;
        int   maxx;
        int   gdriver = DETECT;
        int   gmode;
        int   errorcode;
        initgraph (&gdriver, gmode, PATHTODRIVER);
        errorcode = graphresult ( );
        if (errorcode == grOk)
                {
                clock ( );
                closegraph ( );
                }
        else
                {
                printf ("Ошибка: %d \ n", errorcode);
                puts ("Для завершения программы нажмите <Enter>");
                getch ( );
                return;
                }
        maxx = getmaxx ( );
        x = -10;
        y =100;
        while ( x < maxx )
        {
```

```
            ship (x, y, GREEN);
            delay (20);
            ship (x, y, BLACK);
            x += 5;
        }
        setcolor (GREEN);
        outtextxy (10, 10, “Рейс завершен!”);
        outtextxy (10, 24, “Нажмите  <Enter>”);
        getch ( );
        closegraph ( );
}
```

№2 тапсырма. Минуттық және сағаттық стерлкалары бар сағатты құру программасы төмендегідей.

```
#  include  <graphics.h>
#  include  <stdio.h>
#  include  <conio.h>
#  include  <math.h>
#  include  <dos.h>
#  define PATHTODRIVER “ \\c:\\borlandc\\bgi\\”
void  vector (int a,   int l)
{
        #  define  G  0.0174532
        int   x0, y0;
        int   x1, y1;

        x0 = getx ( );
        y0 = getx ( );
        x1 = x0 + 1 * cos (a * G);
        y1 = x1 – 1 *  sin (a * G);
        lineto (x1, y1);
}
void cloc ( )
{
        int   x0 = 80,
            y0 = 80;
        int   d = 50;
        int   s = 0;
        int   m = 0;
        int   as = 90;
        int   am =90;
        circle (x0, y0, d+5);
        setfillstyle (SOLID_FIL, 0);
        do     {
                moveto (x0, y0);
                setcolor (YELLOW);
                vector (as, d);
                moveto (x0, y0);
                setcolor (GREEN);
                vector (am, d – 10);
                delay (1000);
                setcolor (GREEN);
```

```
                moveto (x0, y0);
                vector (as, d);
                moveto (x0, y0);
                vector (am, d – 10);
                s++;
        if  (s > 60)   {    m++;
                        s = 0;
        am -=6;
        if  (am < 0)        am = 354;
                          }
        as -=6;
        if  (as < 0)         as = 354;
                }
                while ( !kbhit ( ) );
}
void main (void)
{
        int   gdriver = DETECT;
        int   gmode;
        int   errorcode;
        initgraph (&gdriver, gmode, PATHTODRIVER);
        errorcode = graphresult ( );
        if (errorcode == grOk)
                {
                clock ( );
                closegraph ( );          }
        else                {
                printf (“Ошибка: %d \ n”, errorcode);
                puts (“Для завершения программы нажмите <Enter>”);
                getch ( );
                }
}
```

Жеке тапсырманың варианттары:

1 вариант

1. Шеңбер және оның диаметрін сызыңыз
2. Берілген функцияның графигін түзіңіз: $y = 40(\cos^2 x + \sin x) + 50 \qquad x \in [50, 450], h = 2$
3. Снаряд қозғалысының программасын түзіңіз.

2 вариант

1. Үйдің келбетін сызыңдар.
2. Берілген функцияның графигін түзіңіз: $y = 40(\cos x + \sin x) + 100 \qquad x \in [50, 300], h = 4$
3. Боялған үшбұрыштың диоганаль бойынша төменнен жоғары қарай қозғалысын аықтайтын программаны құрыңыз.

3 вариант

1. Сектор сызыңыз.
2. Берілген функцияның графигін түзіңіз: $y = 20(\cos 3x + \sin 3x) + 100 \qquad x \in [50, 350], h = 5$
3. Боялған квадраттың диоганаль бойынша төменнен жоғары қарай қозғалысын аықтайтын программаны құрыңыз.

4 вариант

1. Шеңбер және оның хордасын сызыңыз.

2. Берілген функцияның графигін түзіңіз: $y = 30\cos(x + \frac{\pi}{3}) + 50 \qquad x \in [100, 500], h = 5$
3. Төмен түсіп келе жатқан парашюттің қозғалысын анықтайтын программаны құрыңыз.

5 вариант

1. Жарты дөңгелек сызыңыз.
2. Берілген функцияның графигін түзіңіз: $y = 50 + 30\sin^2(x - \frac{\pi}{3}) \qquad x \in [100, 400], h = 3$
3. Тік бұрышты трапецияның төменнен жоғары қарай қозғалысының программасын құрыңыз.

6 вариант

1. Төртбұрышты қиық пирамида сызыңыз.
2. Берілген функцияның графигін түзіңіз: $y = 50 + 30\sin^2(x - \frac{\pi}{3}) \qquad x \in [100, 400], h = 3$
3. Төртбұрышты қиық пирамиданың жоғарыдан төмен қарай қозғалысының программасын құрыңыз.

7 вариант

1. Дұрыс төртбұрышты қиық пирамида салыңыз.
2. Берілген функцияның графигін түзіңіз: $y = 40 + 30\left|\cos x - e^{-\frac{x}{50}}\right| \qquad x \in [50, 350], h = 2$
3. Ұшып бара жатқан ракетаның қозғалысының программасын жазыңыз

8 вариант

1.Қиық конус сызыңыз

2. Берілген функцияның графигін түзіңіз: $y = 20\cos(x + 10) + 100 \qquad x \in [100, 500], h = 5$
3.Ұша бастаған ракетаның қозғалысының программасын жазыңыз.

9 вариант

1 Қиық пирамида сызыңыз.

2 Берілген функцияның графигін түзіңіз: $y = 30(\sin 2x + \cos 2x) \qquad x \in [30, 350], h = 0{,}4$
3 Ұшып бара жатқан самолет қозғалысының программасын жазыңыз

10 вариант

1 Жарты дөңгелек сызыңыз.

2 Берілген функцияның графигін түзіңіз: $y = 50(\sin x + \cos x) \qquad x \in [100, 400], h = 2$
3 Ұшатын тарелканың программасын жазыңыз

Бақылау сұрақтары:

1. Графикалық жүйені инициалдау қалай орындалады?
2. Графикалық функциялар
3. Түстерді, шрифтерді, сызықтық стильдерін және бояу стильдерін қондыру функциялары

№13 ЗЕРТХАНАЛЫҚ ЖҰМЫС

Ерекше жағдайларды өңдеу

Жұмыстың мақсаты: C++ тіліндегі try, throw, catch механизмдерін меңгеру және қате жағдайларды өңдейтін программа құру.

Студенттерге қойылатын талап: берілген мысалды орындау, жеке тапсырма бойынша программа құру және қалыпты әрі қате деректермен алынған нәтижелерді түсіндіру.

Әдістемелік нұсқаулар

Ерекше жағдай — программаның қалыпты орындалуын бұзатын жағдай туралы хабарлау механизмі. Мысалы, файлдың ашылмауы, рұқсат етілмейтін аргумент немесе контейнер шегінен шығу.

Ерекше жағдайларды өңдеуде:

- try — ерекше жағдай туындауы мүмкін кодты қамтиды;
- throw — ерекше жағдайды тудырады;
- catch — сәйкес ерекше жағдайды ұстап, өңдейді.

Нөлге бөлуді ерекше жағдай арқылы өңдеу үшін бөлгішті алдын ала тексеріп, throw қолдану қажет. Арифметикалық нөлге бөлу өздігінен стандартты C++ ерекше жағдайын тудырмайды. Стандартты ерекше жағдайларға std::invalid_argument, std::out_of_range, std::runtime_error жатады. Ерекше жағдайларды тұрақты сілтеме арқылы ұстау ұсынылады:

```
catch (const std::exception& e) {
   std::cout << e.what();
}
```

Жұмыстың орындалу тәртібі

1. Ерекше жағдайларды өңдеу механизмін қайталау.
2. Төмендегі программаны теріп, іске қосу.
3. Оны қалыпты деректермен тексеру.
4. Нөлдік бөлгіш, қате индекс және санның орнына мәтін енгізу жағдайларын тексеру.
5. Жеке тапсырманы орындау.
6. Нәтижелерді есепке енгізіп, бақылау сұрақтарына жауап беру.

Программалық мысал

Есеп: екі санның бөліндісін есептеу және вектор элементін индексі бойынша шығару. Қате жағдайларды өңдеу.

```
#include <iostream>
#include <vector>
#include <stdexcept>

double divide(double a, double b) {
   if (b == 0.0) {
      throw std::invalid_argument("Bolgis nol bolmauy kerek.");
   }
   return a / b;
}

int main() {
   // Бірінші әрекет: бөлу
   try {
      double a, b;
      std::cout << "Eki san engiziniz: ";

      if (!(std::cin >> a >> b)) {
         throw std::runtime_error("Sandardy engizu qatesi.");
      }

      double result = divide(a, b);
      std::cout << "Bolindi: " << result << '\n';
   }
   catch (const std::invalid_argument& e) {
      std::cout << "Argument qatesi: " << e.what() << '\n';
   }
```

```
    catch (const std::runtime_error& e) {
        std::cout << "Engizu qatesi: " << e.what() << '\n';
        return 1;
    }

    // Екінші әрекет: элементке индекс бойынша қатынау
    try {
        std::vector<int> values{10, 20, 30, 40, 50};

        int index;
        std::cout << "Indeks engiziniz (0-4): ";

        if (!(std::cin >> index)) {
            throw std::runtime_error("Indeks engizu qatesi.");
        }

        if (index < 0) {
            throw std::out_of_range("Indeks teris bolmauy kerek.");
        }

        int value = values.at(static_cast<std::size_t>(index));
        std::cout << "Element: " << value << '\n';
    }
    catch (const std::out_of_range& e) {
        std::cout << "Indeks qatesi: " << e.what() << '\n';
    }
    catch (const std::exception& e) {
        std::cout << "Qate: " << e.what() << '\n';
    }

    return 0;
}
```

Түсіндірме: vector::at() индекс шектен шыққанда std::out_of_range ерекше жағдайын тудырады. operator[] мұндай тексеруді қамтамасыз етпейді.

Тексеру деректері

№	Енгізілетін деректер	Күтілетін нәтиже
1	a = 20, b = 4, индекс 2	Бөлінді 5, элемент 30
2	a = 10, b = 0, индекс 0	Бөлу қатесі өңделеді, элемент 10
3	a = 12, b = 3, индекс 8	Бөлінді 4, индекс қатесі өңделеді
4	a = 12, b = 3, индекс -1	Теріс индекс туралы хабарлама
5	Санның орнына abc енгізу	Енгізу қатесі туралы хабарлама

Жеке тапсырманың нұсқалары

Әр нұсқада қалыпты жағдайды және кемінде екі қате жағдайды тексеріңіз.

Нұсқа	Тапсырма
1	BankAccount класын құрыңыз. Теріс салым мен қалдықтан артық ақша алуды ерекше жағдай арқылы өңдеңіз.
2	Квадрат түбір есептейтін функция құрыңыз. Теріс аргумент пен қате енгізуді өңдеңіз.
3	Студент бағасын қабылдайтын программа құрыңыз. 0–100 аралығынан тыс мәндер мен қате енгізуді өңдеңіз.

Нұсқа Тапсырма

4	Вектор элементін өзгертетін функция құрыңыз. Теріс және шектен тыс индекстерді өңдеңіз.
5	Тауар сатып алу программасын құрыңыз. Теріс сұраныс санын және қоймадағы қордан артық сұранысты өңдеңіз.
6	Файлдан сандарды оқыңыз. Файлдың ашылмауын және сан емес деректерді өңдеңіз.
7	Тіктөртбұрыш ауданын есептеңіз. Нөлдік және теріс өлшемдерді өңдеңіз.
8	Жасты енгізетін программа құрыңыз. 0–150 аралығынан тыс мәндер мен қате енгізуді өңдеңіз.
9	Бос вектордан орташа мән есептеуге әрекет жасағанда ерекше жағдай тудыратын функция құрыңыз.
10	a, b және арифметикалық амал таңбасын қабылдайтын калькулятор құрыңыз. Нөлге бөлу мен белгісіз амалды өңдеңіз.

Бақылау сұрақтары

1. Ерекше жағдайларды өңдеу не үшін қажет?
2. try, throw, catch қандай қызмет атқарады?
3. Бір try блогына бірнеше catch блогын жазуға бола ма?
4. what() әдісі қандай ақпарат береді?
5. Ерекше жағдайды тұрақты сілтеме арқылы ұстаудың артықшылығы қандай?
6. vector::at() пен vector::operator[] айырмашылығы неде?
7. Өңдегіштерді нақты типтен жалпы типке қарай орналастыру не үшін қажет?
8. Ерекше жағдай кезінде стек тарқатылуы дегеніміз не?

№14 ЗЕРТХАНАЛЫҚ ЖҰМЫС

Динамикалық типті анықтау және типтерді түрлендіру

Жұмыстың мақсаты: typeid, dynamic_cast, static_cast, const_cast, reinterpret_cast құралдарының қызметін түсіну және оларды орынды қолдану.

Студенттерге қойылатын талап: полиморфты кластар иерархиясын құру, объектілердің нақты типтерін анықтау және сәтті әрі сәтсіз түрлендірулерді тексеру.

Әдістемелік нұсқаулар

RTTI — программа орындалу кезіндегі тип туралы ақпарат механизмі. Полиморфты класта кемінде бір виртуалды әдіс болады.

Құрал	Қызметі
typeid	Тип туралы ақпарат береді
dynamic_cast	Полиморфты иерархиядағы төмен қарай түрлендіруді орындау кезінде тексереді
static_cast	Сандық және басқа рұқсат етілген айқын түрлендірулерді орындайды
const_cast	const немесе volatile біліктілігін өзгертеді
reinterpret_cast	Төменгі деңгейдегі түрлендірулерге арналған

Нұсқауышпен орындалатын тексерілетін dynamic_cast сәтсіз болса, нәтиже nullptr болады. Сілтememен орындалатын осындай түрлендіру сәтсіз болса, std::bad_cast тудырылады.

const_cast арқылы бастапқыда const болып құрылған объектіні өзгертуге болмайды: бұл анықталмаған әрекетке әкеледі.

Жұмыстың орындалу тәртібі

1. Shape базалық класын құру.
2. Одан Circle және Rectangle кластарын туындату.
3. Объектілерді базалық нұсқауыш арқылы қарастыру.
4. typeid арқылы нақты типтерін тексеру.
5. Сәтті және сәтсіз dynamic_cast орындау.
6. Қалған түрлендіру операторларының мысалдарын талдау.
7. Жеке тапсырманы орындап, есеп дайындау.

Программалық мысал

```
#include <iostream>
#include <memory>
#include <vector>
#include <typeinfo>

class Shape {
public:
   virtual ~Shape() = default;
   virtual double area() const = 0;
};

class Circle : public Shape {
private:
   double radius_;

public:
   explicit Circle(double radius) : radius_(radius) {}

   double area() const override {
      return 3.141592653589793 * radius_ * radius_;
   }

   double radius() const {
      return radius_;
   }
};

class Rectangle : public Shape {
private:
```

```
    double width_;
    double height_;

public:
    Rectangle(double width, double height)
        : width_(width), height_(height) {}

    double area() const override {
        return width_ * height_;
    }

    double width() const {
        return width_;
    }
};

int main() {
    std::vector<std::unique_ptr<Shape>> shapes;
    shapes.push_back(std::make_unique<Circle>(3.0));
    shapes.push_back(std::make_unique<Rectangle>(4.0, 5.0));

    for (const auto& owner : shapes) {
        Shape* shape = owner.get();

        // typeid арқылы нақты типті салыстыру
        if (typeid(*shape) == typeid(Circle)) {
            std::cout << "Tip: Circle\n";
        }
        else if (typeid(*shape) == typeid(Rectangle)) {
            std::cout << "Tip: Rectangle\n";
        }

        std::cout << "Audan: " << shape->area() << '\n';

        // Тексерілетін төмен қарай түрлендіру
        if (auto* circle = dynamic_cast<Circle*>(shape)) {
            std::cout << "Radius: " << circle->radius() << '\n';
        }
        else {
            std::cout << "Circle* turine turlendiru satsiz.\n";
        }

        if (auto* rectangle = dynamic_cast<Rectangle*>(shape)) {
            std::cout << "Eni: " << rectangle->width() << '\n';
        }

        std::cout << '\n';
    }

    // Сандық түрлендіру
    double number = 12.75;
    int integer = static_cast<int>(number);
```

```
    std::cout << "static_cast natizhesi: " << integer << '\n';

    // Бастапқы объект const емес, сондықтан оны өзгертуге болады
    int value = 10;
    const int* readOnly = &value;
    int* writable = const_cast<int*>(readOnly);
    *writable = 25;
    std::cout << "const_cast mysaly: " << value << '\n';

    // Объектінің байттық көрінісін unsigned char арқылы оқу
    const unsigned char* bytes =
        reinterpret_cast<const unsigned char*>(&value);

    std::cout << "Obekt baittary: ";
    for (std::size_t i = 0; i < sizeof(value); ++i) {
        std::cout << static_cast<unsigned int>(bytes[i]) << ' ';
    }
    std::cout << '\n';

    return 0;
}
```

Түсіндірме: std::unique_ptr объектілердің жадын автоматты босатады. Соңғы мысалда объект байттары тек оқылады; олардың реті мен мәндері платформаның деректерді ұсыну тәсіліне тәуелді.

Күтілетін нәтижелер

Әрекет	Нәтиже
Circle объектісінің типін тексеру	Circle анықталады
Circle объектісін Circle* типіне түрлендіру	Сәтті
Rectangle объектісін Circle* типіне түрлендіру	nullptr
Радиусы 3 шеңбердің ауданын есептеу	Шамамен 28.2743
Өлшемдері 4 × 5 тіктөртбұрыштың ауданын есептеу	20
12.75 мәнін int типіне түрлендіру	12
const емес объектіні түрлендірілген нұсқауыш арқылы өзгерту	value = 25

Жеке тапсырманың нұсқалары

Әр нұсқада виртуалды деструктор, кемінде бір виртуалды әдіс, typeid салыстыруы және сәтті әрі сәтсіз dynamic_cast болуы керек.

Нұсқа	Базалық класс	Туынды кластар	Арнайы әрекет
1	Person	Student, Teacher	Студент курсын немесе оқытушы пәнін шығару
2	Vehicle	Car, Bus	Орын санын немесе жүксалғыш көлемін шығару
3	Animal	Dog, Cat	Туынды класқа тән әдісті шақыру
4	Employee	Manager, Developer	Бөлімді немесе программалау тілін шығару
5	Account	SavingsAccount, CreditAccount	Пайыздық мөлшерлемені немесе кредит лимитін шығару
6	Product	Food, Electronics	Жарамдылық мерзімін немесе кепілдік мерзімін шығару

Нұсқа	Базалық класс	Туынды кластар	Арнайы әрекет
7	Document	TextDocument, ImageDocument	Сөз санын немесе кескін өлшемін шығару
8	Device	Printer, Scanner	Басып шығару немесе сканерлеу әдісін шақыру
9	Shape	Triangle, Square	Фигура ауданын және арнайы өлшемін шығару
10	Media	Audio, Video	Дыбыс ұзақтығын немесе бейне ажыратымдылығын шығару

Қосымша тапсырмалар:

1. dynamic_cast сілтеме нұсқасының сәтсіздігін std::bad_cast арқылы өңдеңіз.
2. Бір сандық static_cast мысалын қосыңыз.
3. typeid(*pointer).name() нәтижесін шығарып, оның жазылуы компиляторға тәуелді екенін түсіндіріңіз.

Бақылау сұрақтары

1. RTTI қандай мақсатта қолданылады?
2. Полиморфты класс дегеніміз не?
3. typeid полиморфты объектінің қандай типін анықтайды?
4. dynamic_cast нұсқауыш үшін сәтсіз болса, не қайтарылады?
5. Сілтеме үшін сәтсіз dynamic_cast қалай өңделеді?
6. static_cast пен dynamic_cast айырмашылығы қандай?
7. const_cast қолданғанда қандай шектеуді сақтау қажет?
8. reinterpret_cast неге мұқият қолданылуы керек?

№15 ЗЕРТХАНАЛЫҚ ЖҰМЫС

Стандартты кітапхана және жалпыланған программалау

Жұмыстың мақсаты: функция үлгілерін, STL контейнерлерін, итераторларды, алгоритмдерді және функционалдық объектілерді қолдану дағдыларын қалыптастыру.

Студенттерге қойылатын талап: деректерді контейнерде сақтау, стандартты алгоритмдермен өңдеу және әртүрлі типке қолданылатын үлгілік функция құру.

Әдістемелік нұсқаулар

STL құралдары контейнерлерден, итераторлардан және алгоритмдерден тұрады.

Құрал	Қызметі
std::vector	Өлшемі өзгеретін, элементтері үздіксіз орналасатын контейнер
std::set	Бірегей мәндерді салыстыру ретімен сақтайтын контейнер
std::map	Бірегей кілт–мән жұптарын сақтайтын контейнер

Құрал	Қызметі
begin(), end()	Элементтер диапазонын анықтайтын итераторларды береді
std::sort	Элементтерді сұрыптайды
std::find	Мәнге тең алғашқы элементті іздейді
std::count_if	Шартқа сәйкес элементтер санын есептейді
std::accumulate	Элементтерді бастапқы мәнге жинақтайды

Алгоритмдер жиі [begin, end) диапазонымен жұмыс істейді: бастапқы элемент кіреді, ал соңғы итератор көрсететін позиция кірмейді. end() итераторын дерек алу үшін қолдануға болмайды. Функционалдық объект — operator() анықталған кластың объектісі. bool қайтаратын шарттық функция немесе функционалдық объект предикат ретінде қолданылады.

Жұмыстың орындалу тәртібі

1. Үлгілер мен STL құралдарын қайталау.
2. Берілген программаны іске қосу.
3. Сұрыптау, іздеу, санау және жиынтық есептеу нәтижелерін тексеру.
4. Итератор арқылы элементтерді экранға шығару.
5. Үлгілік функцияны int және double типтерімен қолдану.
6. Жеке тапсырманы орындау.
7. Есеп дайындау.

Программалық мысал

Есеп: бүтін сандарды сұрыптау, олардың қосындысын және оң элементтер санын есептеу, берілген мәнді іздеу және қайталанбайтын элементтерді шығару.

```
#include <iostream>
#include <vector>
#include <set>
#include <map>
#include <algorithm>
#include <numeric>
#include <string>

template<typename T>
void printContainer(const std::vector<T>& values) {
   for (auto it = values.begin(); it != values.end(); ++it) {
      std::cout << *it << ' ';
   }
   std::cout << '\n';
}

template<typename T>
T maxValue(const T& a, const T& b) {
   return (a < b) ? b : a;
}

// Функционалдық объект — предикат
struct IsPositive {
   bool operator()(int value) const {
      return value > 0;
   }
};

int main() {
   std::vector<int> values{5, -2, 8, 5, 0, 3, -1};
```

```
std::cout << "Bastapqy derekter: ";
printContainer(values);

std::sort(values.begin(), values.end());

std::cout << "Suryptalgan derekter: ";
printContainer(values);

long long sum =
    std::accumulate(values.begin(), values.end(), 0LL);

auto positiveCount =
    std::count_if(values.begin(), values.end(), IsPositive{});

std::cout << "Qosyndy: " << sum << '\n';
std::cout << "On elementter sany: " << positiveCount << '\n';

int target = 8;
auto found = std::find(values.begin(), values.end(), target);

if (found != values.end()) {
    std::cout << "Izdelgen element tabyldy: " << *found << '\n';
}
else {
    std::cout << "Element tabylmady.\n";
}

std::set<int> uniqueValues(values.begin(), values.end());

std::cout << "Biregei elementter: ";
for (int value : uniqueValues) {
    std::cout << value << ' ';
}
std::cout << '\n';

std::map<std::string, int> scores{
    {"Ali", 85},
    {"Dana", 92},
    {"Miras", 78}
};

std::cout << "Student bagalary:\n";
for (const auto& entry : scores) {
    std::cout << entry.first << ": " << entry.second << '\n';
}

std::cout << "int maksimumy: "
          << maxValue(10, 25) << '\n';

std::cout << "double maksimumy: "
          << maxValue(4.5, 3.2) << '\n';
```

```
    return 0;
}
```

Күтілетін нәтижелер

Операция	**Нәтиже**
Өсу ретімен сұрыптау	-2 -1 0 3 5 5 8
Элементтер қосындысы	18
Оң элементтер саны	4
8 мәнін іздеу	Элемент табылады
Бірегей элементтер	-2 -1 0 3 5 8
maxValue(10, 25)	25
maxValue(4.5, 3.2)	4.5

Жеке тапсырманың нұсқалары

Әр нұсқада контейнер, итератор, кемінде екі стандартты алгоритм және бір үлгілік функция қолданыңыз.

Нұсқа	**Тапсырма**
1	Студент бағаларын vector ішінде сақтаңыз. Сұрыптап, орташа бағаны және 50-ден төмен бағалар санын есептеңіз.
2	Тауар атауы мен бағасын map ішінде сақтаңыз. Атауы бойынша іздеп, ең қымбат тауарды анықтаңыз.
3	Бүтін сандардан тұратын векторды сұрыптаңыз. Жұп элементтер санын және қосындысын есептеңіз.
4	Сөздерді vector<string> ішінде сақтаңыз. Әліпби ретімен сұрыптап, берілген сөзді іздеңіз.
5	Қайталанатын сандардан set құрыңыз. Бірегей мәндерді шығарып, олардың санын бастапқы элементтер санымен салыстырыңыз.
6	Бір айдағы температураларды сақтаңыз. Ең төмен, ең жоғары және орташа температураны есептеңіз.
7	Қызметкерлердің аты мен жалақысын сақтаңыз. Жалақы бойынша сұрыптап, берілген шектен жоғары жалақы алатындарды санаңыз.
8	Кітап атауларын сақтаңыз. Ұзындығы бойынша сұрыптап, атауы берілген кітапты іздеңіз.
9	Сандық вектордан теріс элементтерді remove_if және erase арқылы жойыңыз. Қалған элементтердің қосындысын есептеңіз.
10	Сөздер тізіміндегі әр сөздің кездесу санын map<string, int> арқылы есептеңіз. Ең жиі кездесетін сөзді анықтаңыз.

Қосымша тапсырмалар:

1. Сұрыптауды кему ретімен орындаңыз.
2. Өз предикатыңызды функционалдық объект түрінде жазыңыз.
3. Бос контейнермен жұмыс істеу жағдайын тексеріңіз. Орташа мән есептеу немесе табылған элементті оқу алдында қажетті шарттарды тексеріңіз.

Бақылау сұрақтары

1. Жалпыланған программалаудың негізгі идеясы қандай?
2. STL қандай құрамдастардан тұрады?
3. vector, set, map айырмашылығы қандай?
4. Итератор қандай қызмет атқарады?
5. Неліктен end() итераторын разымендеуге болмайды?

6. std::sort қандай итераторларды талап етеді?
7. Предикат пен функционалдық объект дегеніміз не?
8. std::accumulate бастапқы мәнінің типі неге маңызды?

3. БІЛІМДІ БАҚЫЛАУ МАТЕРИАЛДАРЫ

C++ тілінің негіздері және деректер типтері

1. C++ программасының орындалуы қай функциядан басталады?
A) start()
B) main()
C) begin()
D) run()

2. Бүтін сандарды сақтауға арналған типті көрсетіңіз.
A) double
B) char
C) int
D) bool

3. Логикалық мәндерді сақтайтын тип қандай?
A) bool
B) float
C) char
D) void

4. sizeof операторы нені анықтайды?
A) Айнымалының адресін
B) Функцияның орындалу уақытын
C) Айнымалының мәнін
D) Типтің немесе объектінің байтпен өлшемін

5. const int n = 10; жазбасындағы const нені білдіреді?
A) Айнымалы міндетті түрде глобалды
B) Объектінің мәнін кейін өзгертуге болмайды
C) Айнымалы нұсқауыш болады
D) Объект динамикалық жадыда орналасады

6. Бір жолдық түсіндірме қандай белгімен басталады?
A) //
B) ##
C) <!--
D) **

7. Стандартты консольдік енгізу-шығару үшін қандай тақырыптық файл қажет?
A) <vector>
B) <string>
C) <iostream>
D) <typeinfo>

8. std::cin >> x; операторының қызметі қандай?
A) x мәнін файлға жазады
B) x айнымалысын жояды
C) x мәнін экранға шығарады
D) Стандартты енгізуден x мәнін оқиды

Операторлар және басқару құрылымдары

9. 17 % 5 өрнегінің нәтижесі қандай?
A) 3
B) 2
C) 5
D) 0

10. Биттік ЖӘНЕ операциясының белгісі қандай?
A) &&
B) ||
C) &
D) !
11. unsigned int x = 3; болса, x << 1 нәтижесі қандай?
A) 6
B) 3
C) 1
D) 4
12. Биттік айрықша НЕМЕСЕ операциясының белгісі қандай?
A) |
B) ~
C) &
D) ^
13. Логикалық ЖӘНЕ операциясының белгісі қандай?
A) &
B) &&
C) |
D) ^
14. Екі мәннің теңдігін тексеретін оператор қандай?
A) ==
B) =
C) !=
D) <=
15. Қай циклдің денесі кемінде бір рет орындалады?
A) for
B) while
C) do ... while
D) Ешбір цикл
16. Цикл ішіндегі break операторы не істейді?
A) Циклді басынан бастайды
B) Программаны міндетті түрде аяқтайды
C) Тек ағымдағы қадамды өткізіп жібереді
D) Өзі орналасқан ең жақын циклден шығады
Массивтер, жолдар және функциялар
17. int a[5]; массивінің дұрыс индекстері қандай?
A) 1–5
B) 0–4
C) 0–5
D) 1–4
18. int a[]{2, 4, 6}; болса, a[1] мәні қандай?
A) 2
B) 6
C) 4
D) 1
19. Нөлдік таңбамен аяқталатын С стиліндегі жолдың соңғы таңбасы қандай?
A) '\0'
B) '\n'

C) '0'
D) '\t'

20. Функцияның мән қайтармайтынын қандай тип білдіреді?
A) int
B) bool
C) char
D) void

21. Параметрді мән бойынша бергенде функция не алады?
A) Міндетті түрде бастапқы объектінің адресін
B) Аргумент мәнінің көшірмесін
C) Айнымалының атауын
D) Файл ағынын

22. Рекурсивті функция дегеніміз не?
A) Өзін тікелей немесе жанама шақыратын функция
B) Міндетті түрде void қайтаратын функция
C) Параметрсіз кез келген функция
D) Тек класс ішінде анықталатын функция

23. Функцияларды қайта жүктеудің дұрыс шарты қандай?
A) Тек қайтарылатын типтері әртүрлі болуы
B) Атаулары міндетті түрде әртүрлі болуы
C) Атаулары бірдей, параметрлер тізімдері әртүрлі болуы
D) Денелері міндетті түрде бірдей болуы

24. Функция прототипі нені сипаттайды?
A) Функцияның орындалу нәтижесін
B) Функция денесін ғана
C) Программаның жады көлемін
D) Функцияның атауын, қайтарылатын типін және параметрлерін

Нұсқауыштар, сілтемелер және динамикалық жады

25. Нұсқауыш қандай ақпаратты сақтай алады?
A) Тек файл атауын
B) Объектінің жадыдағы адресін
C) Тек логикалық мәнді
D) Кластың бастапқы кодын

26. int x = 7; айнымалысының адресін алу өрнегі қандай?
A) *x
B) x&
C) &x
D) address x

27. int* p = &x; болса, *p нені білдіреді?
A) x объектісіне қатынауды
B) Нұсқауыштың атауын
C) Нұсқауыштың өлшемін
D) Кластың типін

28. Ешбір объектіні көрсетпейтін нұсқауышты белгілеуге арналған түйінді сөз қандай?
A) void
B) empty
C) nullpointer
D) nullptr

29. Сілтемені дұрыс жариялау нұсқасын көрсетіңіз.
A) int r = &x;
B) int& r = x;
C) int* r = x;
D) reference r = x;
30. int x = 4; int& r = x; r = 12; орындалғаннан кейін x мәні қандай?
A) 12
B) 4
C) 0
D) 8
31. new int(5) өрнегі не істейді?
A) Бар айнымалыны жояды
B) Бес элементті массив құрады
C) Динамикалық жадыда мәні 5 болатын int объектісін құрады
D) Файлға 5 санын жазады
32. new int[10] арқылы бөлінген массивті қалай босату керек?
A) delete p;
B) free(p);
C) remove(p);
D) delete[] p;
33. Жадының ағып кетуі қандай жағдайда туындайды?
A) Айнымалыға жаңа мән берілгенде
B) Динамикалық бөлінген жады босатылмай, оған қолжетімділік жоғалғанда
C) Консольге дерек шығарылғанда
D) Функция параметр қабылдағанда
34. Сілтеменің нұсқауыштан негізгі айырмашылығы қандай?
A) Инициализацияланған сілтемені басқа объектімен қайта байланыстыруға болмайды
B) Сілтеме тек бүтін санмен жұмыс істейді
C) Сілтеме міндетті түрде nullptr болады
D) Сілтемені әр қолданғанда * белгісі қажет
Кластар және объектілер
35. Класс дегеніміз не?
A) Тек қайталану операторы
B) Тек файлдың атауы
C) Деректер мен әдістерді біріктіретін пайдаланушы анықтайтын тип
D) Компиляция қатесі
36. Объект дегеніміз не?
A) Кез келген тақырыптық файл
B) Функцияның прототипі
C) Атаулар кеңістігі
D) Кластың нақты данасы
37. Инкапсуляцияның мәні қандай?
A) Барлық деректерді глобалды ету
B) Деректер мен әдістерді біріктіріп, қолжетімділікті реттеу
C) Функцияларды жою
D) Программаны бірнеше рет компиляциялау
38. class арқылы анықталған класта мүшелердің әдепкі қолжетімділігі қандай?
A) private
B) public

C) protected
D) global

39. public мүшелеріне кімдер қатынай алады?
A) Тек кластың конструкторы
B) Тек деструктор
C) Объектіге қолжетімді сыртқы код
D) Тек достық функция

40. this нұсқауышы нені көрсетеді?
A) Программаның бірінші объектісін
B) Базалық кластың атауын
C) Соңғы ашылған файлды
D) Әдісі орындалып жатқан ағымдағы объектіні

Конструкторлар және деструкторлар

41. Конструктордың негізгі қызметі қандай?
A) Класс атауын өзгерту
B) Объектіні бастапқы күйге келтіру
C) Файлды сұрыптау
D) Функцияны жою

42. Конструктор атауы қандай болады?
A) Міндетті түрде create
B) Міндетті түрде main
C) Класс атауымен бірдей
D) Кез келген басқа атау

43. Student класының деструкторын дұрыс көрсетіңіз.
A) ~Student()
B) delete Student()
C) Student~()
D) void destroy Student()

44. Конструктордың қайтарылатын типі қандай?
A) Міндетті түрде void
B) Міндетті түрде int
C) Міндетті түрде класс типі
D) Қайтарылатын типі көрсетілмейді

45. Параметрсіз шақыруға болатын конструктор қалай аталады?
A) Көшіру конструкторы
B) Әдепкі конструктор
C) Виртуалды конструктор
D) Достық конструктор

46. Мүшелерді инициализациялау тізімі қай жерде жазылады?
A) Конструктор параметрлерінен кейін, : белгісінен соң
B) Деструктор атауының алдында
C) namespace ішінде ғана
D) catch блогынан кейін

Достық функциялар және операторларды қайта жүктеу

47. Достық функцияны жариялайтын түйінді сөз қандай?
A) virtual
B) public
C) friend
D) override

48. Достық функцияның ерекшелігі қандай?
A) Ол міндетті түрде класс мүшесі болады
B) Ол тек бір объектіні жояды
C) Ол тек базалық класта анықталады
D) Ол класс мүшесі болмаса да, жабық мүшелерге қатынай алады
49. Көшіру конструкторының қалыпты жариялануын көрсетіңіз.
A) void X(X);
B) X(const X&);
C) ~X(const X&);
D) copy X(X);
50. Қосу операторын қайта жүктеу үшін қандай атау қолданылады?
A) operator+
B) plusOperator
C) add+
D) overload+
51. Операторды қайта жүктегенде нені өзгертуге болмайды?
A) Пайдаланушы типімен орындалатын әрекетті
B) Оператор функциясының денесін
C) Операторлың басымдығын
D) Нәтижені есептеу тәсілін
52. Қай операторды қайта жүктеуге болмайды?
A) +
B) ==
C) []
D) ::
Функция және класс үлгілері
53. Үлгілердің негізгі мақсаты қандай?
A) Файлдарды автоматты жабу
B) Әртүрлі типтерге ортақ код құру
C) Барлық айнымалыны тұрақты ету
D) Барлық әдісті виртуалды ету
54. Тип параметрі бар үлгінің дұрыс басталуын көрсетіңіз.
A) class template T
B) typename<T>
C) template<typename T>
D) generic class T
55. template<typename T> жазбасындағы T нені білдіреді?
A) Тип параметрін
B) Файл режимін
C) Объектінің адресін
D) Циклдің шартын
56. Box класс үлгісін int типімен қолданудың дұрыс жазылуы қандай?
A) Box = int;
B) Box(int) b;
C) template Box b;
D) Box<int> b;
57. Үлгіні белгілі бір типке арнайы бейімдеу қалай аталады?
A) Мұрагерлік
B) Мамандандыру

C) Разымендеу
D) Инкапсуляция
58. Типке жатпайтын үлгі параметрінің мысалын көрсетіңіз.
A) int N
B) typename T
C) class T
D) typename U
Мұрагерлік және кластар иерархиясы
59. Ашық мұрагерліктің дұрыс жазылуы қандай?
A) class D extends B {};
B) class D = B {};
C) class D : public B {};
D) class D -> B {};
60. Базалық класс дегеніміз не?
A) Тек main() бар класс
B) Тек объектісі жоқ класс
C) Тек файлмен жұмыс істейтін класс
D) Басқа класс мұра алатын класс
61. protected мүшелеріне қай код тікелей қатынай алады?
A) Программадағы барлық функция
B) Кластың және оның туынды кластарының әдістері
C) Тек глобалды айнымалы
D) Тек стандартты кітапхана
62. Туынды объектіні құруда конструкторлар қандай ретпен орындалады?
A) Алдымен базалық, кейін туынды
B) Алдымен туынды, кейін базалық
C) Тек туынды конструктор
D) Кездейсоқ ретпен
63. Туынды объект жойылғанда деструкторлар қандай ретпен орындалады?
A) Алдымен базалық, кейін туынды
B) Тек базалық деструктор
C) Алдымен туынды, кейін базалық
D) Екеуі бір мезгілде орындалады
64. Бірнеше базалық кластан мұра алу қалай аталады?
A) Абстракция
B) Функцияны қайта жүктеу
C) Инкапсуляция
D) Көптік мұрагерлік
65. Виртуалды мұрагерліктің негізгі мақсаты қандай?
A) Барлық әдісті виртуалды ету
B) Ортақ виртуалды базалық класс бөлігінің бір данасын қамтамасыз ету
C) Конструкторларды жою
D) Барлық мүшені жабық ету
66. Базалық класс мүшесін нақтылап көрсету үшін қандай оператор қолданылады?
A) ::
B) &&
C) %
D) !
Полиморфизм және абстрактылы кластар

67. Виртуалды әдісті жариялайтын түйінді сөз қандай?
A) static
B) friend
C) virtual
D) template
68. Динамикалық полиморфизмде әдіс нұсқасы неге сәйкес таңдалады?
A) Файл атауына
B) Айнымалы атауының ұзындығына
C) Объектінің өлшеміне ғана
D) Объектінің нақты типіне
69. override белгілеуінің қызметі қандай?
A) Әдісті достық функция ету
B) Базалық виртуалды әдістің дұрыс қайта анықталғанын тексерту
C) Объектіні автоматты жою
D) Әдісті статикалық ету
70. Таза виртуалды әдістің дұрыс жазылуын көрсетіңіз.
A) virtual void draw() = 0;
B) void draw() = virtual;
C) virtual draw = 0;
D) void virtual draw = null;
71. Абстрактылы кластың объектісін тікелей құруға бола ма?
A) Иә, барлық жағдайда
B) Иә, тек массив түрінде
C) Жоқ
D) Иә, тек new арқылы
72. Туынды объектіні базалық нұсқауыш арқылы дұрыс жою үшін базалық класта не қажет?
A) Статикалық конструктор
B) Достық деструктор
C) Параметрлі деструктор
D) Виртуалды деструктор
Көріну аймақтары және атаулар кеңістіктері
73. Көріну аймағы дегеніміз не?
A) Объектінің байттық өлшемі
B) Идентификатор атауын қолдануға болатын программа бөлігі
C) Файлдың дискідегі көлемі
D) Программаның орындалу уақыты
74. Блок қандай таңбалармен шектеледі?
A) ( )
B) [ ]
C) { }
D) < >
75. Атаулар кеңістігін анықтайтын түйінді сөз қандай?
A) namespace
B) space
C) named
D) region
76. std::cout жазбасындағы std нені білдіреді?
A) Конструктор атауын
B) Файл режимін

C) Деректер типін
D) Стандартты кітапхананың атаулар кеңістігін

Файлдық енгізу-шығару

77. Файлдан оқу үшін қандай класс қолданылады?
A) std::ofstream
B) std::ifstream
C) std::vector
D) std::exception

78. Файлға жазу үшін қандай класс қолданылады?
A) std::ofstream
B) std::ifstream
C) std::istream
D) std::set

79. Файлды оқуға да, жазуға да арналған класс қандай?
A) std::string
B) std::ostream
C) std::fstream
D) std::map

80. std::ios::app режимі нені білдіреді?
A) Файлды тек оқу
B) Файлды міндетті түрде жою
C) Файлды тек екілік форматта ашу
D) Әр жазуды файл соңына қосу

81. Файл ағынының ашылғанын қандай әдіспен тексеруге болады?
A) size()
B) is_open()
C) begin()
D) push_back()

82. Бос орындары бар толық жолды оқуға қандай функция қолайлы?
A) std::sort()
B) std::find()
C) std::getline()
D) std::swap()

Ерекше жағдайларды өңдеу

83. Ерекше жағдайларды өңдеудің негізгі түйінді сөздері қандай?
A) try, catch, throw
B) for, while, do
C) public, private, protected
D) new, delete, sizeof

84. throw операторының қызметі қандай?
A) Файлды жабу
B) Циклді қайталау
C) Типті анықтау
D) Ерекше жағдайды тудыру

85. catch блогы қандай қызмет атқарады?
A) Объектінің адресін анықтайды
B) Сәйкес ерекше жағдайды ұстап, өңдейді
C) Жады бөледі
D) Класс үлгісін құрады

86. catch (...) нені білдіреді?
A) Осы өңдегішке жеткен кез келген типтегі ерекше жағдайды ұстайды
B) Тек бүтін санды ұстайды
C) Тек файл қатесін ұстайды
D) Ерекше жағдай тудырады
87. std::exception::what() әдісі не қайтарады?
A) Қате болған жолдың нөмірін міндетті түрде
B) Объектінің өлшемін
C) Ерекше жағдайды сипаттайтын мәтіндік хабарламаны
D) Файл адресін
88. Индекс шектен шыққанда ерекше жағдай тудыратын әдіс қандай?
A) vector::size()
B) vector::begin()
C) vector::operator[]
D) vector::at()
Динамикалық типті анықтау және түрлендіру
89. typeid операторының қызметі қандай?
A) Жады босату
B) Тип туралы ақпарат алу
C) Файлды сұрыптау
D) Объектіні көшіру
90. Полиморфты иерархияда тексерілетін төмен қарай түрлендіруге қай оператор арналған?
A) static_cast
B) const_cast
C) dynamic_cast
D) reinterpret_cast
91. Нұсқауышпен орындалған тексерілетін dynamic_cast сәтсіз болса, не алынады?
A) nullptr
B) Міндетті түрде 1
C) Объектінің көшірмесі
D) Кездейсоқ адрес
92. static_cast<int>(7.9) нәтижесі қандай?
A) 8
B) 7.9
C) 0
D) 7
93. const_cast операторы не үшін қолданылады?
A) Файлды екілік режимде ашу үшін
B) const немесе volatile біліктілігін өзгерту үшін
C) Виртуалды әдісті шақыру үшін
D) Массивті сұрыптау үшін
94. reinterpret_cast қандай мақсатта қолданылады?
A) Төменгі деңгейдегі айқын тип түрлендірулерін орындау үшін
B) Әрбір объектіні қауіпсіз көшіру үшін
C) Ерекше жағдайларды ұстау үшін
D) Функция үлгісін жариялау үшін
Стандартты кітапхана және жалпыланған бағдарламалау
95. STL-дің негізгі құрамдастары қандай?
A) Серверлер, браузерлер, протоколдар

B) Кестелер, формалар, сұраныстар
C) Контейнерлер, алгоритмдер, итераторлар
D) Файлдар, каталогтар, дискілер

96. std::vector контейнерінің негізгі қасиеті қандай?
A) Тек бір элемент сақтайды
B) Тек кілт арқылы қатынауға мүмкіндік береді
C) Өлшемін өзгертуге болмайды
D) Элементтерді үздіксіз орналастырып, өлшемін өзгерте алады

97. std::map қандай деректерді сақтайды?
A) Тек символдарды
B) Бірегей кілт–мән жұптарын
C) Тек нұсқауыштарды
D) Тек қайталанатын кілттерді

98. end() итераторы нені білдіреді?
A) Соңғы элементтен кейінгі позицияны
B) Бірінші элементті
C) Соңғы элементтің мәнін
D) Контейнердің өлшемін

99. std::sort қандай итераторларды талап етеді?
A) Тек енгізу итераторларын
B) Тек шығару итераторларын
C) Еркін қатынау итераторларын
D) Тек екі бағытты итераторларды

100. Функционалдық объект дегеніміз не?
A) Кез келген глобалды айнымалы
B) Файл ағыны ғана
C) Тек абстрактылы кластың нұсқауышы
D) operator() анықталған кластың объектісі

Жауап кілті

№	Жауап	№	Жауап	№	Жауап	№	Жауап
1	B	26	C	51	C	76	D
2	C	27	A	52	D	77	B
3	A	28	D	53	B	78	A
4	D	29	B	54	C	79	C
5	B	30	A	55	A	80	D
6	A	31	C	56	D	81	B
7	C	32	D	57	B	82	C
8	D	33	B	58	A	83	A
9	B	34	A	59	C	84	D
10	C	35	C	60	D	85	B
11	A	36	D	61	B	86	A
12	D	37	B	62	A	87	C
13	B	38	A	63	C	88	D
14	A	39	C	64	D	89	B
15	C	40	D	65	B	90	C
16	D	41	B	66	A	91	A
17	B	42	C	67	C	92	D

№	Жауап	№	Жауап	№	Жауап	№	Жауап
18	C	43	A	68	D	93	B
19	A	44	D	69	B	94	A
20	D	45	B	70	A	95	C
21	B	46	A	71	C	96	D
22	A	47	C	72	D	97	B
23	C	48	D	73	B	98	A
24	D	49	B	74	C	99	C
25	B	50	A	75	A	100	D

ҚОРЫТЫНДЫ

«C++ тілінде машинаға бағытталған бағдарламалау» электрондық оқулық C++ тілінің негізгі мүмкіндіктері, деректермен жұмыс істеу тәсілдері және бағдарламаларды құрылымдау қағидалары қарастырылды. Теориялық материалдар, бағдарламалық мысалдар, зертханалық жұмыстар және тест тапсырмалары білім алушының алған білімін практикада қолдануына бағытталған.

Машинаға бағытталған бағдарламалауда деректердің типін, жадыдағы орналасуын және оларға қатынау тәсілдерін түсіну маңызды. Сілтемелер мен нұсқауыштарды дұрыс қолдану, объектілердің өмірлік циклін ескеру және ресурстарды уақытында босату бағдарламаның сенімділігіне әсер етеді. Сондықтан бағдарлама құру барысында есептеу нәтижесімен бірге жадыны пайдалану дұрыстығына да назар аудару қажет.

Оқулықта қарастырылған кластар мен объектілер деректерді және оларды өңдейтін әрекеттерді бір құрылымға біріктіруге мүмкіндік береді. Инкапсуляция ішкі деректерге қолжетімділікті реттейді, мұрагерлік ортақ қасиеттерді қайта пайдалануға жағдай жасайды, ал полиморфизм әртүрлі объектілерді ортақ интерфейс арқылы басқаруға көмектеседі. Бұл құралдар бағдарламаның құрылымын түсінікті етіп, оны кеңейту мен сүйемелдеуді жеңілдетеді.

Файлдық енгізу-шығару тәсілдері деректерді сақтау мен қайта өңдеуге, ерекше жағдайларды өңдеу механизмі қате жағдайларға жауап беруге мүмкіндік береді. Типтерді анықтау және түрлендіру құралдарын қолдану олардың талаптары мен шектеулерін түсінуді қажет етеді. Үлгілер, контейнерлер, итераторлар және стандартты алгоритмдер әртүрлі деректерге ортақ шешімдер әзірлеуге көмектеседі.

Зертханалық жұмыстарды орындау барысында білім алушы алгоритм құру, бағдарламалық код жазу, бастапқы деректерді енгізу және нәтижелерді талдау дағдыларын дамытады. Қалыпты, шекаралық және қате деректермен тексеру бағдарламаның әрекетін бағалауға мүмкіндік береді. Бақылау сұрақтары мен тест тапсырмалары теориялық білімді бекітуге қызмет етеді.

Электрондық оқулық материалдарын меңгеру C++ тілін әрі қарай тереңдетіп оқуға, жүйелік бағдарламалау мен күрделі деректер құрылымдарын зерттеуге негіз болады. Алынған білімді шағын жобаларда тұрақты қолдану, кодты талдау және жетілдіру білім алушының бағдарламалау құзыретін дамытуға мүмкіндік береді.

ПАЙДАЛАНЫЛҒАН ӘДЕБИЕТТЕР ТІЗІМІ

Негізгі әдебиеттер

1. Stroustrup, B. The C++ Programming Language. 4th ed. Addison-Wesley, 2013.
2. Stroustrup, B. Programming: Principles and Practice Using C++. 2nd ed. Addison-Wesley, 2014.
3. Lippman, S. B., Lajoie, J., Moo, B. E. C++ Primer. 5th ed. Addison-Wesley, 2012.
4. Deitel, P., Deitel, H. C++ How to Program. 10th ed. Pearson, 2017.
5. Savitch, W. Problem Solving with C++. 10th ed. Pearson, 2017.
6. Malik, D. S. C++ Programming: Program Design Including Data Structures. 8th ed. Cengage Learning, 2018.
7. Gaddis, T. Starting Out with C++: From Control Structures through Objects. 9th ed. Pearson, 2017.
8. Horton, I., Van Weert, P. Beginning C++20: From Novice to Professional. 6th ed. Apress, 2020.
9. Gregoire, M. Professional C++. 5th ed. Wiley, 2021.
10. Meyers, S. Effective Modern C++. O'Reilly Media, 2014.
11. Hennessy, J. L., Patterson, D. A. Computer Architecture: A Quantitative Approach. 6th ed. Morgan Kaufmann, 2019.
12. Null, L., Lobur, J. The Essentials of Computer Organization and Architecture. 5th ed. Jones & Bartlett Learning, 2018.
13. Clements, A. The Principles of Computer Hardware. 4th ed. Oxford University Press, 2006.
14. Patterson, D. A., Hennessy, J. L. Computer Organization and Design RISC-V Edition. 2nd ed. Morgan Kaufmann, 2020.
15. Harris, D. M., Harris, S. L. Digital Design and Computer Architecture. 2nd ed. Morgan Kaufmann, 2012.
16. Tanembaum, A. S., Austin, T. Structured Computer Organization. 6th ed. Pearson, 2012.
17. Carter, N. Schaum's Outline of Computer Architecture. McGraw-Hill, 2001.

Машинаға бағытталған және төмен деңгейлі бағдарламалау

11. Bryant, R. E., O'Hallaron, D. R. Computer Systems: A Programmer's Perspective. 3rd ed. Pearson, 2015.
12. Patt, Y. N., Patel, S. J. Introduction to Computing Systems: From Bits and Gates to C/C++ and Beyond. 3rd ed., 2020.
13. Irvine, K. R. Assembly Language for x86 Processors. 7th ed. Pearson, 2015.
14. Duntemann, J. Assembly Language Step-by-Step. 3rd ed. Wiley, 2009.
15. Hyde, R. The Art of Assembly Language. 2nd ed. No Starch Press, 2010.
16. Detmer, R. C. Introduction to 80X86 Assembly Language and Computer Architecture. 2nd ed. Jones & Bartlett Learning, 2001.
17. Mano, M. M. Computer System Architecture. 3rd ed. Pearson, 1992.
18. Stallings, W. Computer Organization and Architecture: Designing for Performance. 11th ed. Pearson, 2019.
19. Patterson, D. A., Hennessy, J. L. Computer Organization and Design: The Hardware/Software Interface. 6th ed. Morgan Kaufmann, 2020.

20. Nisan, N., Schocken, S. The Elements of Computing Systems: Building a Modern Computer from First Principles. 2nd ed. MIT Press, 2021.

Қосымша әдебиеттер

21. Kernighan, B. W., Ritchie, D. M. The C Programming Language. 2nd ed. Prentice Hall, 1988.
22. Gorman, M. Understanding the Linux Virtual Memory Manager. Prentice Hall, 2004.
23. Love, R. Linux System Programming: Talking Directly to the Kernel and C Library. 2nd ed. O'Reilly Media, 2013.
24. Kerrisk, M. The Linux Programming Interface. No Starch Press, 2010.
25. Stevens, W. R., Rago, S. A. Advanced Programming in the UNIX Environment. 3rd ed. Addison-Wesley, 2013.
26. Cox, K. The Intel Microprocessors. 8th ed. Pearson, 2009.
27. Hall, D. V. Microprocessors and Interfacing: Programming and Hardware. 2nd ed. McGraw-Hill, 1992.
28. Barry, B. Understanding the Linux Kernel. 3rd ed. O'Reilly Media, 2005.
29. Tanenbaum, A. S., Bos, H. Modern Operating Systems. 4th ed. Pearson, 2015.
30. Meyers, S. Effective C++: 55 Specific Ways to Improve Your Programs and Designs. 3rd ed. Addison-Wesley, 2005.
31. Williams, A. C++ Concurrency in Action. 2nd ed. Manning Publications, 2019.
32. Lakos, J. Large-Scale C++ Software Design. Addison-Wesley, 1996.
33. Gamma, E., Helm, R., Johnson, R., Vlissides, J. Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley, 1994.

Электрондық оқу ресурстары

- ISO C++ — ресми C++ ресурстары
- C++ Reference — тіл және стандартты кітапхана анықтамалығы
- MIT OpenCourseWare — C/C++ бағдарламалау
- Computer Architecture and Assembly Language Programming әдебиеттері